FURNISTORE · PROGRAMACIÓN WEB

Guía de la API de Furnistore

.NET 10, ASP.NET Core, EF Core y PostgreSQL explicados sobre el código real de la API, con los diagramas de la base de datos.

Universidad Tecnológica de La Habana «José Antonio Echeverría» (CUJAE)
Lian Carlos Gené Galvez · Eriet Dario Armas González · Darell Perdomo González
Octubre de 2026

Índice

Parte I. Fundamentos de .NET, con Furnistore como ejemplo 4

1. Mapa general 4
2. Anatomía de una solución .NET 5
3. Program.cs paso a paso 9
4. Inyección de dependencias 13
5. Configuración 15

Parte II. Arquitectura 18

6. Las capas y por qué existen 18
7. Recorrido completo de una petición: el checkout 21
8. Patrones transversales 24
9. Logging 29

Parte III. Base de datos 31

10. Modelo de datos completo 31
11. EF Core desde cero 41
12. Conexión a PostgreSQL 45
13. Migraciones 48
14. Transacciones y concurrencia 52

Parte IV. Seguridad 58

15. ASP.NET Core Identity 58
16. JWT y refresh tokens 61
17. Autorización, límites de peticiones y CORS 64
18. Verificación de correo 68

Parte V. Referencia del dominio 71

19. Controladores y endpoints 71
20. Servicios de aplicación 73
21. Catálogo de errores 76
22. Integraciones externas 79

Parte VI. Calidad y operación 81

23. Pruebas 81
24. Contrato OpenAPI 84
25. Recetas para modificar el proyecto 85
26. Deuda conocida y glosario 86

Esta guía explica la API de Furnistore (`apps/api`) de punta a punta: qué es cada pieza, cómo encaja con las demás y por qué se construyó así. Está pensada para alguien que sabe programar pero no conoce .NET, y quiere llegar a poder modificar este proyecto, o cualquier otro proyecto .NET existente, con criterio.

Cómo leerla. Casi todos los apartados siguen el mismo esquema:

- **Concepto:** qué es la pieza en .NET y qué problema resuelve, independientemente de Furnistore.
- **En Furnistore:** dónde está en el código, con fragmentos reales.
- **Por qué así:** qué alternativa se descartó y en qué situación convendría cambiar la decisión.

Las rutas de archivo son relativas a `apps/api/`. Los fragmentos de código se copiaron del repositorio; a veces se recortan con `…` para no repetir lo que no aporta.

Parte I. Fundamentos de .NET, con Furnistore como ejemplo

1. Mapa general

1.1 Qué es la API

La API es una aplicación **ASP.NET Core Web API** escrita en **C#** sobre **.NET 10**. Su trabajo es recibir peticiones HTTP, aplicar las reglas de negocio de la tienda, leer y escribir en una base de datos **PostgreSQL** y responder en **JSON**. No genera HTML: la interfaz la pinta la aplicación web (`apps/web`, Next.js), que es el único cliente de la API.

Tres nombres que conviene no confundir:

Nombre	Qué es
.NET	La plataforma: el *runtime* que ejecuta el código, la biblioteca estándar y las herramientas (`dotnet`).
C#	El lenguaje en que está escrito el código.
ASP.NET Core	El framework web de .NET: servidor HTTP (Kestrel), enrutado, controladores, autenticación, etc.

1.2 Qué expone al final

Cuando la API está en marcha ofrece esto:

Qué	Dónde	Para qué
54 endpoints REST	`http://localhost:5135/api/...`	Todo lo que usa la web: catálogo, auth, carrito, pedidos, panel de administración.
Swagger UI	`http://localhost:5135/swagger`	Página interactiva para probar los endpoints. **Solo en entorno Development.**
Health check	`http://localhost:5135/health`	Responde `Healthy` si la API está viva y llega a la base de datos.
Contrato OpenAPI	`apps/api/openapi.json` (archivo en el repo)	Descripción formal de todos los endpoints; de él se generan los tipos TypeScript de la web.
Comando de consola	`dotnet run ... -- admin promote <correo>`	Dar el rol de administrador a una cuenta sin pasar por HTTP.

Los endpoints se agrupan así (el detalle está en la Parte V):

Prefijo	Quién puede llamarlo	Qué hace
`/api/authentication`	Cualquiera	Registro, login, verificación de correo, renovación de sesión, logout.
`/api/products`, `/api/product-categories`	Cualquiera	Catálogo público.

Prefijo	Quién puede llamarlo	Qué hace
`/api/clients/me`	Usuario con sesión	Perfil del cliente.
`/api/cart`	Usuario con sesión	Carrito.
`/api/orders`	Usuario con sesión	Checkout, historial y cancelación de sus pedidos.
`/api/admin/...`	Solo rol `Admin`	Panel: pedidos, productos, categorías, clientes, reportes, auditoría.

1.3 Cómo se arranca

Hay dos formas:

```
# Desde la raíz del monorepo: arranca la API y la web a la vez
pnpm dev

# Solo la API, desde apps/api
dotnet watch --project API.Furnistore.API/API.Furnistore.API.csproj run
```

La primera llega a la segunda: Turborepo ejecuta el script `dev` de `apps/api/package.json`, que es exactamente ese `dotnet watch`. El `package.json` de la API existe solo para que Turborepo la trate como un paquete más del monorepo; .NET no lo lee.

`dotnet watch` compila, arranca y **vuelve a compilar y reiniciar** cada vez que guardas un archivo `.cs`. El puerto `5135` sale de `API.Furnistore.API/Properties/launchSettings.json` (perfil `http`), y la web lo conoce por su variable `API_BASE_URL=http://localhost:5135`.

1.4 Las carpetas de un vistazo

```
apps/api/
├── API.sln                          la "solución": lista los proyectos
├── global.json                      fija la versión del SDK de .NET
├── .config/dotnet-tools.json        herramientas de línea de comandos del proyecto
├── openapi.json                     contrato generado de la API
├── package.json                     solo para Turborepo
├── API.Furnistore.API/              proyecto web: el ejecutable
├── API.Furnistore.Application/      reglas de negocio
├── API.Furnistore.Data/             acceso a datos (EF Core) y migraciones
├── API.Furnistore.Shared/           entidades y tipos básicos
├── API.Furnistore.IntegrationTests/ pruebas
└── API.Furnistore.UnitTests/        restos de compilación, no es un proyecto (ver 2.2)
```

---

2. Anatomía de una solución .NET

2.1 SDK, runtime y la herramienta `dotnet`

Concepto. Para *ejecutar* una aplicación .NET basta el **runtime**. Para *compilarla* hace falta el **SDK**, que incluye el runtime más el compilador y la herramienta `dotnet` (`dotnet build`, `dotnet run`, `dotnet test`, `dotnet publish`...).

En Furnistore. `global.json` fija qué SDK usar:

```
{
  "sdk": {
    "version": "10.0.100",
    "rollForward": "latestMinor",
    "allowPrerelease": false
  }
}
```

`rollForward: latestMinor` significa: "usa la 10.0.100 o cualquier versión 10.x más nueva que tengas instalada, pero nunca la 11". Así todo el equipo compila con la misma versión mayor sin obligar a tener exactamente el mismo parche.

2.2 La solución (`.sln`) y los proyectos (`.csproj`)

Concepto. En .NET el código se organiza en **proyectos**. Cada proyecto es un archivo `.csproj` más los `.cs` de su carpeta, y al compilarse produce un **ensamblado** (un `.dll`, o un ejecutable). Una **solución** (`.sln`) es solo una lista de proyectos que se abren y compilan juntos; no contiene código.

En Furnistore. `API.sln` lista cinco proyectos:

Proyecto	Tipo	Produce	Archivos .cs*	Líneas
`API.Furnistore.API`	Web (`Microsoft.NET.Sdk.Web`)	El ejecutable de la API	29	~1.4
`API.Furnistore.Application`	Biblioteca (`Microsoft.NET.Sdk`)	`API.Furnistore.Application.dll`	41	~4.8
`API.Furnistore.Data`	Biblioteca	`API.Furnistore.Data.dll`	9	~2
`API.Furnistore.Shared`	Biblioteca	`API.Furnistore.Shared.dll`	13	~3
`API.Furnistore.IntegrationTests`	Pruebas (xUnit)	Ensamblado de pruebas	14	~2.2

* Sin contar migraciones, que son código generado.

La carpeta `API.Furnistore.UnitTests` **no es un proyecto**: no tiene `.csproj` ni aparece en `API.sln`. Solo contiene carpetas `bin` y `obj` que quedaron de un proyecto que se borró. Se puede eliminar sin consecuencias.

2.3 Qué hay dentro de un `.csproj`

Este es el de la capa `Application`, completo:

```
<Project Sdk="Microsoft.NET.Sdk">

  <ItemGroup>
    <ProjectReference Include="../API.Furnistore.Data/API.Furnistore.Data.csproj" />
    <ProjectReference Include="../API.Furnistore.Shared/API.Furnistore.Shared.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Extensions.Logging.Abstractions" Version="10.0.0" />
    <PackageReference Include="System.IdentityModel.Tokens.Jwt" Version="8.14.0" />
  </ItemGroup>

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>
```

```
</Project>
```

Pieza por pieza:

- **`Sdk="..."`**: qué tipo de proyecto es. `Microsoft.NET.Sdk` es una biblioteca o una app de consola; `Microsoft.NET.Sdk.Web` añade todo lo de ASP.NET Core (servidor web, `appsettings.json`, `launchSettings.json`...). Solo `API.Furnistore.API` usa el SDK Web.
- **`TargetFramework`**: para qué versión de .NET se compila (`net10.0`).
- **`ImplicitUsings`**: el compilador añade solo los `using` más comunes (`System`, `System.Linq`, `System.Threading.Tasks`...), por eso muchos archivos no los declaran.
- **`Nullable`**: activa el análisis de nulos. Con él, `string` significa "nunca null" y `string?` significa "puede ser null", y el compilador avisa si usas algo que podría ser null sin comprobarlo. Por eso verás `Phone!` (el `!` le dice al compilador "sé que aquí no es null").
- **`ProjectReference`**: este proyecto usa el código de otro proyecto de la solución.
- **`PackageReference`**: este proyecto usa un paquete de **NuGet**, el gestor de paquetes de .NET (el equivalente de npm).

2.4 Los paquetes NuGet y para qué sirve cada uno

Paquete	Proyecto	Para qué
`Microsoft.EntityFrameworkCore`	API, Data	EF Core, el ORM: traduce LINQ a SQL y objetos a filas.
`Npgsql.EntityFrameworkCore.PostgreSQL`	API, Data	El "proveedor" que enseña a EF Core a hablar con PostgreSQL.
`Microsoft.EntityFrameworkCore.Design` y `.Tools`	API, Data	Soporte para la herramienta `dotnet ef` (crear y aplicar migraciones). No se incluyen en el ejecutable final (`PrivateAssets=all`).
`Microsoft.AspNetCore.Identity.EntityFrameworkCore`	Data	Tablas de usuarios y roles de ASP.NET Core Identity guardadas con EF Core.
`Microsoft.AspNetCore.Identity.UI`	API	Aporta `AddDefaultIdentity`, el método con el que `Program.cs` registra Identity.
`Microsoft.AspNetCore.Authentication.JwtBearer`	API	Validar los tokens JWT que llegan en la cabecera `Authorization`.
`System.IdentityModel.Tokens.Jwt`	Application	Generar los tokens JWT en el login.

Paquete	Proyecto	Para qué
`Microsoft.Extensions.Logging.Abstractions`	Application	La interfaz `ILogger<T>`, sin atarse a ninguna librería de logging concreta.
`NLog.Web.AspNetCore`	API	La librería de logging real (ver capítulo 9).
`Swashbuckle.AspNetCore`	API	Genera el documento OpenAPI y la página Swagger UI.
`Microsoft.Extensions.Diagnostics.HealthChecks.EntityFrameworkCore`	API	El health check que comprueba la conexión a la base de datos.
`MailKit`	API	Enviar correos por SMTP (códigos de verificación).
`dotenv.net`	API	Cargar el archivo `.env` como variables de entorno en desarrollo.
`System.Security.Cryptography.Xml`	Data	Fija una versión concreta de una dependencia indirecta.
`Microsoft.AspNetCore.OpenApi`	API	No se usa directamente en el código; es un resto que podría quitarse.

Que algunos paquetes aparezcan en dos proyectos es normal: `Data` necesita EF Core y Npgsql para definir el modelo y las migraciones, y `API` los necesita para llamar a `UseNpgsql(...)` al configurar la conexión y para que `dotnet ef` pueda arrancarse desde el proyecto ejecutable.

2.5 Compilar: `restore`, `build`, `bin` y `obj`

Al compilar (`dotnet build`) pasan dos cosas:

1. **Restore:** se descargan los paquetes NuGet a una caché global (`~/.nuget/packages`) y se escribe en `obj/project.assets.json` qué versión exacta de cada paquete se usa.
2. **Build:** se compila cada proyecto en orden de dependencias. En `obj/` quedan los archivos intermedios y en `bin/<Configuración>/net10.0/` queda el resultado.

En `bin/` del proyecto `API` se copian **todos** los `.dll` que necesita para ejecutarse: los de los otros cuatro proyectos y los de todos los paquetes NuGet. Por eso `bin` y `obj` pesan cientos de megas mientras el código fuente pesa unos pocos. No se suben a Git y se regeneran con `dotnet build`.

`Debug` y `Release` son las dos **configuraciones**: `Debug` (la de `dotnet run` y `dotnet watch`) incluye información para depurar; `Release` (la de `pnpm build`) está optimizada.

2.6 Herramientas locales

Concepto. Las herramientas de línea de comandos de .NET se instalan como paquetes NuGet. Pueden ser globales (para toda la máquina) o **locales** (fijadas por proyecto, con su versión).

En Furnistore. `apps/api/.config/dotnet-tools.json` declara dos herramientas locales:

Herramienta	Versión	Comando	Para qué
`dotnet-ef`	10.0.11	`dotnet ef ...`	Crear y aplicar migraciones de la base de datos.
`swashbuckle.aspnetcore.cli`	6.6.2	`dotnet swagger ...`	Generar `openapi.json` desde el `.dll` compilado.

`dotnet tool restore` las instala con la versión exacta del manifiesto. **Por qué así:** si cada desarrollador usara su `dotnet-ef` global, versiones distintas podrían generar migraciones distintas.

2.7 Namespaces

Cada archivo declara un **namespace** que coincide con su carpeta: `API.Furnistore.Application/Orders/OrderService.cs` está en `namespace API.Furnistore.Application.Orders`. Para usar una clase de otro namespace se importa con `using`. Es una convención, no una obligación del compilador, pero toda la API la respeta y conviene mantenerla: así el nombre completo de una clase dice dónde está su archivo.

---

3. `Program.cs` paso a paso

Concepto. `Program.cs` es el punto de entrada. En ASP.NET Core moderno se escribe con **instrucciones de nivel superior** (sin `class Program` ni `static void Main`) y tiene siempre tres fases:

1. **Construir:** `WebApplication.CreateBuilder(args)` crea el `builder`, y sobre `builder.Services` se **registran servicios** en el contenedor de inyección de dependencias.
2. **Compilar:** `builder.Build()` crea la aplicación (`app`). A partir de aquí ya no se pueden registrar servicios.
3. **Configurar el pipeline y ejecutar:** con `app.Use...` se declara por qué middleware pasa cada petición, con `app.Map...` qué endpoints hay, y `app.Run()` arranca el servidor.

En Furnistore. `API.Furnistore.API/Program.cs` tiene 261 líneas. Lo recorremos en orden.

3.1 Envoltorio de arranque

```
var logger = NLog.LogManager.Setup().LoadConfigurationFromAppSettings().GetCurrentClassLogger();

try
{
    var builder = WebApplication.CreateBuilder(args);
    …
    app.Run();
}
catch (Exception ex)
{
    logger.Error(ex, "Stopped program because of exception");
    throw;
}
```

```
finally
{
    NLog.LogManager.Shutdown();
}
```

Se crea un logger de NLog **antes** que nada, para poder registrar errores que ocurran mientras se configura la app (por ejemplo, que falte `DATABASE_URL`). El `finally` vacía los logs pendientes al apagar.

3.2 Fase 1: registrar servicios

En orden de aparición:

Línea(s)	Qué registra	Detalle en
`AddCors(...)`	Política `WebApp`: qué orígenes (dominios) pueden llamar a la API desde un navegador. Lee `CORS_ORIGINS`; por defecto `localhost:3000` y `3001`.	17
`DotEnv.Load(...)`	Solo en Development: carga `.env` como variables de entorno.	5
`AddApiContract()`	Controladores, JSON con enums como texto, Swagger.	8
`AddProblemDetails(...)`	Formato estándar de errores, con `traceId`.	8.2
`AddExceptionHandler<GlobalExceptionHandler>()`	Qué hacer con una excepción no controlada.	8.2
`AddHttpContextAccessor()`	Permite leer la petición actual fuera de un controlador.	—
`AddApplicationServices()`	Todos los servicios de negocio.	4
`AddHealthChecks().AddDbContextCheck<...>()`	El health check de `/health`.	11
`AddDbContext<APIFurnistoreContext>(...)`	La conexión a PostgreSQL.	11
`AddHostedService<DatabaseWarmupService>()`	Tarea de fondo que "calienta" la base al arrancar.	11

Línea(s)	Qué registra	Detalle en
`AddSingleton(new JwtOptions {...})`	Secreto, emisor, audiencia y duración de los JWT.	16
`Configure<SmtpSettings>(...)`, `Configure<SupabaseStorageSettings>(...)`	Configuración tipada de correo e imágenes.	5
`Configure<ForwardedHeadersOptions>(...)`	Confiar en la IP real que envía un proxy.	17
`AddAuthRateLimiting()`	Límites de peticiones en auth.	17
`AddAuthentication(...).AddJwtBearer(...)`	Autenticación por JWT.	16
`AddDefaultIdentity<IdentityUser>(...).AddRoles<IdentityRole>()...`	Usuarios, contraseñas, bloqueo y roles.	15
`builder.Host.UseNLog()`	NLog como proveedor de logging.	9

Varios de estos registros están agrupados en **métodos de extensión** propios (`AddApiContract`, `AddApplicationServices`, `AddAuthRateLimiting`) en la carpeta `Extensions/`. Es la convención de ASP.NET Core para que `Program.cs` se lea como un índice y cada bloque grande viva en su archivo.

3.3 Fase 2: `Build()` y tareas de arranque

```
var app = builder.Build();
```

Después del `Build`, y antes de atender peticiones, `Program.cs` hace cuatro cosas:

1. **Avisos de configuración incompleta.** Si faltan las credenciales SMTP o las de Supabase Storage, escribe un *warning* en el log. La API arranca igualmente; solo fallaría el envío de correos. El aviso de Supabase quedó de cuando las imágenes se subían allí: hoy se guardan en `assets/` (6.4) y ese aviso se puede ignorar.
2. **Crear los roles.** Si no existen los roles `Admin` y `User`, los crea:

   ```
   using (var scope = app.Services.CreateScope())
   {
       var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
       if (!await roleManager.RoleExistsAsync("Admin"))
           await roleManager.CreateAsync(new IdentityRole("Admin"));
       …
   }
   ```

   Se crea un *scope* a mano porque `RoleManager` depende del `DbContext`, que es *Scoped* (capítulo 4) y fuera de una petición no hay scope.
3. **Comando de consola.** `AdminCommand.TryRunAsync(app, args)` mira si la API se lanzó como `... --admin promote <correo>`. Si es así, da el rol de administrador, imprime el resultado y **termina sin levantar el servidor**. Así se crea el primer administrador, que de otro modo sería imposible: para dar el rol desde el panel hace falta ya ser administrador.

3.4 Fase 3: el pipeline de middleware

Concepto. Un **middleware** es un eslabón por el que pasa cada petición. Cada uno puede hacer algo antes, llamar al siguiente y hacer algo después, cuando vuelve la respuesta. **El orden en que se declaran es el orden en que se ejecutan**, y en muchos casos el orden es obligatorio.

En Furnistore.

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseForwardedHeaders();
app.UseMiddleware<RequestLoggingMiddleware>();
app.UseExceptionHandler();

if (!app.Environment.IsDevelopment())
    app.UseHttpsRedirection();

app.UseRouting();
app.UseCors("WebApp");
app.UseAuthentication();
app.UseAuthorization();
app.UseRateLimiter();
app.MapControllers();

app.LogRegisteredEndpoints();
app.MapHealthChecks("/health");
app.Run();
```

#	Middleware	Qué hace	Por qué en esta posición
1	`UseSwagger`, `UseSwaggerUI`	Sirve `/swagger`.	Solo en Development; no debe exponerse en producción.
2	`UseForwardedHeaders`	Si hay un proxy delante, toma la IP real del cliente de `X-Forwarded-For`.	Antes que cualquiera que use la IP (el log y el rate limiting).
3	`RequestLoggingMiddleware`	Cronometra la petición y escribe una línea de log al terminar.	Muy arriba, para medir el tiempo total y registrar también las peticiones que fallan.
4	`UseExceptionHandler`	Si algo más abajo lanza una excepción, la convierte en un 500 con formato estándar.	Solo atrapa excepciones de lo que está **después**; por eso va antes de todo lo que puede fallar.
5	`UseHttpsRedirection`	Redirige HTTP a HTTPS.	Solo fuera de Development.
6	`UseRouting`	Decide qué endpoint corresponde a la URL.	Antes de CORS y autorización, que necesitan saber el endpoint.
7	`UseCors`	Añade las cabeceras CORS y responde a las peticiones *preflight*.	Después del routing y **antes** de autenticación: un *preflight* no lleva token y no debe rechazarse.
8	`UseAuthentication`	Lee el JWT y construye el usuario (`HttpContext.User`).	Antes de la autorización: no se puede decidir si alguien tiene permiso sin saber quién es.
9	`UseAuthorization`	Aplica `[Authorize]`: responde 401 o 403 si no hay permiso.	Después de autenticación.

#	Middleware	Qué hace	Por qué en esta posición
10	`UseRateLimiter`	Aplica los límites de peticiones.	Después del routing, porque los límites se asignan por endpoint.
11	`MapControllers`	Registra los endpoints de los controladores.	Es el final del camino.
12	`MapHealthChecks("/health")`	Registra `/health`.	—

`LogRegisteredEndpoints()` no es un middleware: en Development escribe al arrancar la lista de todos los endpoints con su método y nivel de acceso.

3.5 La última línea

```
public partial class Program;
```

Con instrucciones de nivel superior, el compilador genera una clase `Program` interna. Esta línea la hace pública para que el proyecto de pruebas pueda arrancar la API completa en memoria con `WebApplicationFactory<Program>` (lo usa `AdminAuthorizationTests`).

---

4. Inyección de dependencias

4.1 Concepto

La **inyección de dependencias** (DI) consiste en que una clase **no crea** los objetos que necesita: los **pide en su constructor** y alguien se los entrega. En ASP.NET Core ese "alguien" es el **contenedor de servicios**, que viene integrado.

```
public sealed class OrdersController(OrderService orders) : ControllerBase
```

Esta línea usa un **constructor primario** (sintaxis de C# 12): equivale a declarar un campo `orders` y un constructor que lo recibe. Cuando llega una petición a `/api/orders`, ASP.NET Core le pide al contenedor un `OrdersController`; el contenedor ve que necesita un `OrderService`, ve que `OrderService` necesita un `APIFurnistoreContext`, un `OrderWorkflow` y un `ILogger<OrderService>`, y así hasta el final de la cadena. Construye todo el árbol y entrega el controlador listo.

Ventajas:

- Ninguna clase decide cómo se construyen sus dependencias, así que se puede cambiar una implementación sin tocar a quien la usa.
- En las pruebas se puede registrar otra implementación (por ejemplo, un almacén de imágenes falso).
- El contenedor controla **cuánto vive** cada objeto.

4.2 Tiempos de vida

Tiempo de vida	Se crea…	Ejemplo en Furnistore
Singleton	Una vez para toda la aplicación.	`SmtpEmailSender`, `TimeProvider.System`, `JwtOptions`, `TokenValidationParameters`.
Scoped	Una vez por petición HTTP. Todos los que lo pidan en la misma petición reciben el mismo.	`APIFurnistoreContext` y todos los servicios de negocio.
Transient	Cada vez que alguien lo pide.	Ningún servicio propio hoy. Los clientes HTTP tipados, como el antiguo `SupabaseImageStorage`, lo son.

La regla que no se puede romper: **un servicio no puede depender de otro que vive menos que él.** Si un Singleton guardara un `DbContext` (Scoped), ese `DbContext` quedaría vivo para siempre, compartido entre peticiones simultáneas, y EF Core no es seguro para eso. Esto se llama **dependencia cautiva**, y en Development ASP.NET Core lanza un error al detectarla.

4.3 En Furnistore

Los servicios de negocio se registran en `Extensions/ApplicationServiceCollectionExtensions.cs`:

```
services.AddScoped<ProductService>();
services.AddScoped<OrderService>();
services.AddScoped<OrderWorkflow>();
services.AddScoped<AuditLog>();
… (14 servicios Scoped en total)

services.AddSingleton<IVerificationEmailSender, SmtpEmailSender>();
services.AddSingleton<IProductImageStorage, LocalImageStorage>();
services.AddSingleton(TimeProvider.System);
```

Por qué los servicios son Scoped. Todos usan el `DbContext`, que es Scoped. Además, que todos los servicios de una misma petición compartan **el mismo** `DbContext` es útil: si `OrderService` y `AuditLog` modifican entidades en la misma petición, un único `SaveChangesAsync()` guarda todo junto.

Por qué la mayoría se registran sin interfaz (`AddScoped<OrderService>()` y no `AddScoped<IOrderService, OrderService>()`). Una interfaz solo aporta algo si existe, o se espera, más de una implementación. `OrderService` tiene una sola y las pruebas usan la real contra una base de datos real (capítulo 23), así que una interfaz `IOrderService` sería un archivo más sin beneficio.

Por qué dos servicios sí tienen interfaz. `IVerificationEmailSender` (enviar correo) e `IProductImageStorage` (guardar imágenes) hablan con **sistemas externos**. Tener la interfaz permite dos cosas:

- La interfaz se declara en `Application`, pero la implementación (`SmtpEmailSender`, `LocalImageStorage`) vive en `API`. Así la lógica de negocio no depende de MailKit ni de dónde se guarden los archivos (ver 6.4).
- En las pruebas se registra `FakeImageStorage` en lugar del disco.

`LocalImageStorage` es Singleton porque no guarda estado propio: escribe el archivo en la carpeta `assets/` del repositorio y arma la URL con el `IHttpContextAccessor`, que siempre devuelve la petición en curso. Hasta el 16 de octubre la implementación era `SupabaseImageStorage`, registrada con `AddHttpClient<IProductImageStorage, SupabaseImageStorage>()`: un **cliente HTTP tipado**, al que el

contenedor inyecta un `HttpClient` gestionado por una fábrica que reutiliza las conexiones. Esa clase sigue en el código, sin uso. Crear `new HttpClient()` en cada llamada es un error clásico que agota los *sockets*, y por eso existe esa fábrica.

`TimeProvider.System` es el reloj de .NET registrado como servicio. Los servicios que necesitan la hora la piden por inyección (`TimeProvider clock`) en lugar de llamar a `DateTime.UtcNow`, y así una prueba puede inyectar un reloj falso para simular, por ejemplo, que un código ya caducó. *Nota:* `OrderService.CheckoutAsync` todavía usa `DateTime.UtcNow` directamente; es una inconsistencia menor.

Servicios de fondo. `DatabaseWarmupService` es un `BackgroundService` (Singleton). Como no puede recibir un `DbContext` en el constructor (sería una dependencia cautiva), recibe un `IServiceScopeFactory` y crea su propio scope:

```
using var scope = scopeFactory.CreateScope();
var db = scope.ServiceProvider.GetRequiredService<APIFurnistoreContext>();
```

Es el patrón correcto siempre que algo de larga duración necesita un servicio Scoped.

---

5. Configuración

5.1 Concepto: fuentes y precedencia

ASP.NET Core junta la configuración de varias **fuentes** en un único diccionario de claves. Si una clave aparece en varias, **gana la última fuente cargada**. Por defecto, de menor a mayor prioridad:

1. `appsettings.json`
2. `appsettings.{Entorno}.json`, por ejemplo `appsettings.Development.json`
3. *User secrets* (solo en Development)
4. Variables de entorno
5. Argumentos de la línea de comandos

Las claves son jerárquicas y se separan con `:`. En una variable de entorno no se puede usar `:`, así que se usa **doble guion bajo**: la variable `SmtpSettings__Password` es la clave `SmtpSettings:Password`, la misma que en JSON sería:

```
{ "SmtpSettings": { "Password": "..." } }
```

5.2 En Furnistore

`appsettings.json` tiene los valores no secretos y por defecto: niveles de log, `JwtConfig:ExpiryTime` (`01:00:00`), servidor y puerto SMTP, nombre del remitente. Los campos secretos están vacíos.

El archivo `.env` tiene los secretos. Lo carga la librería `dotenv.net`, **solo en Development**:

```
if (builder.Environment.IsDevelopment())
{
    DotEnv.Load(
        options: new DotEnvOptions(
            envFilePaths: new[] { ".env", "../.env", "../../.env" },
            overwriteExistingVars: false
        )
    );
```

```
    builder.Configuration.AddEnvironmentVariables();
}
```

- Busca el `.env` en tres rutas porque el directorio de trabajo cambia según desde dónde se arranque (`dotnet run` en la carpeta del proyecto, `pnpm dev` desde la raíz...).
- `overwriteExistingVars: false`: si una variable ya existe en el entorno real, el `.env` no la pisa.
- `AddEnvironmentVariables()` se vuelve a llamar porque el `builder` ya había leído las variables de entorno **antes** de que se cargara el `.env`.

En producción no se carga ningún `.env`: las variables las pone el entorno de ejecución (un contenedor Docker, un servidor...). Es la práctica estándar: los secretos no viajan en archivos dentro de la aplicación.

Variables que usa la API (lista completa en `apps/api/.env.example`):

Variable	Para qué
`DATABASE_URL`	Cadena de conexión a PostgreSQL.
`JWT_SECRET`, `JWT_ISSUER`, `JWT_AUDIENCE`	Firmar y validar los tokens.
`SmtpSettings__UserName`, `SmtpSettings__Password`, `SmtpSettings__SenderEmail`	Credenciales del correo.
`SupabaseStorage__Url`, `SupabaseStorage__ServiceKey`	Subida de imágenes a Supabase. Sin uso desde que las imágenes se guardan en `assets/`.
`ForwardedHeaders__KnownProxies`	IPs de proxies de confianza.
`CORS_ORIGINS`	Orígenes permitidos, separados por comas.
`ASPNETCORE_ENVIRONMENT`, `ASPNETCORE_URLS`	Entorno y dirección en la que escucha el servidor. Las lee ASP.NET Core directamente.

5.3 Dos estilos de leer la configuración

En `Program.cs` conviven dos estilos.

Estilo 1: leer valores sueltos. Se usa para la base de datos y JWT:

```
var connectionString =
    Environment.GetEnvironmentVariable("DATABASE_URL")
    ?? builder.Configuration.GetConnectionString("APIFurnistoreContext")
    ?? throw new InvalidOperationException("DATABASE_URL not configured");
```

Se mira primero la variable de entorno, después la configuración, y si no hay nada **la aplicación no arranca**. Fallar al arrancar es deliberado: es mucho mejor que arrancar y fallar en la primera petición.

Estilo 2: el patrón Options. Se usa para SMTP y Supabase Storage:

```
builder.Services.Configure<SmtpSettings>(builder.Configuration.GetSection("SmtpSettings"));
```

Esto enlaza la sección `SmtpSettings` con una clase normal de C# (`Configuration/SmtpSettings.cs`), propiedad por propiedad. Quien la necesite pide `IOptions<SmtpSettings>` por inyección y recibe un objeto tipado:

```
public class SmtpSettings
{
    public string Server { get; set; } = string.Empty;
```

```
    public int Port { get; set; }
    public SecureSocketOptions Security { get; set; } = SecureSocketOptions.StartTls;
    …
    public bool HasCredentials =>
        !string.IsNullOrWhiteSpace(UserName) && !string.IsNullOrWhiteSpace(Password);
}
```

Por qué así. El patrón Options es el idiomático en .NET: tipado, con valores por defecto, y la clase puede incluir lógica como `HasCredentials`. Que JWT y la base de datos usen el estilo 1 es histórico: se configuraron antes y con nombres de variable "planos" (`DATABASE_URL`) en lugar de jerárquicos. Si se añade configuración nueva, conviene usar Options.

5.4 El entorno: Development y Production

`ASPNETCORE_ENVIRONMENT` (por defecto `Production`; `launchSettings.json` lo pone en `Development` al trabajar en local) cambia el comportamiento en estos puntos:

Comportamiento	Development	Production
Carga `.env`	Sí	No
Swagger UI	Sí	No
Lista de endpoints al arrancar	Sí	No
Redirección a HTTPS	No	Sí
Formato de logs (`nlog.config`)	Texto con colores	JSON

5.5 Sobre los *user secrets*

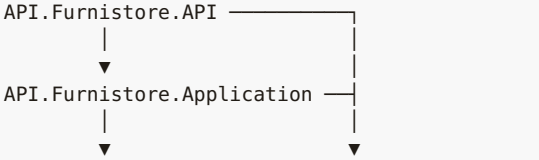
El `.csproj` de la API tiene un `<UserSecretsId>`, que habilita la fuente "user secrets" de .NET: un archivo fuera del repositorio donde guardar secretos de desarrollo. El proyecto no la usa (usa `.env`), así que esa línea puede ignorarse. Se menciona porque la encontrarás en muchos proyectos .NET.

Parte II. Arquitectura

6. Las capas y por qué existen

6.1 La regla

La API está dividida en cuatro proyectos de código más el de pruebas, y **las dependencias solo van en una dirección**:



Proyecto	Depende de	No puede conocer
`Shared`	Nada	Nada de los demás, ni EF Core, ni ASP.NET Core
`Data`	`Shared`	Las reglas de negocio ni HTTP
`Application`	`Data`, `Shared`	HTTP, controladores, MailKit, Supabase
`API`	Los tres	—

El compilador **hace cumplir** esta regla: como `Data` no tiene un `ProjectReference` a `Application`, es imposible escribir en `Data` código que use un servicio de negocio. Esa es la razón principal para separar en proyectos y no solo en carpetas: con carpetas, nada impediría que un día una entidad llamara a un controlador.

6.2 Qué hay en cada capa

`API.Furnistore.Shared`: el modelo del dominio.

- Las **entidades**: `Product`, `ProductCategory`, `Client`, `CartItem`, `Order`, `OrderDetail`, `OrderStatus`, `AuditEntry`, `RefreshToken`, `EmailVerificationCode`.
- Tipos básicos que usan todas las capas: `Result`/`Result<T>`/`Error` (8.1), `PagedResult<T>` (8.5) y `RandomGenerator`.

Las entidades son clases con propiedades y casi sin comportamiento:

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public int Stock { get; set; }
    public int ProductCategoryId { get; set; }
    public ProductCategory Category { get; set; } = null!;
    public bool IsActive { get; set; } = true;
    public uint Version { get; set; }
    …
}
```

Este estilo se llama **modelo anémico**: las reglas no viven en la entidad, viven en los servicios. La única excepción es `Client.IsProfileComplete`, una propiedad calculada.

Sobre el nombre. "Shared" sugiere "cosas compartidas", pero en la práctica es la capa de **dominio**. En otros proyectos .NET la encontrarás como `Domain` o `Core`.

`API.Furnistore.Data`: persistencia.

- `APIFurnistoreContext`: el `DbContext` de EF Core, el objeto a través del cual se consulta y se guarda en la base de datos.
- `Configurations/`: una clase por entidad que define tabla, índices, restricciones y relaciones.
- `Migrations/`: el historial de cambios del esquema.

Se explica a fondo en la Parte III. Depende de Identity porque el `DbContext` hereda de `IdentityDbContext`, que añade las tablas de usuarios y roles.

`API.Furnistore.Application`: la lógica de negocio. Es el proyecto más grande (~4.900 líneas). Está organizado **por funcionalidad**, no por tipo de archivo:

```
Application/
├── Auth/                AuthService, AuthContracts, VerificationCodes, RefreshTokenHasher, …
├── Carts/               CartService, CartContracts
├── Clients/             ClientService, ClientContracts, IdentityCardAttribute
├── Orders/              OrderService, OrderWorkflow, OrderMapping, ShippingPolicy, OrderContracts
├── Products/            ProductService, ProductContracts
├── ProductCategories/   ProductCategoryService, ProductCategoryContracts
├── Admin/
│   ├── Audit/           AuditLog, AuditService, AuditContracts
│   ├── Categories/      …
│   ├── Customers/       …
│   ├── Orders/          …
│   ├── Products/        AdminProductService, IProductImageStorage, …
│   └── Reports/         AdminReportService, ReportPeriod, ReportContracts
└── Common/              ApiEvents, SortMap, LikePattern, TextInput
```

Cada funcionalidad tiene casi siempre dos archivos:

- **`XxxService.cs`**: los métodos que implementan los casos de uso ("hacer checkout", "cancelar pedido"). Devuelven `Result<T>`.
- **`XxxContracts.cs`**: los **contratos**, es decir, las clases que entran (`CheckoutRequest`, `OrderQuery`) y salen (`OrderResponse`) de la API, con sus reglas de validación (8.3).

Por qué por funcionalidad. Cuando cambias "pedidos", todo lo de pedidos está en una carpeta. La alternativa (una carpeta `Services/` con todos los servicios y otra `Dtos/` con todos los contratos) obliga a saltar entre carpetas para cada cambio y crece mal.

`API.Furnistore.API`: el host web.

```
API/
├── Program.cs            arranque (capítulo 3)
├── Controllers/          traducen HTTP ↔ llamadas a servicios
│   └── Admin/            controladores del panel, con AdminControllerBase
├── Middleware/           RequestLoggingMiddleware, GlobalExceptionHandler
├── Extensions/           registro de servicios y utilidades HTTP
├── Configuration/        clases de Options (SmtpSettings, SupabaseStorageSettings)
├── Services/             SmtpEmailSender, LogVerificationEmailSender, LocalImageStorage,
│                         SupabaseImageStorage (sin uso), DatabaseWarmupService
├── Commands/             AdminCommand (comando de consola)
├── SwaggerHostFactory.cs host mínimo para generar openapi.json (capítulo 24)
├── appsettings*.json
└── nlog.config
```

Los controladores son finos a propósito: reciben la petición, sacan el usuario del token, llaman a **un** método del servicio y convierten el `Result` en una respuesta HTTP. No contienen reglas ni tocan el `DbContext`.

```
[HttpGet("{id:int}")]
public async Task<IActionResult> GetById(int id, CancellationToken cancellationToken) =>
    (await orders.GetByIdAsync(id, User.UserId(), cancellationToken)).ToActionResult(this);
```

6.3 Qué se ejecuta al final

Solo `API.Furnistore.API` es ejecutable. `Application`, `Data` y `Shared` son bibliotecas (`.dll`) que se copian a su carpeta `bin` al compilar. Desplegar la API es publicar ese proyecto (`dotnet publish`), que produce una carpeta con el ejecutable y todos los `.dll`.

6.4 Inversión de dependencias en un caso concreto

`AdminProductService` (en `Application`) necesita guardar imágenes, pero `Application` no puede saber si van a un disco, a Supabase o a otro servicio. La solución:

1. `Application` declara **qué necesita**, como una interfaz:

   ```
   // Application/Admin/Products/IProductImageStorage.cs
   public interface IProductImageStorage { … }
   ```

2. `API` aporta **cómo se hace**: `Services/LocalImageStorage.cs` implementa esa interfaz guardando el archivo en `assets/`, y la API sirve esa carpeta en `/assets`.
3. El registro los une: `AddSingleton<IProductImageStorage, LocalImageStorage>()`.
4. Las pruebas registran `FakeImageStorage` en su lugar.

La dependencia "natural" sería servicio → almacenamiento; con la interfaz se **invierte**: el almacenamiento depende de una interfaz definida por el negocio. Y esto no es teoría: hasta el 16 de octubre las imágenes se subían a Supabase Storage con `SupabaseImageStorage`. Para que la aplicación funcionara sin conexión se escribió `LocalImageStorage` y se cambió **una línea** del registro; `AdminProductService` y sus pruebas no se tocaron. Lo mismo ocurre con `IVerificationEmailSender`: en desarrollo se puede cambiar `SmtpEmailSender` por `LogVerificationEmailSender`, que escribe el código en la consola (18.6).

6.5 Por qué esta arquitectura y no otra

Hay tres puntos de referencia habituales en .NET:

Opción	Cómo es	Por qué no
Un solo proyecto	Controladores que usan el `DbContext` directamente.	Funciona para proyectos pequeños, pero las reglas acaban repartidas por los controladores y nada impide mezclar capas. La API empezó así; un refactor movió la lógica a servicios y los cinco controladores que existían entonces pasaron de 727 a 294 líneas.
"Clean Architecture" completa	Proyectos `Domain`, `Application`, `Infrastructure`, `WebApi`; repositorios (`IProductRepository`…) entre la lógica y EF Core; a menudo MediatR (un comando y un *handler* por caso de uso).	Mucha ceremonia: varios archivos por caso de uso. Los repositorios sobre EF Core suelen limitarse a reenviar llamadas, porque el `DbContext` ya es un repositorio (cada `DbSet`) y una unidad de trabajo (`SaveChangesAsync`).

Opción	Cómo es	Por qué no
Capas pragmáticas (la elegida)	Cuatro capas con dependencias en un sentido; servicios que usan el `DbContext` directamente; interfaces solo en las integraciones externas.	—

La concesión consciente es que **`Application` depende de `Data`, y por tanto de EF Core**: los servicios escriben consultas LINQ sobre el `DbContext` y no son independientes de la base de datos. A cambio:

- Las consultas se escriben donde se usan, sin una capa intermedia.
- Se puede usar todo EF Core y PostgreSQL (transacciones, `FOR UPDATE`, `ExecuteUpdateAsync`, bloqueos *advisory*) sin pelear con una abstracción.
- Las pruebas usan un PostgreSQL real (capítulo 23) en lugar de simular repositorios, así que prueban también el SQL.

Cuándo habría que revisarla. Si se necesitara cambiar de motor de base de datos con frecuencia, o probar la lógica sin ninguna base de datos, valdría la pena introducir repositorios. Si una funcionalidad creciera mucho (por ejemplo, reportes), podría separarse en su propio proyecto.

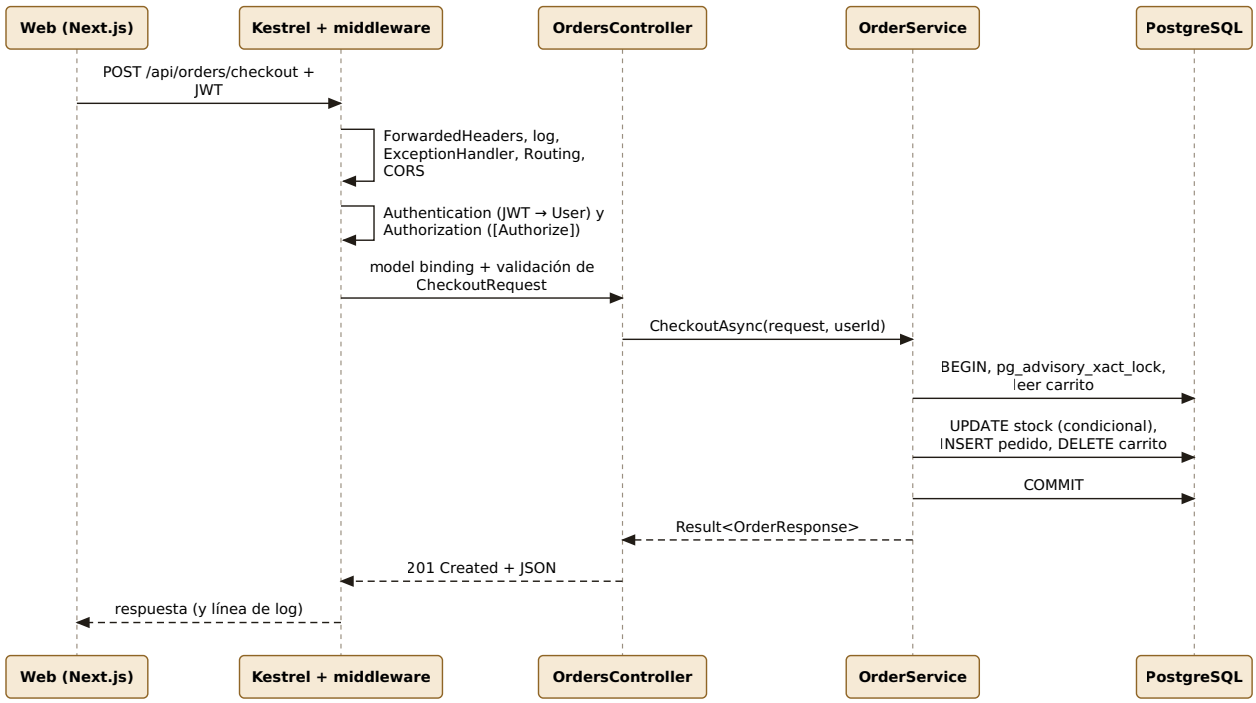
---

7. Recorrido completo de una petición: el checkout

Este capítulo sigue una petición real de principio a fin: un cliente con sesión confirma su compra.

```
POST /api/orders/checkout
Authorization: Bearer eyJhbGciOi…
Content-Type: application/json

{ "expectedTotal": 526.90 }
```



Paso 1. Kestrel recibe la conexión

Kestrel es el servidor HTTP integrado en ASP.NET Core. Lee la petición y crea un `HttpContext`, el objeto que representa la petición y la respuesta durante todo el recorrido.

Paso 2. Middleware de entrada

En el orden del capítulo 3.4:

- `UseForwardedHeaders` fija la IP real del cliente.
- `RequestLoggingMiddleware` arranca un cronómetro y abre un **scope de log** con `TraceId` y `UserId`: todo lo que se registre durante esta petición llevará esos datos.
- `UseExceptionHandler` queda preparado para atrapar cualquier excepción.
- `UseRouting` encuentra que `POST /api/orders/checkout` corresponde a `OrdersController.Checkout`, gracias a:

  ```
  [Route("api/orders")]                 // en la clase
  [HttpPost("checkout")]                // en el método
  ```

- `UseCors` comprueba el origen.

Paso 3. Autenticación y autorización

`UseAuthentication` lee la cabecera `Authorization: Bearer ...` y valida el JWT: la firma con el secreto, el emisor, la audiencia y la caducidad. Si es válido, construye `HttpContext.User` (un `ClaimsPrincipal`) con los datos del token, entre ellos el *claim* `"Id"` con el identificador del usuario.

`UseAuthorization` ve que `OrdersController` tiene `[Authorize]`. Sin token válido, la petición termina aquí con **401**.

Paso 4. Model binding y validación

ASP.NET Core convierte el JSON del cuerpo en un objeto (*model binding*):

```
public sealed record CheckoutRequest
{
    [Range(typeof(decimal), "0", "100000000")]
    public decimal ExpectedTotal { get; init; }
}
```

Como el controlador tiene `[ApiController]`, **valida automáticamente** las anotaciones. Si `expectedTotal` fuera negativo, respondería **400** con la lista de errores sin llegar a ejecutar el método del controlador.

El `CancellationToken cancellationToken` del método se rellena solo: se cancela si el cliente cierra la conexión (8.6).

Paso 5. El controlador

```
[HttpPost("checkout")]
public async Task<IActionResult> Checkout(CheckoutRequest request, CancellationToken cancellationToken)
{
    var result = await orders.CheckoutAsync(request, User.UserId(), cancellationToken);

    return result.IsSuccess
        ? CreatedAtAction(nameof(GetById), new { id = result.Value.Id }, result.Value)
```

```
        : result.ToActionResult(this);
}
```

`User.UserId()` es un método de extensión (`Extensions/ClaimsPrincipalExtensions.cs`) que lee el *claim* `"Id"`. El controlador no decide nada: delega.

Paso 6. El servicio: `OrderService.CheckoutAsync`

Aquí está toda la lógica. Resumida:

1. **Busca el cliente** asociado al usuario. Si no existe → `checkout.client_not_found`. Si el perfil está incompleto → `checkout.profile_incomplete`.
2. **Abre una transacción dentro de una estrategia de reintentos:**

   ```
   var strategy = db.Database.CreateExecutionStrategy();
   return await strategy.ExecuteAsync(async () =>
   {
       await using var transaction = await db.Database.BeginTransactionAsync(cancellationToken);
       …
   });
   ```

   La conexión está configurada para reintentar ante fallos transitorios de red, y en ese caso hay que repetir **la transacción entera**, no solo la última sentencia. Por eso se envuelve en `ExecuteAsync`.
3. **Bloquea al cliente** con un bloqueo *advisory* de PostgreSQL:

   ```
   await db.Database.ExecuteSqlAsync(
       $"SELECT pg_advisory_xact_lock({CheckoutLockNamespace}, {client.ID})", cancellationToken);
   ```

   Si el mismo cliente pulsa "Pagar" dos veces, la segunda petición espera a que termine la primera y encontrará el carrito vacío. El bloqueo se libera solo al terminar la transacción.
4. **Lee el carrito con los productos** en una sola consulta (un `join` en LINQ).
5. **Comprueba**, en orden: que el carrito no esté vacío, que ningún producto esté archivado, y que el total calculado coincida con `expectedTotal` (si no, `checkout.price_changed`: el precio cambió desde que el cliente lo vio).
6. **Reserva stock** con un `UPDATE` condicional por producto:

   ```
   var affected = await db.Products
       .Where(p => p.Id == line.ProductId && p.Stock >= line.Quantity)
       .ExecuteUpdateAsync(s => s.SetProperty(p => p.Stock, p => p.Stock - line.Quantity),
   cancellationToken);
   ```

   Si `affected == 0`, otro cliente se llevó las últimas unidades → `checkout.insufficient_stock`, y la transacción se deshace entera.
7. **Crea el pedido** con sus líneas, copiando la dirección del cliente y los precios del momento. `OrderNumber` lo asigna PostgreSQL con una secuencia que empieza en 1000.
8. **Vacía el carrito** (`ExecuteDeleteAsync`) y hace **`COMMIT`**.
9. **Registra** `CheckoutCompleted` en el log y devuelve `Result.Ok(OrderResponse)`.

Si cualquier comprobación falla, se hace `RollbackAsync` y se devuelve `Result.Fail(...)` con un `Error`. **No se lanza ninguna excepción** para un fallo esperado (8.1). Los detalles de transacciones y bloqueos están en el capítulo 14.

Paso 7. Vuelta al controlador y respuesta

- **Éxito:** `CreatedAtAction(...)` responde **201 Created**, con el pedido en el cuerpo y una cabecera `Location: /api/orders/{id}` que apunta a `GetById`.
- **Fallo:** `ToActionResult` traduce el `Error` a un código HTTP (por ejemplo, 409 para `checkout.insufficient_stock`) y a un cuerpo `ProblemDetails` (8.2).

El objeto se serializa a JSON con `System.Text.Json`: nombres en *camelCase* (`orderNumber`) y enums como texto (`"status": "Paid"`).

Paso 8. Salida

La respuesta recorre el middleware en sentido inverso. `RequestLoggingMiddleware` para el cronómetro y escribe:

```
POST /api/orders/checkout responded 201 in 184ms
```

Si hubiera tardado más de un segundo, la línea saldría como *warning* (`SlowRequest`).

---

8. Patrones transversales

Estas piezas no son de una funcionalidad concreta: aparecen en toda la API.

8.1 `Result<T>`: fallos esperados sin excepciones

Concepto. Una operación puede fallar por motivos **esperados** (no hay stock, el correo ya existe) o **inesperados** (se cayó la base de datos, un bug). Usar excepciones para los esperados tiene dos problemas: la firma del método no dice que puede fallar, y las excepciones son caras y ensucian los logs.

En Furnistore. `Shared/Common/Result.cs` define:

```
public enum ErrorType { Validation, NotFound, Conflict, Unauthorized, Forbidden, TooManyRequests,
Unexpected }

public sealed record Error(string Code, string Message, ErrorType Type)
{
    public static Error NotFound(string code, string message) => new(code, message,
ErrorType.NotFound);
    …
}

public class Result
{
    public bool IsSuccess { get; }
    public Error? Error { get; }
    public static Result<T> Ok<T>(T value) => …;
    public static Result<T> Fail<T>(Error error) => …;
}
```

Un servicio devuelve `Result<T>`: o un valor, o un `Error` con tres datos:

- **`Code`:** identificador estable para máquinas, como `checkout.insufficient_stock`. La web decide qué mostrar según este código.
- **`Message`:** texto para personas, en español.
- **`Type`:** qué clase de fallo es; determina el código HTTP.

Leer `Value` de un `Result` fallido lanza una excepción: obliga a comprobar `IsSuccess` antes.

La traducción a HTTP está en un solo lugar, `Extensions/ResultExtensions.cs`:

ErrorType	HTTP
`Validation`	400 Bad Request
`Unauthorized`	401 Unauthorized
`Forbidden`	403 Forbidden
`NotFound`	404 Not Found
`Conflict`	409 Conflict
`TooManyRequests`	429 Too Many Requests
`Unexpected`	500 Internal Server Error

Por qué así. La capa `Application` no sabe nada de HTTP: dice "esto es un conflicto" y la capa `API` decide que un conflicto es un 409. Las excepciones quedan para lo verdaderamente inesperado.

8.2 `ProblemDetails`: un único formato de error

Concepto. `ProblemDetails` es un formato estándar (RFC 9457, antes RFC 7807) para describir errores HTTP en JSON. ASP.NET Core lo trae integrado.

En Furnistore. Todos los errores tienen esta forma:

```
{
  "type": "https://tools.ietf.org/html/rfc9110#section-15.5.10",
  "title": "No hay stock suficiente de Silla Plegable Bambú: quedan 2.",
  "status": 409,
  "code": "checkout.insufficient_stock",
  "traceId": "0HN7…"
}
```

Hay cuatro sitios que producen errores, y todos usan este formato:

Origen	Cuándo	Qué añade
`ResultExtensions`	Un servicio devuelve `Result.Fail`	`title` = mensaje del `Error`; `code` = código del `Error`
`[ApiController]`	Falla la validación de un contrato	Un diccionario `errors` con los mensajes de cada campo (`ValidationProblemDetails`), status 400
`GlobalExceptionHandler`	Una excepción no controlada	Status 500, título genérico "Ocurrió un error inesperado."
Rate limiter (`RateLimitingExtensions`)	Demasiadas peticiones	Status 429, `code = "rate_limited"` y cabecera `Retry-After`

`AddProblemDetails` en `Program.cs` añade el `traceId` a los errores, el mismo identificador que aparece en el log de esa petición, para poder cruzar un error que ve el usuario con su línea de log.

`GlobalExceptionHandler` (`Middleware/GlobalExceptionHandler.cs`) implementa `IExceptionHandler`: registra la excepción completa en el log, pero al cliente solo le devuelve un mensaje genérico. **Nunca se devuelve el detalle de una excepción al cliente**, porque puede revelar información interna. Si la excepción es una cancelación porque el cliente cerró la conexión, no hace nada.

8.3 Contratos (DTOs): nunca exponer entidades

Concepto. Un **DTO** (*Data Transfer Object*) es una clase que solo sirve para transportar datos de entrada o salida. Aquí se llaman "contratos".

En Furnistore. Los controladores nunca reciben ni devuelven entidades (`Order`, `Product`); siempre contratos (`CheckoutRequest`, `OrderResponse`). Se escriben como **records**:

```
public sealed record OrderResponse(
    int Id,
    int OrderNumber,
    OrderStatus Status,
    bool CanCancel,
    …
    IReadOnlyList<OrderLineResponse> Lines
);
```

Un `record` es una clase pensada para datos: se declara en una línea, es inmutable si usa `init`, y dos records con los mismos valores se consideran iguales. En los contratos de entrada se usa `required` para que el compilador exija rellenar el campo y `init` para que no se pueda cambiar después de crearse.

Por qué no exponer entidades:

- **Seguridad:** devolver `Client` expondría `UserId` y `Version`; aceptar `Product` en un `PUT` permitiría a un cliente enviar campos que no debería tocar.
- **Estabilidad:** si cambia una tabla, el contrato de la API no cambia por accidente.
- **Forma:** la respuesta puede tener otra estructura que la tabla. `OrderResponse` incluye `CanCancel` y la URL de la imagen de cada línea, que no son columnas de `Orders`.

La conversión entidad → contrato se escribe a mano (por ejemplo, `Orders/OrderMapping.cs`), sin librerías como AutoMapper. Es más código, pero es explícito y el compilador detecta un campo olvidado.

8.4 Validación en capas

Cada tipo de regla se comprueba en el lugar más adecuado:

Nivel	Dónde	Ejemplo	Respuesta
1. Formato	Anotaciones en los contratos	`[Required]`, `[StringLength(60, MinimumLength = 2)]`, `[Range(1, 100)]`, `[RegularExpression(...)]`	400 automático
2. Entre campos	`IValidatableObject` en el contrato	`ProductQuery`: `minPrice` no puede ser mayor que `maxPrice`	400 automático
3. Atributo propio	Clase que hereda de `ValidationAttribute`	`[IdentityCard]`: 11 dígitos y fecha válida	400 automático
4. Negocio	Servicios, con `Result.Fail`	Stock suficiente, transición de estado válida, carnet no repetido	400/404/409
5. Base de datos	Restricciones `CHECK`, `UNIQUE`, claves foráneas	`Price > 0`, `Stock >= 0`	Última defensa

Un atributo de validación propio es corto. Este es el del carnet (`Application/Clients/IdentityCardAttribute.cs`):

```
public sealed class IdentityCardAttribute()
    : ValidationAttribute("El carnet de identidad debe tener 11 dígitos y empezar por una fecha de
nacimiento válida y no futura.")
{
    public override bool IsValid(object? value) => value is null || (value is string text &&
IsValidNumber(text));
    …
}
```

y se usa así:

```
[Required, IdentityCard]
public required string IdentityCard { get; init; }
```

Por qué en capas. Las reglas de formato no necesitan base de datos y deben rechazarse cuanto antes. Las de negocio necesitan datos ("¿hay stock?") y viven en el servicio. Las restricciones de la base de datos protegen los datos incluso de un error en el código o de alguien que escriba SQL a mano.

8.5 Paginación y ordenación

Los listados nunca devuelven toda la tabla. El cliente pide `?page=2&pageSize=20` y recibe:

```
public sealed record PagedResult<T>(IReadOnlyList<T> Items, int Total, int Page, int PageSize)
{
    public int TotalPages => …;
}
```

En el servicio, el patrón es siempre contar el total, ordenar, saltar y tomar:

```
var total = await orders.CountAsync(cancellationToken);
var items = await orders
    .OrderByDescending(o => o.PlacedAt)
    .Skip((query.Page - 1) * query.PageSize)
    .Take(query.PageSize)
    .ToListAsync(cancellationToken);
```

`PageSize` está limitado con `[Range(1, 100)]` para que nadie pida un millón de filas.

Los listados del panel aceptan `?sort=price` o `?sort=-price` (el `-` invierte el orden). `Common/SortMap.cs` traduce ese texto a un `OrderBy` **solo** sobre campos permitidos:

```
private static readonly SortMap<Product> Sorts = new SortMap<Product>()
    .Add("name", p => p.Name)
    .Add("price", p => p.Price)
    .Add("stock", p => p.Stock)
    .Add("createdAt", p => p.CreatedAt);
```

Un campo desconocido devuelve `query.invalid_sort` con la lista de opciones válidas. Así el cliente no puede ordenar por columnas arbitrarias, y no hay que escribir un `switch` en cada servicio.

8.6 Asincronía, `CancellationToken` y `AsNoTracking`

`async`/`await`. Todo acceso a la base de datos o a la red es asíncrono (`ToListAsync`, `SaveChangesAsync`). Mientras se espera la respuesta de PostgreSQL, el hilo queda libre para atender otras peticiones. Un método `async` devuelve `Task<T>`, y quien lo llama usa `await`. Regla práctica: si un método llama a algo `async`, él también es `async`, hasta el controlador.

`CancellationToken`. Los métodos de los controladores lo reciben como último parámetro y ASP.NET Core lo conecta con la conexión del cliente. Se pasa hacia abajo hasta cada llamada a EF Core. Si el usuario cierra la pestaña, la consulta en curso se cancela en lugar de seguir gastando la base de datos para una respuesta que nadie leerá. La API lo pasa en más de un centenar de métodos.

`AsNoTracking()`. Por defecto EF Core "sigue" cada entidad que lee, para detectar cambios y guardarlos en `SaveChangesAsync`. En una consulta de solo lectura ese seguimiento es memoria y tiempo perdidos. `AsNoTracking()` lo desactiva:

```
var order = await db.Orders.AsNoTracking()
    .Include(o => o.OrderDetails)
    .FirstOrDefaultAsync(o => o.Id == id && o.ClientId == clientId, cancellationToken);
```

Regla: `AsNoTracking()` en todo lo que solo se lee; sin él cuando se va a modificar y guardar la entidad.

8.7 Búsquedas de texto seguras

Las búsquedas (`?search=mesa`) usan `ILIKE` de PostgreSQL, que ignora mayúsculas. Como en `LIKE` los caracteres `%` y `_` son comodines, `Common/LikePattern.cs` los escapa antes de construir el patrón. Lo usan las búsquedas del panel; el catálogo público (`ProductService`) tiene su propio método `EscapeLikePattern`, que hace lo mismo y podría sustituirse por `LikePattern`:

```
public static string Contains(string value) =>
    $"%{value.Trim().Replace("\\", "\\\\").Replace("%", "\\%").Replace("_", "\\_")}%";
```

Sin esto, buscar `%` devolvería todos los productos. No hay riesgo de inyección SQL: EF Core envía el valor como parámetro, nunca concatenado en el SQL.

8.8 Serialización JSON

`AddApiContract()` configura `System.Text.Json`, el serializador integrado en .NET:

- Las propiedades se escriben en *camelCase* (`OrderNumber` → `orderNumber`), el comportamiento por defecto de ASP.NET Core.
- Los enums se escriben como texto gracias a `JsonStringEnumConverter`: `"status": "Paid"` en lugar de `"status": 0`. Es más legible y no se rompe si cambia el orden del enum.

8.9 Detalles de C# que verás en todo el código

Sintaxis	Qué significa
`public sealed class X`	Nadie puede heredar de `X`. Se usa por defecto salvo que la herencia sea intencionada.
`class X(Dep dep)`	Constructor primario: `dep` está disponible en toda la clase.
`record`	Clase de datos con igualdad por valor (8.3).
`required`, `init`	Propiedad obligatoria al crear el objeto; solo se puede asignar al crearlo.
`string?` frente a `string`	Puede ser null / nunca es null (2.3).
`x!`	"Sé que `x` no es null aquí".
`x ?? y`	Si `x` es null, usa `y`.

Sintaxis	Qué significa
`is int id`	Comprueba el tipo y, si coincide, lo guarda en `id`.
`switch` con `=>`	*Switch expression*: devuelve un valor según el caso (ver `StatusFor` en 8.1).
`[OrderStatus.Paid]`, `[]`	Expresión de colección: crea una lista o un array con esos elementos.
`value[..6]`, `label[1..]`	Rango: los 6 primeros caracteres / todo desde el segundo.
`=> …` en un método	Método de una sola expresión.
`$"Texto {valor}"`	Cadena interpolada. En `ExecuteSqlAsync` se convierte en parámetros SQL, no en texto concatenado.

---

9. Logging

9.1 Concepto: abstracción y proveedor

.NET separa **cómo se escribe un log** de **adónde va**:

- El código usa la interfaz `ILogger<T>` (paquete `Microsoft.Extensions.Logging.Abstractions`). La pide por inyección y llama a `LogInformation`, `LogWarning`, `LogError`…
- Un **proveedor** decide qué se hace con cada mensaje: consola, archivo, un servicio externo. Furnistore usa **NLog**.

Por eso `Application` solo referencia el paquete de abstracciones: no sabe que existe NLog. Cambiar de proveedor sería cambiar `Program.cs` y la configuración, sin tocar los servicios.

9.2 Logs estructurados

```
logger.LogInformation(
    ApiEvents.CheckoutCompleted,
    "Checkout created order {OrderId} ({OrderNumber}) for client {ClientId}, total {Total}",
    order.Id, order.OrderNumber, order.ClientId, order.Total);
```

Las `{llaves}` **no son interpolación**: son nombres de propiedades. El proveedor recibe el mensaje y además los valores con su nombre (`OrderId = 57`), de modo que en un sistema de logs se puede filtrar por `OrderId` en lugar de buscar texto. Por eso nunca se escribe `$"... {order.Id}"` en un log.

9.3 El catálogo de eventos

`Application/Common/ApiEvents.cs` da un número fijo (`EventId`) a cada suceso relevante:

```
public static readonly EventId CheckoutCompleted = new(3004, nameof(CheckoutCompleted));
public static readonly EventId OrderCancelled    = new(3007, nameof(OrderCancelled));
```

Rango	Área
1001–1311	Catálogo, clientes y carrito
2001–2022	Autenticación y correo

Rango	Área
3004–3013	Pedidos
5001–5010	Infraestructura: peticiones, excepciones, arranque, base de datos

Por qué. Un número estable permite contar o alertar sobre un tipo de suceso ("cuántos `CheckoutRejected` hubo hoy") aunque cambie el texto del mensaje. Y tenerlos en un solo archivo evita números repetidos.

9.4 Qué nivel usar

La API sigue esta política:

Nivel	Cuándo	Ejemplos
`Information`	Algo salió bien y es relevante	`CheckoutCompleted`, `LoginSucceeded`, cada petición completada
`Warning`	Algo se rechazó por una regla, o algo va lento	`CheckoutRejected`, `LoginFailed`, `SlowRequest`, `RequestRateLimited`, configuración incompleta
`Error`	Algo falló de forma inesperada	`UnhandledException`

Que un cliente se quede sin stock no es un error del sistema: es un *warning*. Reservar `Error` para lo inesperado permite alertar sobre los `Error` sin ruido.

9.5 El middleware de log de peticiones

`Middleware/RequestLoggingMiddleware.cs` hace tres cosas en cada petición:

1. Abre un **scope** con `TraceId` y `UserId`. Todo log escrito durante la petición, desde cualquier servicio, lleva esos dos datos sin que el servicio los pase.
2. Al terminar, escribe método, ruta, status y milisegundos. Más de 1.000 ms → `Warning`.
3. **Oculta** los valores de parámetros sensibles en la URL (`code`, `token`, `refreshToken`, `password`) como `***`. Un log nunca debe contener secretos.

9.6 `nlog.config`

El archivo `API.Furnistore.API/nlog.config` define **destinos** (*targets*) y **reglas** (*rules*):

- **En Development:** consola con colores. Los logs propios (`API.Furnistore.*`) desde nivel `Debug`, con sus propiedades estructuradas; el SQL que ejecuta EF Core (`Microsoft.EntityFrameworkCore.Database.Command`) desde `Info`, útil para ver qué consultas genera cada LINQ; y de las librerías (`Microsoft.*`, `Npgsql.*`, `System.*`) solo `Warning` o más.
- **En Production:** una línea **JSON** por log en la salida estándar, con fecha UTC, nivel, propiedades y scope. Es el formato que esperan Docker y los sistemas de recogida de logs.

El cambio de formato se decide dentro del propio `nlog.config` con la variable `isDevelopment`, que lee el entorno de ASP.NET Core.

Parte III. Base de datos

Esta parte explica dónde viven los datos de Furnistore, cómo los lee y escribe la API, cómo cambia la estructura de la base con el tiempo y cómo se evita que dos personas estropeen los mismos datos a la vez. Empieza por los diagramas, porque todo lo demás se entiende mejor con el dibujo delante.

10. Modelo de datos completo

10.1 Los diagramas

Los tres diagramas están en la carpeta `guia-api-diagramas/`, junto a esta guía, en PNG para leerlos y en SVG para ampliarlos sin perder nitidez. Los genera el script `generar-diagramas-bd.py` de la misma carpeta, que **lee el esquema real de la base local**: no hay nada dibujado a mano, así que lo que ves es exactamente lo que hay. Si mañana se añade una migración, basta con volver a ejecutarlo desde la raíz del repositorio, con la base local en marcha:

```
python3 docs/guia-api-diagramas/generar-diagramas-bd.py docs/guia-api-diagramas
```

Diagrama 1. Mapa general. Las 17 tablas agrupadas por área, con una flecha por cada clave foránea. Sirve para orientarse.

Mapa general de la base de datos

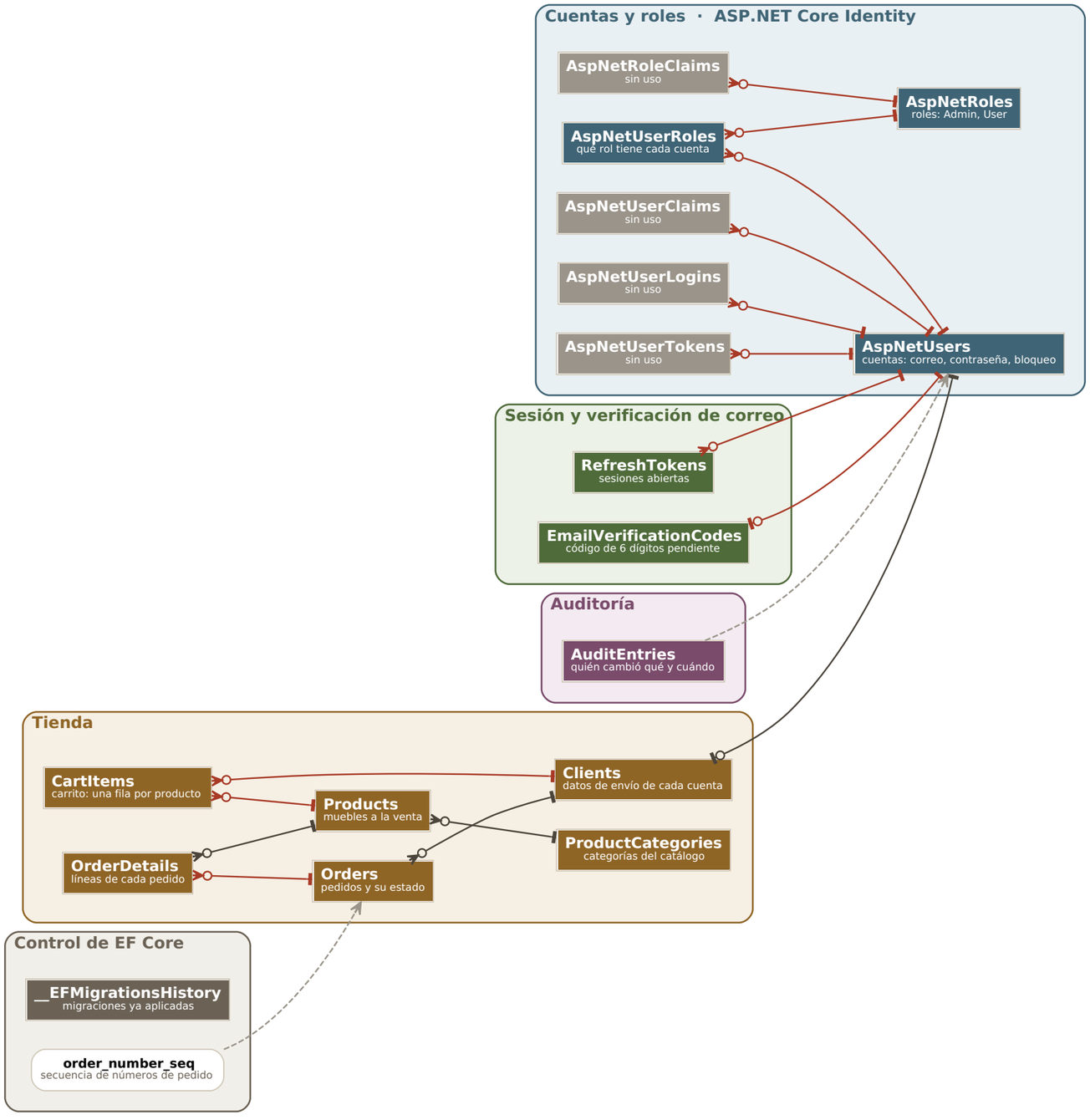


Diagrama 2. La tienda en detalle. Catálogo, clientes, carrito y pedidos con todas sus columnas, sus tipos, sus restricciones y la regla de borrado de cada relación. Es el diagrama más importante.

La tienda en detalle

Furnistore: la tienda en detalle

Catálogo, clientes, carrito y pedidos · todas las columnas, restricciones y reglas de borrado

Cómo leer el diagrama	
PK	clave primaria: identifica cada fila
FK	clave foránea: apunta a la PK de otra tabla
UQ	no puede repetirse en la tabla
tipo ?	la columna admite NULL (vacío)
xmin	columna oculta de PostgreSQL, usada como versión
—|— · —<	lado «uno» · lado «muchos» (pata de gallo)
o	el lado puede no tener ninguna fila
CASCADE	borrar el padre borra también los hijos
RESTRICT	no se puede borrar el padre si tiene hijos
- - -	relación que la base no comprueba

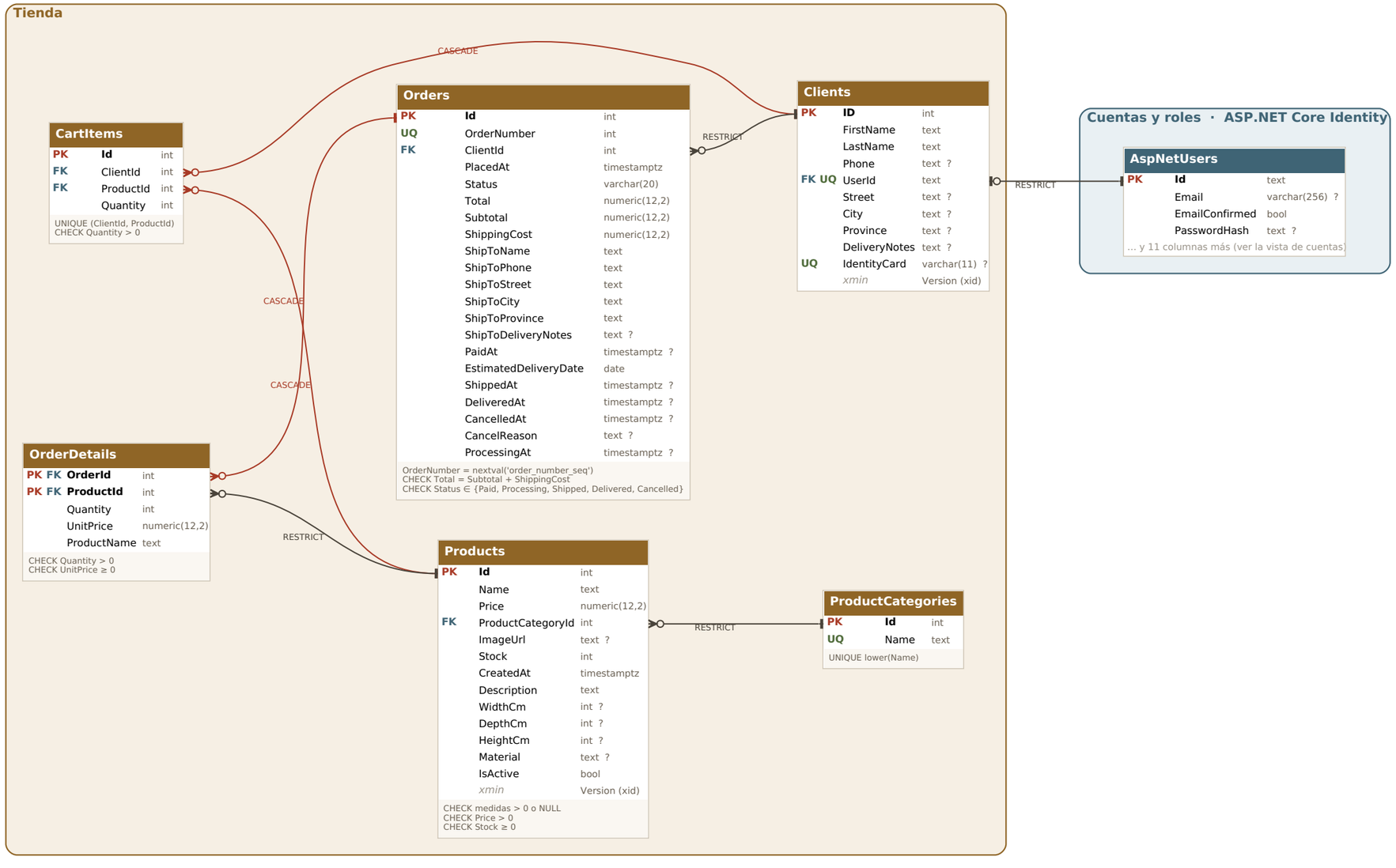


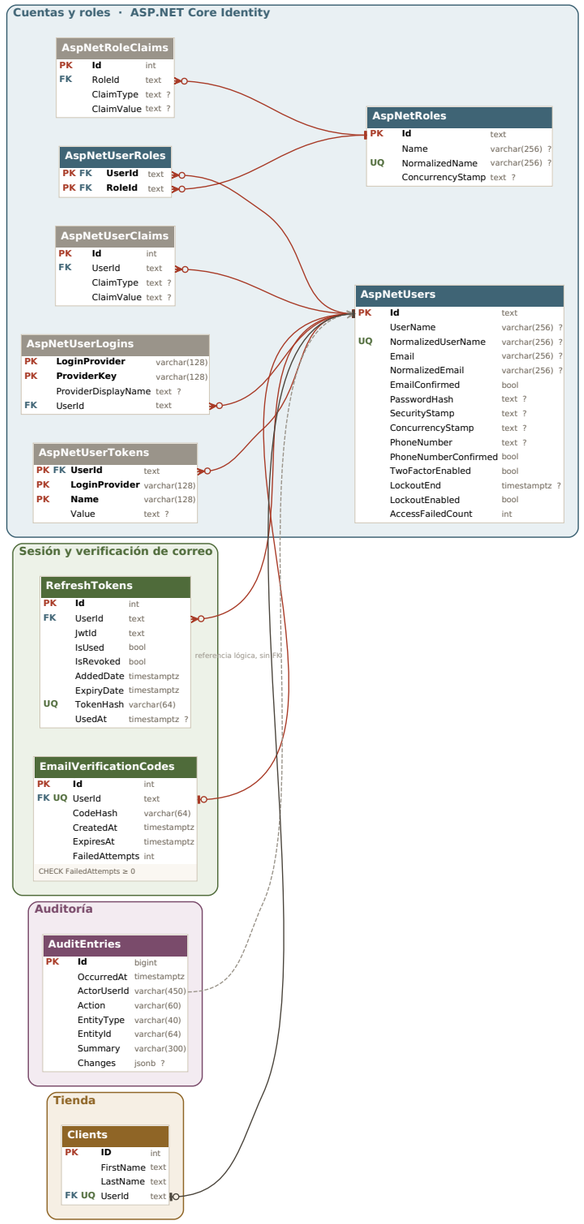
Diagrama 3. Cuentas, sesiones y auditoría. Las tablas de ASP.NET Core Identity y las tablas propias de la API que dependen de una cuenta.

Cuentas, sesiones y auditoría

Furnistore: cuentas, sesiones y auditoría

Tablas de ASP.NET Core Identity y las propias de la API · línea roja = CASCADE, gris oscura = RESTRICT · las tablas grises vienen con Identity pero no se usan

Cómo leer el diagrama	
PK	clave primaria: identifica cada fila
FK	clave foránea: apunta a la PK de otra tabla
UQ	no puede repetirse en la tabla
tipo ?	la columna admite NULL (vacío)
xmin	columna oculta de PostgreSQL, usada como versión
—|— · —<	lado «uno» · lado «muchos» (pata de gallo)
o	el lado puede no tener ninguna fila
CASCADE	borrar el padre borra también los hijos
RESTRICT	no se puede borrar el padre si tiene hijos
- - -	relación que la base no comprueba



Cómo leerlos. Cada caja es una tabla y cada fila, una columna. A la izquierda va el papel de la columna (`PK`, `FK`, `UQ`), en el centro su nombre y a la derecha su tipo en PostgreSQL; un `?` tras el tipo significa que admite `NULL`. Las líneas usan la notación "pata de gallo": el extremo con tres patas es el lado "muchos" (el de la clave foránea) y el extremo con una raya es el lado "uno". Un círculo indica que ese lado puede no tener ninguna fila: una categoría puede no tener productos todavía. El color dice qué pasa al borrar: rojo es `CASCADE`, gris oscuro es `RESTRICT` (10.4).

10.2 Las áreas de un vistazo

Área	Tablas	Quién las escribe
Tienda	`ProductCategories`, `Products`, `Clients`, `CartItems`, `Orders`, `OrderDetails`	Los servicios de `Application`: catálogo, carrito, checkout, perfil y panel
Cuentas y roles	`AspNetUsers`, `AspNetRoles`, `AspNetUserRoles` y cuatro tablas de Identity sin uso	ASP.NET Core Identity (`UserManager`, `RoleManager`) y el panel de clientes
Sesión y verificación	`RefreshTokens`, `EmailVerificationCodes`	`AuthService`
Auditoría	`AuditEntries`	`AuditLog`, desde los servicios del panel y el checkout
Control de EF Core	`__EFMigrationsHistory` y la secuencia `order_number_seq`	Las migraciones

Hay una separación importante que el diagrama 1 deja ver: **la cuenta y el cliente son tablas distintas**. `AspNetUsers` guarda lo necesario para iniciar sesión (correo, hash de la contraseña, bloqueos) y la gestiona Identity. `Clients` guarda lo necesario para vender (nombre, teléfono, carnet, dirección) y la gestiona la API. Se unen uno a uno por `Clients.UserId`. Así Identity puede cambiar su esquema en una versión futura sin tocar los datos de la tienda, y la tienda no mezcla datos de envío con datos de seguridad.

10.3 Tabla por tabla

`ProductCategories`

Las categorías del catálogo (Sillas, Mesas, Estanterías, Lámparas). Solo tiene `Id` y `Name`.

- El nombre no se puede repetir **sin distinguir mayúsculas**: "Mesas" y "mesas" chocan. Lo garantiza el índice único `UX_ProductCategories_Name_Lower`, creado sobre `lower("Name")`. EF Core no sabe declarar un índice sobre una expresión, así que se creó con SQL dentro de la migración `ReshapeDomainModel` (13.5).

`Products`

Los muebles a la venta.

- **Dinero:** `Price` es `numeric(12,2)`: hasta 9.999.999.999,99 con exactamente dos decimales. Nunca se usa `float` ni `double` para dinero, porque redondean en binario (`0.1 + 0.2` no da `0.3`). En C# es `decimal`. Los precios están en USD: el 16 de octubre se pasaron a CUP y el 17 se devolvieron a USD (13.5).
- **Restricciones `CHECK`:** precio mayor que 0, stock mayor o igual que 0, y medidas positivas o vacías. Aunque la API ya valida todo esto, la base lo vuelve a comprobar: si un error de programación intentara guardar stock negativo, PostgreSQL lo rechazaría.

- **`IsActive`:** en lugar de borrar un producto que ya se vendió, se archiva (`false`). Desaparece del catálogo, pero los pedidos antiguos siguen apuntando a él.
- **`CreatedAt`:** lo rellena la propia base con `now()` al insertar.
- **`xmin`:** no aparece entre las columnas normales porque es una columna oculta que PostgreSQL tiene en todas las tablas. El modelo la usa como número de versión del producto para detectar ediciones simultáneas (14.6).

`Clients`

Los datos de cada cliente para vender y enviar.

- Su clave primaria se llama `ID`, en mayúsculas, a diferencia del resto (`Id`). Es una herencia de la primera migración; cambiarla obligaría a renombrar la columna y todas sus referencias, y no aporta nada.
- `UserId` es único: una cuenta tiene como mucho un cliente. El registro crea siempre los dos a la vez.
- `Phone`, `Street`, `City` y `Province` admiten `NULL`, porque al registrarse solo se piden nombre, carnet y correo. `IdentityCard` también admite `NULL`: el registro lo pide, pero las cuentas creadas antes de ese cambio pueden no tenerlo. El checkout comprueba que todo esté completo (`IsProfileComplete`) antes de dejar comprar.
- `IdentityCard` es `varchar(11)` y único: dos clientes no pueden tener el mismo carnet. Si alguien lo intenta, la base lanza una violación de unicidad y la API la traduce a un mensaje claro (14.8).
- También usa `xmin` como versión, por el mismo motivo que `Products`.

`CartItems`

El carrito: una fila por cada producto que un cliente tiene en él.

- La pareja `(ClientId, ProductId)` es única: si añades una silla que ya estaba, se suma la cantidad a la fila existente en lugar de crear otra.
- `Quantity` debe ser mayor que 0. Quitar del carrito es borrar la fila, no dejarla en 0.
- Las dos claves foráneas son `CASCADE`: si se borra el cliente o el producto, su línea de carrito desaparece. El carrito es efímero; no hay nada que conservar.

`Orders`

Los pedidos. Es la tabla más rica del modelo.

- **Dos identificadores.** `Id` es la clave interna (1, 2, 3…). `OrderNumber` es el número que ve el cliente ("Pedido #1004"): lo da la secuencia `order_number_seq`, que empezó en 1000. Separarlos permite que el número público tenga un formato propio sin depender de la clave interna.
- **Estado.** `Status` es texto (`Paid`, `Processing`, `Shipped`, `Delivered`, `Cancelled`) y un `CHECK` impide cualquier otro valor. En C# es el enum `OrderStatus`; EF Core lo convierte a texto con `HasConversion<string>()`. Guardarlo como texto hace la base legible con cualquier herramienta y no se rompe si cambia el orden del enum.
- **Importes.** `Subtotal`, `ShippingCost` y `Total` son `numeric(12,2)`, y un `CHECK` exige `Total = Subtotal + ShippingCost`. Un total incoherente es imposible de guardar.
- **Copia de la dirección.** Las columnas `ShipTo*` copian el nombre, el teléfono y la dirección del cliente **en el momento de comprar**. Si mañana el cliente se muda y cambia su perfil, el pedido de ayer conserva la dirección a la que se envió. Un pedido es un documento histórico, no una vista del perfil actual.
- **Fechas del ciclo de vida.** `PlacedAt`, `PaidAt`, `ProcessingAt`, `ShippedAt`, `DeliveredAt` y `CancelledAt`. Todas son `timestamptz` y se guardan en UTC; la web las muestra en hora de La Habana. `EstimatedDeliveryDate` es `date`, sin hora.

- **Índices.** `(ClientId, PlacedAt)` acelera "Mis pedidos, los más recientes primero"; `(Status, PlacedAt)` acelera el panel cuando filtra por estado.

`OrderDetails`

Las líneas de cada pedido: qué producto, cuántas unidades y a qué precio.

- **Clave compuesta** `(OrderId, ProductId)`: un mismo producto aparece una sola vez por pedido, con su cantidad total.
- **Copia del precio y del nombre.** `UnitPrice` y `ProductName` guardan los valores de ese momento. Si mañana la silla sube de precio o cambia de nombre, la factura de ayer no cambia. Es el mismo principio que las columnas `ShipTo*`.
- Hacia `Orders` es `CASCADE`: las líneas no tienen sentido sin su pedido. Hacia `Products` es `RESTRICT`: un producto vendido no se puede borrar (10.4).

`AspNetUsers`, `AspNetRoles` y `AspNetUserRoles`

Las crea ASP.NET Core Identity; la Parte IV las explica en detalle. Lo esencial:

- `AspNetUsers` es la cuenta. Su `Id` es un texto (un GUID), no un número. Guarda el correo, si está confirmado (`EmailConfirmed`), el hash de la contraseña, el control de bloqueo (`LockoutEnd`, `AccessFailedCount`) y dos sellos: `SecurityStamp` cambia cuando cambia algo de seguridad de la cuenta y `ConcurrencyStamp` detecta ediciones simultáneas.
- `NormalizedEmail` y `NormalizedUserName` son el correo en mayúsculas. Identity busca siempre por ellos, con índice, para que `Ana@Gmail.com` y `ana@gmail.com` sean la misma cuenta.
- `AspNetRoles` tiene dos filas, `Admin` y `User`, que crea `Program.cs` al arrancar si no existen. `AspNetUserRoles` dice qué rol tiene cada cuenta.

Las cuatro tablas grises

`AspNetUserClaims`, `AspNetUserLogins`, `AspNetUserTokens` y `AspNetRoleClaims` vienen con Identity y están vacías. Servirían para permisos finos (*claims*), inicio de sesión con Google o Facebook (*logins*) y tokens propios de Identity. Furnistore no usa nada de eso, pero no se eliminan: forman parte del esquema estándar de `IdentityDbContext` y quitarlas exigiría personalizar Identity sin ningún beneficio.

`RefreshTokens`

Una fila por cada sesión abierta (16 en la Parte IV).

- Nunca se guarda el token en sí, sino su hash SHA-256 (`TokenHash`, único). Si alguien robara una copia de la base, no podría usar esos tokens.
- `IsUsed` y `UsedAt` marcan que un token ya se canjeó por otro; `IsRevoked` lo anula al cerrar sesión o al desactivar la cuenta.

`EmailVerificationCodes`

El código de 6 dígitos pendiente de cada cuenta, guardado también como hash.

- `UserId` es único: como mucho hay un código vivo por cuenta. Pedir otro borra el anterior.
- El código caduca a los 15 minutos (`ExpiresAt`).
- `FailedAttempts` cuenta los intentos fallidos; con 5 se borra el código y hay que pedir uno nuevo.

`AuditEntries`

El registro de auditoría: quién cambió qué y cuándo.

- `Id` es `bigint`, no `int`, porque es la tabla que más crece.

- `EntityType` y `EntityId` dicen sobre qué se actuó (`Order` y `57`, `Product` y `12`…). Como una entrada puede apuntar a tablas distintas, **no hay clave foránea**: una FK solo puede apuntar a una tabla.
- `ActorUserId` tampoco es FK, aunque normalmente sea el `Id` de una cuenta. Puede valer `console` (una acción desde la línea de comandos) y, sobre todo, **la auditoría debe sobrevivir a la cuenta**: si algún día se borrara un usuario, su rastro no debe desaparecer. Por eso en el diagrama 3 la línea es discontinua.
- `Changes` es `jsonb`: un objeto con el antes y el después de cada campo, por ejemplo `{"price": {"from": "79200", "to": "96000"}}`. Una tabla con una columna por campo auditado sería imposible de mantener; `jsonb` admite cualquier forma y PostgreSQL puede consultarlo.

`__EFMigrationsHistory`

La lleva EF Core: una fila por migración aplicada (13.1). No se toca nunca a mano.

10.4 Qué pasa al borrar: `CASCADE` frente a `RESTRICT`

Cada clave foránea tiene una regla para cuando se borra la fila a la que apunta:

Relación	Regla	Por qué
`CartItems` → `Clients`, `Products`	CASCADE	El carrito es temporal; sin cliente o sin producto no significa nada
`OrderDetails` → `Orders`	CASCADE	Las líneas son parte del pedido
`OrderDetails` → `Products`	**RESTRICT**	Un producto vendido no se puede borrar: se archiva
`Orders` → `Clients`	**RESTRICT**	Un cliente con pedidos no se puede borrar: se desactiva su cuenta
`Products` → `ProductCategories`	**RESTRICT**	No se borra una categoría con productos
`Clients` → `AspNetUsers`	**RESTRICT**	No se borra una cuenta que tiene datos de cliente
`RefreshTokens`, `EmailVerificationCodes` → `AspNetUsers`	CASCADE	Sesiones y códigos mueren con la cuenta
Tablas de Identity entre sí	CASCADE	Así lo define Identity

La regla práctica: **`CASCADE` para datos que no valen nada sin su padre y `RESTRICT` para todo lo que tenga valor histórico o contable.** La API además comprueba estas situaciones antes de intentar el borrado y devuelve un mensaje claro ("No se puede borrar un producto que ya aparece en pedidos. Archívalo para ocultarlo del catálogo."), pero la regla en la base es la red de seguridad.

10.5 Detalles y herencias que conviene conocer

Al leer la base con una herramienta como `psql` verás algunas cosas que no salen en el código C#:

- **Valores por defecto heredados.** `Products.Stock` tiene `50` como valor por defecto, `Orders.EstimatedDeliveryDate` tiene `-infinity` y varias columnas de dinero tienen `0.0`. No son reglas del negocio: son restos de las migraciones que añadieron esas columnas a tablas que ya tenían filas. Al añadir una columna `NOT NULL`, PostgreSQL necesita un valor para las filas existentes, y EF Core lo deja como valor por defecto. La API siempre envía el valor real, así que no afectan.

- **Columnas `text` sin longitud.** `FirstName`, `LastName`, `Street` y otras de `Clients` son `text` sin límite en la base. El límite (60 caracteres para el nombre, 200 para la calle…) lo pone la validación de la API (8.4). En PostgreSQL `text` y `varchar(n)` rinden igual, así que no hay penalización.
- **Las claves primarias numéricas** son `GENERATED BY DEFAULT AS IDENTITY`: la base asigna el siguiente número al insertar. Es la forma moderna en PostgreSQL de lo que antes era `serial`.

---

11. EF Core desde cero

11.1 Concepto: un ORM

Una base de datos relacional habla SQL y guarda filas; un programa en C# trabaja con objetos. Un **ORM** (*Object-Relational Mapper*) traduce entre los dos mundos: tú escribes consultas en C# y él genera el SQL, ejecuta la consulta y convierte las filas en objetos.

Entity Framework Core (EF Core) es el ORM oficial de .NET. Sus piezas:

Pieza	Qué es
Entidad	Una clase normal de C# que representa una tabla: `Product`, `Order`
`DbContext`	La sesión con la base: sabe qué tablas hay, abre conexiones y guarda los cambios
`DbSet<T>`	Una tabla vista como colección: `db.Products`
Proveedor	El traductor para un motor concreto. Furnistore usa `Npgsql.EntityFrameworkCore.PostgreSQL`
Configuración del modelo	Las reglas de cómo se mapea cada clase: claves, longitudes, relaciones, índices
Migraciones	El historial de cambios del esquema, en C# (capítulo 13)

11.2 En Furnistore: el `DbContext`

`API.Furnistore.Data/APIFurnistoreContext.cs`:

```
public class APIFurnistoreContext : IdentityDbContext
{
    public APIFurnistoreContext(DbContextOptions options) : base(options) { }

    public DbSet<Client> Clients { get; set; }
    public DbSet<Product> Products { get; set; }
    public DbSet<Order> Orders { get; set; }
    public DbSet<ProductCategory> ProductCategories { get; set; }
    public DbSet<OrderDetail> OrderDetails { get; set; }
    public DbSet<RefreshToken> RefreshTokens { get; set; }
    public DbSet<CartItem> CartItems { get; set; }
    public DbSet<EmailVerificationCode> EmailVerificationCodes { get; set; }
    public DbSet<AuditEntry> AuditEntries { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.HasSequence<int>("order_number_seq").StartsAt(1000);

        modelBuilder.ApplyConfigurationsFromAssembly(typeof(APIFurnistoreContext).Assembly);
```

```
    }
}
```

Tres cosas a fijarse:

1. **Hereda de `IdentityDbContext`, no de `DbContext`.** Esa clase base ya trae los `DbSet` de las tablas `AspNet*`, y por eso `db.Users` o `db.UserRoles` funcionan sin declararlos. La llamada a `base.OnModelCreating` es la que configura esas tablas: si se olvida, Identity deja de funcionar.
2. **`HasSequence`** declara la secuencia de números de pedido, que luego usa `Orders.OrderNumber`.
3. **`ApplyConfigurationsFromAssembly`** busca en el proyecto todas las clases que implementan `IEntityTypeConfiguration<T>` y las aplica. Cada entidad tiene su archivo en `Data/Configurations/`, en lugar de un `OnModelCreating` de 300 líneas.

11.3 Convención frente a configuración

EF Core deduce casi todo por **convención**, sin que se lo digas:

Si en C# hay…	EF Core entiende…
Una propiedad `Id` o `<Clase>Id`	La clave primaria, autonumérica
`int ProductCategoryId` junto a `ProductCategory Category`	Una clave foránea hacia `ProductCategories`
`string Name` (no anulable)	Columna `text NOT NULL`
`string? Material`	Columna `text` que admite `NULL`
`int? WidthCm`	Columna `integer` que admite `NULL`
`List<OrderDetail> OrderDetails` en `Order`	Relación uno a muchos, con borrado en cascada

Lo que la convención no puede adivinar se dice con la **Fluent API**, en las clases de configuración. Por ejemplo, `Configurations/OrderConfiguration.cs`:

```
public void Configure(EntityTypeBuilder<Order> builder)
{
    builder.Property(order => order.OrderNumber).HasDefaultValueSql("nextval('order_number_seq')");
    builder.HasIndex(order => order.OrderNumber).IsUnique();

    builder.Property(order => order.Status).HasConversion<string>().HasMaxLength(20);
    builder.Property(order => order.Total).HasPrecision(12, 2);

    builder
        .HasOne<Client>()
        .WithMany()
        .HasForeignKey(order => order.ClientId)
        .OnDelete(DeleteBehavior.Restrict);

    builder.HasIndex(order => new { order.ClientId, order.PlacedAt });

    builder.ToTable(table =>
        table.HasCheckConstraint(
            "CK_Orders_Amounts",
            "\"Subtotal\" >= 0 AND \"ShippingCost\" >= 0 AND \"Total\" = \"Subtotal\" +
\"ShippingCost\""));
}
```

Cada línea se corresponde con algo visible en el diagrama 2: el valor por defecto de `OrderNumber`, su `UQ`, el `varchar(20)` de `Status`, el `numeric(12,2)`, la línea `RESTRICT` hacia `Clients`, el índice y el `CHECK`.

Por qué Fluent API y no atributos. EF Core también admite atributos sobre las propias entidades (`[MaxLength(20)]`, `[Key]`). Furnistore no los usa porque las entidades viven en `Shared` y no deben depender de EF Core (6.2): quien lea `Order.cs` ve un objeto de negocio limpio, y todo lo que es "cómo se guarda" está en `Data`.

11.4 De LINQ a SQL

Las consultas se escriben con **LINQ**, el lenguaje de consultas integrado en C#. Lo importante es que **no se ejecutan en C#**: EF Core traduce la expresión completa a SQL y la manda a PostgreSQL. Este es el catálogo público, `ProductService.SearchAsync`:

```
var products = db.Products.AsNoTracking().Where(p => p.IsActive);

if (!string.IsNullOrWhiteSpace(query.Search))
{
    var pattern = $"%{EscapeLikePattern(query.Search.Trim())}%";
    products = products.Where(p => EF.Functions.ILike(p.Name, pattern, "\\"));
}

var total = await products.CountAsync(cancellationToken);

var items = await products
    .OrderBy(p => p.Name)
    .ThenBy(p => p.Id)
    .Skip((query.Page - 1) * query.PageSize)
    .Take(query.PageSize)
    .Select(ToResponse)
    .ToListAsync(cancellationToken);
```

Y este es el SQL que EF Core generó para `GET /api/products?search=mesa&pageSize=4`, copiado del log de la API:

```
SELECT count(*)::int
FROM "Products" AS p
WHERE p."IsActive" AND p."Name" ILIKE @pattern ESCAPE '\'

SELECT p1."Id", p1."Name", p1."Description", p1."Price", p1."Stock", p0."Id", p0."Name",
       p1."ImageUrl", p1."WidthCm", p1."DepthCm", p1."HeightCm", p1."Material", p1."IsActive"
FROM (
    SELECT p."Id", p."DepthCm", p."Description", p."HeightCm", p."ImageUrl", p."IsActive",
           p."Material", p."Name", p."Price", p."ProductCategoryId", p."Stock", p."WidthCm"
    FROM "Products" AS p
    WHERE p."IsActive" AND p."Name" ILIKE @pattern ESCAPE '\'
    ORDER BY p."Name", p."Id"
    LIMIT @p0 OFFSET @p
) AS p1
INNER JOIN "ProductCategories" AS p0 ON p1."ProductCategoryId" = p0."Id"
ORDER BY p1."Name", p1."Id"
```

Lo que conviene notar:

- **Construcción por partes.** Cada `Where` añadido con un `if` no consulta nada; solo amplía la expresión. La consulta sale a la base al llegar a `CountAsync` y a `ToListAsync`. Por eso se pueden encadenar filtros opcionales sin escribir SQL dinámico.
- **Parámetros, nunca texto pegado.** El texto buscado viaja como `@pattern`, igual que la página (`@p0`, `@p`). Eso hace imposible la inyección SQL.
- **`Select(ToResponse)` se traduce también.** Como la respuesta incluye el nombre de la categoría, EF Core añade el `JOIN` con `ProductCategories` por su cuenta, y solo pide las columnas que el DTO usa.

- **`Skip` y `Take` se convierten en `OFFSET` y `LIMIT`.** La paginación la hace la base, no C#: nunca se traen todos los productos a memoria para quedarse con cuatro.
- **El orden es estable.** `ThenBy(p => p.Id)` desempata dos productos con el mismo nombre; sin él, una misma fila podría salir en dos páginas distintas.

Para verlo tú. En Development, `nlog.config` muestra en consola cada SQL que ejecuta EF Core (9.6). Levanta la API, navega por la web y mira la terminal: es la mejor forma de entender qué hace cada consulta LINQ.

11.5 Seguimiento de cambios y `SaveChangesAsync`

Cuando EF Core lee entidades sin `AsNoTracking()`, las registra en su **seguimiento de cambios** (*change tracker*). Si después modificas una propiedad y llamas a `SaveChangesAsync`, EF Core compara con el valor original y genera solo el `UPDATE` de lo que cambió:

```
var client = await db.Clients.FirstOrDefaultAsync(c => c.UserId == userId, cancellationToken);
client.Phone = "+5351234567";
await db.SaveChangesAsync(cancellationToken);
// SQL generado, simplificado: UPDATE "Clients" SET "Phone" = @p0 WHERE "ID" = @p1 AND xmin = @p2
```

`SaveChangesAsync` envía **todos** los cambios pendientes en una sola transacción: si guardas un pedido y su entrada de auditoría y falla una, no se guarda ninguna. Es el patrón *unidad de trabajo*.

El `AND xmin = @p2` del ejemplo es el control de concurrencia de `Clients` (14.6). Las otras dos herramientas de este apartado, `AsNoTracking()` para lecturas y `ChangeTracker.Clear()` para empezar de cero, se explican en 8.6 y en 14.3.

11.6 Cambios masivos sin cargar nada: `ExecuteUpdate` y `ExecuteDelete`

A veces no hace falta traer filas a memoria para cambiarlas. `ExecuteUpdateAsync` y `ExecuteDeleteAsync` generan un `UPDATE` o un `DELETE` directo:

```
await db.RefreshTokens
    .Where(t => t.UserId == userId && !t.IsRevoked && !t.IsUsed)
    .ExecuteUpdateAsync(s => s.SetProperty(t => t.IsRevoked, true), cancellationToken);
// SQL generado, simplificado: UPDATE "RefreshTokens" SET "IsRevoked" = TRUE WHERE "UserId" = @p AND
NOT "IsRevoked" AND NOT "IsUsed"
```

Se usan para vaciar el carrito tras el checkout, revocar sesiones, borrar códigos de verificación y, sobre todo, para **reservar stock** (14.4). Dos cosas a tener en cuenta:

- Se ejecutan **en el momento**, no al llamar a `SaveChangesAsync`.
- No pasan por el seguimiento de cambios: si tenías la entidad cargada, su copia en memoria queda desactualizada.

11.7 SQL escrito a mano

LINQ cubre casi todo, pero en tres sitios Furnistore escribe SQL directamente, porque necesita algo que LINQ no expresa:

Dónde	SQL	Para qué
`OrderWorkflow`	`SELECT * FROM "Orders" WHERE "Id" = {orderId} FOR UPDATE`	Bloquear el pedido mientras cambia de estado (14.5)
`OrderService`, `AdminCustomerService`	`SELECT pg_advisory_xact_lock(…)`	Bloqueos con nombre propio (14.4 y 14.7)

Dónde	SQL	Para qué
Migraciones	`CREATE UNIQUE INDEX … (lower("Name"))`, `UPDATE …`	Lo que el modelo no puede declarar (13.5)

Aunque parezca una cadena interpolada, `FromSql($"…{orderId}…")` y `ExecuteSqlAsync($"…")` **no** concatenan: EF Core convierte cada `{valor}` en un parámetro SQL. Se puede usar con datos del usuario sin riesgo de inyección. Lo peligroso sería `FromSqlRaw` con una cadena construida a mano, que el proyecto no usa.

11.8 Por qué EF Core

- **Frente a SQL a mano (ADO.NET o Dapper).** Dapper es más rápido de ejecutar y da control total, pero cada consulta se escribe y se mantiene a mano, y las migraciones habría que resolverlas aparte. Para una tienda con este volumen, la productividad de LINQ y las migraciones integradas pesan más que unos milisegundos.
- **Cuándo cambiaría la decisión.** Si una consulta concreta fuera lenta, EF Core permite bajar a SQL solo en ese punto (11.7), sin abandonar el ORM. Furnistore ya lo hace en tres sitios.

---

12. Conexión a PostgreSQL

12.1 Concepto

PostgreSQL es la base de datos relacional de Furnistore. Un programa se conecta a ella con una **cadena de conexión** que dice dónde está (servidor y puerto), qué base usar y con qué usuario. El controlador de .NET para PostgreSQL se llama **Npgsql**; EF Core lo usa por debajo a través de su proveedor.

Npgsql mantiene un **pool de conexiones**: abrir una conexión con PostgreSQL es caro (red, autenticación, un proceso nuevo en el servidor), así que Npgsql guarda las conexiones abiertas y las reutiliza. Cuando el código "abre" una conexión, en realidad toma una libre del pool, y al "cerrarla" la devuelve.

12.2 En Furnistore

La cadena se lee en `Program.cs` y se registra el `DbContext`:

```
var connectionString =
    Environment.GetEnvironmentVariable("DATABASE_URL")
    ?? builder.Configuration.GetConnectionString("APIFurnistoreContext")
    ?? throw new InvalidOperationException("DATABASE_URL not configured");

builder.Services.AddDbContext<APIFurnistoreContext>(options =>
    options.UseNpgsql(
        connectionString,
        npgsql =>
        {
            npgsql.EnableRetryOnFailure(
                maxRetryCount: 3,
                maxRetryDelay: TimeSpan.FromSeconds(2),
                errorCodesToAdd: null
            );
            npgsql.CommandTimeout(20);
        }
```

```
    )
);
```

- **`DATABASE_URL`**, en `apps/api/.env`, tiene el formato de Npgsql, con pares `Clave=Valor`:

  ```
  DATABASE_URL=Host=localhost;Port=5433;Database=furnistore;Username=furnistore;Password=furnistore
  ```

  Si no existe, se busca una cadena llamada `APIFurnistoreContext` en `appsettings`. Si tampoco, la API no arranca: es mejor fallar al inicio que en la primera petición.

- **`AddDbContext` registra el contexto como *scoped*** (4.2): cada petición HTTP recibe su propio `DbContext`, que se desecha al terminarla. Un `DbContext` no es seguro entre hilos ni está pensado para vivir mucho, y este tiempo de vida encaja exactamente con una petición.

- **`EnableRetryOnFailure`** reintenta hasta 3 veces, con hasta 2 segundos de espera, cuando falla algo transitorio: la conexión se cortó o el servidor está reiniciando. No reintenta errores de lógica, como violar un `CHECK`. Tiene una consecuencia importante para las transacciones, que se explica en 14.3.

- **`CommandTimeout(20)`**: ninguna consulta puede tardar más de 20 segundos. Una consulta colgada no debe retener una conexión del pool para siempre.

Dos piezas más hablan con la base al arrancar:

- **`DatabaseWarmupService`** (`API/Services/`): un servicio en segundo plano que, nada más arrancar, hace una consulta mínima. La primera consulta de EF Core es lenta porque tiene que compilar el modelo y abrir la primera conexión; así ese coste lo paga el arranque y no el primer cliente. Si falla, solo escribe un *warning*: la API sigue arrancando.
- **El *health check*** `/health` (`AddDbContextCheck`) responde 200 solo si la base contesta. Es lo que consultaría un orquestador para saber si la API está sana.

12.3 La base local con Docker

Desde el 16 de octubre la base de desarrollo es un PostgreSQL 17 dentro de un contenedor Docker, definido en `docker-compose.yml`, en la raíz del repositorio:

```yaml
services:
  db:
    image: postgres:17-alpine
    container_name: furnistore-db
    restart: unless-stopped
    environment:
      POSTGRES_DB: furnistore
      POSTGRES_USER: furnistore
      POSTGRES_PASSWORD: furnistore
    ports:
      - "127.0.0.1:5433:5432"
    volumes:
      - furnistore-db:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U furnistore -d furnistore"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  furnistore-db:
```

Cada línea, explicada:

Línea	Qué hace
`db`	El nombre del **servicio**. Es el que usan los comandos de `docker compose`
`container_name: furnistore-db`	El nombre del **contenedor**. Es el que usan los comandos de `docker` a secas
`image: postgres:17-alpine`	PostgreSQL 17 sobre Alpine Linux, una imagen pequeña
`restart: unless-stopped`	Docker lo vuelve a levantar al encender el equipo, salvo que lo pares tú
`environment`	Crea la base `furnistore` y su usuario la primera vez que arranca
`ports: 127.0.0.1:5433:5432`	El puerto 5432 del contenedor se ve como 5433 en tu equipo, y solo desde tu equipo (`127.0.0.1`). Se usa el 5433 porque tu sistema ya tiene otro PostgreSQL en el 5432
`volumes`	Los datos viven en un **volumen** con nombre, fuera del contenedor
`healthcheck`	Docker comprueba cada 5 segundos que PostgreSQL acepta conexiones

Contenedor frente a volumen. El contenedor es el programa en ejecución y se puede borrar y recrear sin miedo. El volumen es el disco donde PostgreSQL guarda los datos, y **sobrevive al contenedor**. Por eso `docker compose down` no pierde nada: borra el contenedor, no el volumen. El volumen se llama `furniture-store_furnistore-db`, con el nombre del proyecto delante.

Comandos útiles, desde la raíz del repositorio:

Comando	Qué hace
`docker compose up -d`	Crea y arranca la base en segundo plano
`docker compose ps`	Muestra si está en marcha y sana
`docker compose stop`	La detiene sin borrar nada
`docker compose down`	Borra el contenedor; los datos se conservan en el volumen
`docker compose down -v`	**Borra también el volumen: todos los datos se pierden**
`docker compose logs -f db`	Muestra los registros de PostgreSQL en vivo
`docker exec -it furnistore-db psql -U furnistore -d furnistore`	Abre una consola SQL dentro de la base

Dentro de `psql`, `\dt` lista las tablas, `\d "Orders"` muestra la estructura de una tabla y `\q` sale. Los nombres de tabla van entre comillas dobles porque EF Core los crea con mayúsculas, y PostgreSQL convierte a minúsculas todo lo que no va entre comillas.

Por qué Docker. Cualquier compañero obtiene la misma versión de PostgreSQL con un solo comando, sin instalar nada en su sistema y sin chocar con otros PostgreSQL. Y la API funciona sin internet, que era el objetivo para la presentación.

12.4 De Supabase a la base local

Hasta el 16 de octubre la base estaba en Supabase, un PostgreSQL alojado en la nube. Se pasó a la base local con `pg_dump`, exportando solo el esquema `public` (el de la aplicación, sin los esquemas propios de Supabase), y cargándolo en el contenedor. La copia está en `~/Desktop/furnistore-supabase.dump` y la cadena antigua sigue en `.env` como `SUPABASE_DATABASE_URL`.

Para la API el cambio fue una sola línea: `DATABASE_URL`. Como el código solo depende de "un PostgreSQL" y no de Supabase, cualquier PostgreSQL sirve: el de Docker, uno instalado en el sistema o uno en la nube.

---

13. Migraciones

13.1 Concepto

El esquema de una base cambia con el proyecto: hoy se añade una columna, mañana una tabla. Una **migración** es un archivo de C# que describe un cambio del esquema, con dos métodos:

- `Up`: aplica el cambio.
- `Down`: lo deshace.

EF Core las genera **comparando el modelo actual del código con una foto del modelo anterior**. Esa foto es `APIFurnistoreContextModelSnapshot.cs`, y se actualiza con cada migración. Si añades una propiedad a una entidad, la diferencia con la foto es "una columna nueva", y eso es lo que escribe la migración.

En la base, la tabla `__EFMigrationsHistory` guarda el nombre de cada migración aplicada. Al ejecutar `database update`, EF Core compara esa lista con las migraciones del proyecto y aplica solo las que faltan, en orden.

Cada migración genera tres cosas:

Archivo	Qué es
`20261011155859_AddClientIdentityCard.cs`	`Up` y `Down`: lo que se lee y, a veces, se edita
`20261011155859_AddClientIdentityCard.Designer.cs`	El modelo completo en ese momento. No se toca
`APIFurnistoreContextModelSnapshot.cs`	La foto actualizada del modelo. No se toca

El número del nombre es la fecha y hora en UTC (`2026-10-11 15:58:59`): así se ordenan solas.

13.2 Los comandos en Furnistore

Se ejecutan desde `apps/api`. La herramienta `dotnet-ef` está fijada como herramienta local (2.6), en `.config/dotnet-tools.json`:

```
# Ver las migraciones y cuáles faltan por aplicar
dotnet ef migrations list --project API.Furnistore.Data --startup-project API.Furnistore.API -- --
environment Development

# Crear una migración a partir de los cambios del modelo
dotnet ef migrations add NombreDelCambio --project API.Furnistore.Data --startup-project
API.Furnistore.API -- --environment Development

# Aplicar las pendientes a la base de DATABASE_URL
dotnet ef database update --project API.Furnistore.Data --startup-project API.Furnistore.API -- --
environment Development

# Ver el SQL que se ejecutaría, sin ejecutarlo
dotnet ef migrations script UltimaAplicada NuevaMigracion --project API.Furnistore.Data --startup-
project API.Furnistore.API -- --environment Development

# Deshacer la última migración, solo si todavía no se aplicó
```

```
dotnet ef migrations remove --project API.Furnistore.Data --startup-project API.Furnistore.API -- --
environment Development
```

Cada parte del comando tiene su motivo:

- **`--project API.Furnistore.Data`**: dónde están el `DbContext` y la carpeta `Migrations`.
- **`--startup-project API.Furnistore.API`**: qué proyecto arrancar para obtener la configuración. EF Core ejecuta `Program.cs` hasta `builder.Build()`, toma de ahí el `DbContext` ya configurado, con su cadena de conexión, y detiene el arranque.
- **`-- --environment Development`**: lo que va tras el `--` suelto se pasa a `Program.cs`, que así carga la configuración de desarrollo.

Al ejecutarlos verás en la salida un error largo de tipo `HostAbortedException`. **Es normal**: es la forma en que EF Core detiene el arranque de la API una vez que tiene lo que necesita. Lo que importa es la línea final, `Done.`

Las migraciones no se aplican solas al arrancar la API. Se aplican con `database update`, a propósito: un cambio de esquema es una operación delicada que alguien debe decidir y vigilar. Aplicarlas al arrancar haría que dos copias de la API arrancando a la vez intentaran migrar la misma base.

13.3 La historia de la base: las 22 migraciones

Leerlas en orden es leer la historia del proyecto:

Fecha	Migración	Qué hizo
23 jul	`initial`	Crea `Clients`, `Orders`, `ProductCategories` y `Products`
26 jul	`Order Details added`	Crea `OrderDetails`
31 jul	`Identity to the API`	Crea las 7 tablas de Identity: aparecen las cuentas
3 sep	`UpdateModelChanges`	Ajusta longitudes de dos tablas de Identity
3 sep	`Adding Refresh Tokens table`	Crea `RefreshTokens`
4 sep	`SeedCatalogTestData`	Inserta 4 categorías y los productos de ejemplo
5 sep	`RemoveFloatingShelfProduct`	Borra un producto de ejemplo
12 sep	`RemoveProductSeeds`	Saca los datos de ejemplo del modelo sin borrarlos de la base. `Up` está vacío
12 sep	`AddProductImageUrl`	Añade `Products.ImageUrl` y rellena la imagen de cada producto con SQL
25 sep	`AddClientUserOwnership`	Une cada cliente a su cuenta (`Clients.UserId`)
27 sep	`HardenOrdersAndProducts`	Crea la secuencia de números de pedido; añade stock, estado y totales
27 sep	`AddCartItems`	Crea el carrito
2 oct	`AddCheckoutSupport`	Copia el nombre del producto en cada línea de pedido; claves del carrito

Fecha	Migración	Qué hizo
2 oct	`ReshapeDomainModel`	La gran reforma: estados como texto, copia de la dirección en el pedido, fechas del ciclo de vida, `CHECK` de importes y cantidades, tokens guardados como hash
2 oct	`DropOrderStatusDefault`	Quita un valor por defecto que sobraba en `Status`
6 oct	`AddEmailVerificationCodes`	Crea la tabla de códigos de verificación
10 oct	`AddBackofficeSupport`	Crea `AuditEntries`, añade el estado "En preparación" y los índices del panel
10 oct	`AddRefreshTokenUsedAt`	Añade `RefreshTokens.UsedAt`
11 oct	`AddClientIdentityCard`	Añade el carnet de identidad, único
16 oct	`ConvertPricesToCup`	Pasa todos los importes a CUP
17 oct	`ConvertPricesToUsd`	Devuelve todos los importes a USD

Tres de las primeras tienen espacios en el nombre (`Order Details added`, `Identity to the API` y `Adding Refresh Tokens table`). Funcionan, pero obligan a escribir comillas en la terminal; las demás siguen la convención de C#, `PascalCase`, que es la recomendable.

13.4 Anatomía de una migración

La mayoría son pequeñas y las escribe EF Core solo. `AddClientIdentityCard`, entera:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.AddColumn<string>(
        name: "IdentityCard",
        table: "Clients",
        type: "character varying(11)",
        maxLength: 11,
        nullable: true);

    migrationBuilder.CreateIndex(
        name: "IX_Clients_IdentityCard",
        table: "Clients",
        column: "IdentityCard",
        unique: true);
}
```

Salió de dos cambios en el código: la propiedad `public string? IdentityCard` en `Client.cs` y dos líneas en `ClientConfiguration.cs` (`HasMaxLength(11)` e `IsUnique()`). EF Core vio la diferencia con la foto anterior y escribió esto. Fíjate en `nullable: true`: los clientes que ya existían no tenían carnet, así que la columna tiene que admitir vacío. Que sea obligatorio para comprar lo comprueba el checkout, no la base.

13.5 Migraciones de datos y SQL a mano

EF Core escribe migraciones de **esquema** (tablas, columnas, índices). Cuando además hay que **transformar los datos existentes**, se añade SQL a mano con `migrationBuilder.Sql(...)`. El proyecto tiene varios ejemplos que enseñan algo distinto:

Comprobar antes de transformar. `ReshapeDomainModel` iba a hacer obligatorio que cada cliente tuviera cuenta. Antes de cambiar nada, comprueba que los datos lo permiten y, si no, detiene la migración con un mensaje que explica qué arreglar:

```
DO $$
BEGIN
    IF EXISTS (SELECT 1 FROM "Clients" WHERE "UserId" IS NULL) THEN
        RAISE EXCEPTION 'ReshapeDomainModel: hay clientes sin cuenta (UserId NULL). Asócialos a un
usuario o elimínalos antes de migrar.';
    END IF;
    …
END $$;
```

Como cada migración corre dentro de una transacción, si salta la excepción no queda nada a medias.

Convertir datos al cambiar el tipo. En la misma migración, el estado del pedido pasó de número a texto, y el SQL tradujo los valores antiguos:

```
UPDATE "Orders"
SET "Status" = CASE "Status"
    WHEN '0' THEN 'Paid'
    WHEN '2' THEN 'Shipped'
    WHEN '3' THEN 'Delivered'
    …
```

Lo que el modelo no sabe declarar. El índice que impide categorías repetidas sin distinguir mayúsculas:

```
CREATE UNIQUE INDEX "UX_ProductCategories_Name_Lower" ON "ProductCategories" (lower("Name"));
```

Guardar secretos como hash. Los refresh tokens se guardaban en claro; la migración calculó el hash de los existentes con funciones de PostgreSQL antes de borrar la columna original:

```
UPDATE "RefreshTokens"
SET "TokenHash" = encode(sha256(convert_to("Token", 'UTF8')), 'hex');
```

Cambiar todos los importes de moneda. `ConvertPricesToCup` es una migración solo de datos: el modelo no cambió, así que EF Core la generó vacía y el SQL se escribió a mano. Multiplica por 800 los precios, redondeados a la centena, y también los importes de pedidos, líneas y auditoría, estos exactos para que los totales sigan cuadrando:

```
private const int CupPerUnit = 800;

protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.Sql($"""
        UPDATE "Products"
        SET "Price" = GREATEST(ROUND("Price" * {CupPerUnit} / 100) * 100, 100);

        UPDATE "Orders"
        SET "Subtotal" = "Subtotal" * {CupPerUnit},
            "ShippingCost" = "ShippingCost" * {CupPerUnit},
            "Total" = "Total" * {CupPerUnit};
        …
        """);
}
```

Su `Down` divide por 800, pero no puede recuperar los céntimos que se perdieron al redondear: no todas las migraciones son perfectamente reversibles, y conviene saberlo antes de aplicarlas.

Deshacer una migración que ya se compartió. Al día siguiente se decidió volver a USD. `ConvertPricesToCup` ya estaba en el repositorio y alguien podía haberla aplicado, así que no se borró: se escribió otra, `ConvertPricesToUsd`, cuyo `Up` hace lo mismo que el `Down` anterior (divide por 800).

Así cualquier base, la haya aplicado o no, termina en USD con un `database update`. Los pedidos, las líneas y la auditoría vuelven exactos. Los cuatro precios que se habían redondeado no recuperan solos sus céntimos (79,90 se queda en 79,88); se corrigen con un `UPDATE` puntual a partir de los valores originales.

13.6 Reglas de trabajo

1. **Nunca edites una migración que ya se aplicó en alguna base.** EF Core no vuelve a ejecutarla: la base y el código quedarían descuadrados. Si algo salió mal, se corrige con una migración nueva.
2. **Revisa el SQL antes de aplicar** con `migrations script`, sobre todo si la migración transforma datos. Es lo que se hizo con `ConvertPricesToCup` y `ConvertPricesToUsd`.
3. **Una migración por cambio con sentido, con un nombre que lo explique.** `AddClientIdentityCard` se entiende sin abrirla; `UpdateModelChanges` no.
4. **Las migraciones viajan con el código.** Cada compañero tiene su propia base local: después de traer cambios del repositorio, ejecuta `database update` y su base queda igual que la tuya. Los datos no viajan; el esquema sí.
5. **Nunca toques `__EFMigrationsHistory` ni el *snapshot* a mano.** Si se descuadran, EF Core intentará volver a crear tablas que ya existen, o no aplicará migraciones que faltan.
6. **Los datos de prueba no van en el modelo.** El proyecto los tuvo con `HasData` y los sacó en `RemoveProductSeeds`: con `HasData`, cada cambio en un dato de ejemplo genera una migración, y se mezcla lo que es estructura con lo que es contenido.

---

14. Transacciones y concurrencia

14.1 Concepto

Una **transacción** agrupa varias operaciones para que se cumplan todas o ninguna. Sus garantías se resumen con la sigla ACID:

Letra	Garantía	En una compra
Atomicidad	Todo o nada	No puede quedar el stock descontado sin pedido creado
Consistencia	Se respetan las reglas (`CHECK`, `FK`, `UNIQUE`)	No puede quedar un total que no cuadre
a**I**slamiento	Las transacciones simultáneas no se pisan	Dos compras a la vez no venden la misma última silla
Durabilidad	Lo confirmado no se pierde	Un pedido confirmado sobrevive a un apagón

El aislamiento es la parte difícil. PostgreSQL usa por defecto el nivel **Read Committed**: cada consulta ve los datos confirmados en el instante en que empieza. Eso evita leer datos a medias, pero no evita las **condiciones de carrera**: dos peticiones leen "queda 1 silla", las dos deciden que pueden venderla y las dos la venden.

Hay dos familias de soluciones:

- **Pesimista: bloquear antes de leer.** Quien llega primero bloquea y el segundo espera. Es segura y simple, pero hace esperar.

- **Optimista: no bloquear, pero detectar el choque al guardar.** Cada fila lleva una versión; al guardar se comprueba que nadie la cambió entretanto y, si cambió, se rechaza. No hace esperar, pero alguien tiene que reintentar.

Furnistore usa cada una donde encaja mejor, y este capítulo recorre todos los casos.

14.2 La transacción implícita de `SaveChangesAsync`

Sin hacer nada especial, cada `SaveChangesAsync` ya es una transacción: todos los `INSERT`, `UPDATE` y `DELETE` pendientes se confirman juntos o no se confirma ninguno. Es lo que usan la mayoría de operaciones, como editar el perfil o crear una categoría.

Hace falta una transacción **explícita** cuando hay que agrupar varios `SaveChangesAsync`, operaciones `ExecuteUpdate` o SQL a mano: todo eso queda fuera de la transacción implícita.

14.3 El patrón de transacción explícita con reintentos

Todas las transacciones explícitas del proyecto siguen esta forma:

```
var strategy = db.Database.CreateExecutionStrategy();

return await strategy.ExecuteAsync(async () =>
{
    db.ChangeTracker.Clear();
    await using var transaction = await db.Database.BeginTransactionAsync(cancellationToken);

    // … lecturas, bloqueos y cambios …

    await db.SaveChangesAsync(cancellationToken);
    await transaction.CommitAsync(cancellationToken);
    return resultado;
});
```

Cada pieza tiene un motivo:

- **`CreateExecutionStrategy().ExecuteAsync(...)`.** Con `EnableRetryOnFailure` activo (12.2), EF Core puede repetir una operación si se corta la conexión. Pero repetir solo la última consulta de una transacción a medias sería un desastre. Por eso EF Core **prohíbe** abrir una transacción sin este envoltorio y lanza una excepción si lo intentas: el envoltorio hace que, si algo transitorio falla, se repita **el bloque entero**, desde el principio de la transacción.
- **`ChangeTracker.Clear()`.** Si el bloque se repite, lo que quedó en memoria del intento anterior ya no vale. Limpiar al empezar garantiza que cada intento parte de cero.
- **`await using var transaction`.** Si el bloque sale sin llegar a `CommitAsync`, por un `return` temprano o por una excepción, `await using` desecha la transacción, y desecharla sin confirmar equivale a un *rollback*. Aun así el código llama a `RollbackAsync` explícitamente en los caminos de error, para que se lea claro.
- **`CommitAsync` al final.** Hasta este punto nada es visible para otras peticiones.

14.4 El checkout: bloqueo por cliente y reserva atómica de stock

Es el caso más delicado del proyecto: cobrar, descontar stock, crear el pedido y vaciar el carrito tienen que ocurrir juntos, y nunca vender más de lo que hay. `OrderService.CheckoutAsync` resuelve dos problemas distintos.

Problema 1: el mismo cliente confirma dos veces. Un doble clic, o dos pestañas, lanzan dos checkouts del mismo carrito. Ambos leerían el carrito y crearían dos pedidos. La solución es un **bloqueo consultivo** (*advisory lock*) de PostgreSQL:

```
await db.Database.ExecuteSqlAsync(
    $"SELECT pg_advisory_xact_lock({CheckoutLockNamespace}, {client.ID})",
    cancellationToken
);
```

`pg_advisory_xact_lock(a, b)` bloquea un "nombre" formado por dos números, sin asociarlo a ninguna fila. Aquí el nombre es `(3004, id del cliente)`: el segundo checkout del mismo cliente espera a que termine el primero, que vacía el carrito, y entonces ya no tiene nada que comprar. El bloqueo se libera solo al terminar la transacción (por eso lleva `xact`). Los checkouts de clientes distintos no se esperan entre sí, porque su segundo número es distinto. El `3004` es el "espacio de nombres" del checkout, para no chocar con otros bloqueos consultivos del proyecto.

Problema 2: dos clientes compran la última unidad. No se puede leer el stock, comprobarlo en C# y luego restarlo: entre la lectura y la resta otro cliente podría comprar. La solución es que **la comprobación y la resta sean la misma sentencia**:

```
foreach (var line in lines.OrderBy(l => l.ProductId))
{
    var affected = await db
        .Products.Where(p => p.Id == line.ProductId && p.Stock >= line.Quantity)
        .ExecuteUpdateAsync(
            setters => setters.SetProperty(p => p.Stock, p => p.Stock - line.Quantity),
            cancellationToken
        );

    if (affected == 0)
        return line.ProductId;
}
```

Esto genera, simplificado, `UPDATE "Products" SET "Stock" = "Stock" - @q WHERE "Id" = @id AND "Stock" >= @q`. PostgreSQL bloquea la fila mientras la actualiza, así que dos de estas sentencias sobre el mismo producto se ejecutan una detrás de otra, y la segunda vuelve a evaluar la condición con el stock ya descontado. Si devuelve 0 filas afectadas, no había stock suficiente: se deshace toda la transacción y el cliente recibe "No hay stock suficiente de X: quedan N". Además, el `CHECK Stock >= 0` de la tabla impediría un stock negativo aunque este código tuviera un error.

Por qué se recorre en orden de `ProductId`. Si un checkout reserva primero la silla y luego la mesa, y otro reserva primero la mesa y luego la silla, cada uno podría quedarse esperando la fila que tiene el otro: un **interbloqueo** (*deadlock*). Si todos reservan en el mismo orden, eso no puede ocurrir.

El checkout completo, en orden:

1. Abre la transacción y toma el bloqueo del cliente.
2. Lee el carrito con los productos, ordenado por `ProductId`.
3. Rechaza si algún producto se archivó o si el total cambió desde que el cliente lo vio (`ExpectedTotal`).
4. Reserva el stock línea a línea, con la sentencia atómica.
5. Crea el pedido con sus líneas, copiando precios, nombres y dirección.
6. Registra `order.created` en la auditoría.
7. Vacía el carrito con `ExecuteDelete`.
8. Confirma. Si cualquier paso falló, no ocurrió nada.

14.5 Cambios de estado de un pedido: `SELECT … FOR UPDATE`

Dos administradores podrían pulsar a la vez "Enviar" y "Cancelar" sobre el mismo pedido. `OrderWorkflow.ApplyAsync`, que hacen todos los cambios de estado, bloquea la fila del pedido al leerla:

```
var order = (
    await db
        .Orders.FromSql($"SELECT * FROM \"Orders\" WHERE \"Id\" = {orderId} FOR UPDATE")
        .ToListAsync(cancellationToken)
).SingleOrDefault();
```

`FOR UPDATE` bloquea esa fila hasta el final de la transacción. El segundo administrador espera y, cuando lee, ve el estado que dejó el primero. Entonces la comprobación de transición válida rechaza su acción: no se puede cancelar un pedido que acaba de salir.

Es un bloqueo **pesimista**, y encaja aquí porque los cambios de estado son rápidos y el conflicto tiene consecuencias reales: un pedido enviado y cancelado a la vez. Al cancelar, `RestockAsync` devuelve el stock de cada línea, también con `ExecuteUpdate` y dentro de la misma transacción.

14.6 Concurrencia optimista con `xmin`: productos y clientes

Editar un producto en el panel lleva tiempo: abres el formulario, cambias el precio, piensas, guardas. Si mientras tanto otro administrador cambió el stock y tú guardas encima, borrarías su cambio sin saberlo. Bloquear la fila durante minutos no es viable, así que aquí se usa concurrencia **optimista**.

La versión. PostgreSQL tiene en cada fila una columna oculta, `xmin`, con el identificador de la transacción que la escribió por última vez. Cambia sola cada vez que la fila se modifica. EF Core la usa como versión:

```
builder.Property(p => p.Version).IsRowVersion();   // ProductConfiguration
```

```
public uint Version { get; set; }                  // Product, en Shared
```

El flujo completo:

1. El panel pide el producto y recibe, junto con los datos, `"version": 24871`.
2. El formulario guarda esa versión y la devuelve al pulsar Guardar.
3. `AdminProductService.UpdateAsync` indica a EF Core que la versión original es la que trajo el formulario:

   ```
   db.Entry(product).Property(p => p.Version).OriginalValue = request.Version;
   ```

4. Al guardar, EF Core genera `UPDATE "Products" SET … WHERE "Id" = @id AND xmin = @version`.
5. Si otro guardó entretanto, `xmin` ya no coincide, el `UPDATE` afecta 0 filas y EF Core lanza `DbUpdateConcurrencyException`.
6. El servicio la traduce a un `409 Conflict` con el código `admin.version_conflict`, y la web avisa de que el producto cambió mientras lo editabas y ofrece recargarlo.

`Clients` usa el mismo mecanismo cuando el cliente edita su perfil o un administrador edita sus datos (`client.version_conflict`).

Por qué `xmin` y no una columna propia. Una columna `Version` normal habría que incrementarla a mano en cada cambio, y bastaría un olvido para romper la protección. `xmin` la mantiene PostgreSQL sola y no ocupa espacio.

14.7 Cambios de rol: un bloqueo global

La regla "siempre debe quedar al menos un administrador" no protege una fila, sino una condición sobre toda la tabla. Si dos administradores se quitaran el rol el uno al otro a la vez, cada uno vería que "quedan 2" y ambos lo conseguirían, dejando la tienda sin administradores.

`AdminCustomerService` lo evita con un bloqueo consultivo de nombre fijo, `pg_advisory_xact_lock(3005, 0)`. Todas las acciones sobre cuentas (desactivar, reactivar, dar o quitar el rol) se ejecutan **de una en una**, y cada una cuenta los administradores después de obtener el bloqueo:

```
if (target.AdminCount <= 1)
    return Error.Conflict("customer.last_admin", "No se puede quitar el rol al último administrador.");
```

Serializar estas acciones no cuesta nada, porque son raras y rápidas.

14.8 La base como último árbitro: violaciones de unicidad

Comprobar en C# que un carnet no está en uso y luego guardarlo tiene la misma carrera de siempre: dos clientes podrían pasar la comprobación a la vez. La protección real es el índice único de la base. El código intenta guardar y, si PostgreSQL rechaza el duplicado, traduce el error:

```
catch (DbUpdateException exception) when (AdminErrors.IsUniqueViolation(exception))
{
    return Result.Fail(IdentityCardTaken());
}
```

En el registro hay un paso más. La cuenta (`AspNetUsers`) se crea antes que el cliente, así que si el carnet choca al guardar el cliente quedaría una cuenta sin cliente. Por eso `AuthService` primero pregunta si el carnet ya existe (el caso normal: responde 409 sin crear nada) y, si aun así pierde la carrera, borra la cuenta recién creada antes de devolver el mismo error:

```
catch (DbUpdateException exception) when (AdminErrors.IsUniqueViolation(exception))
{
    db.ChangeTracker.Clear();
    await userManager.DeleteAsync(user);
    return IdentityCardRejected(email);
}
```

`IsUniqueViolation` mira si el error interno es una `PostgresException` con el código SQL `23505` (`unique_violation`). Así se resuelven el carnet repetido y el nombre de categoría repetido. Lo mismo vale para las restricciones `CHECK` y las claves foráneas: la API valida antes para dar buenos mensajes, y la base garantiza que, aunque la API fallara, los datos nunca serían incoherentes.

14.9 Resumen

Mecanismo	Dónde	Qué evita
Transacción implícita de `SaveChangesAsync`	Casi todas las escrituras	Guardar la mitad de los cambios

Mecanismo	Dónde	Qué evita
Transacción explícita con estrategia de ejecución	Checkout, cambios de estado, acciones sobre cuentas, productos y categorías	Lo mismo, cuando hay varios pasos, y reintentos que dejen basura
Bloqueo consultivo por cliente (`3004`, cliente)	Checkout	Dos pedidos del mismo carrito
`UPDATE … WHERE Stock >= cantidad`	Reserva de stock	Vender más unidades de las que hay
Orden por `ProductId`	Reserva de stock y reposición	Interbloqueos entre checkouts
`SELECT … FOR UPDATE`	Cambios de estado de un pedido	Enviar y cancelar el mismo pedido a la vez
Versión `xmin`	Edición de productos y clientes	Pisar sin saberlo lo que otro guardó
Bloqueo consultivo global (`3005`, `0`)	Roles y estado de las cuentas	Quedarse sin administradores
Índices únicos, `CHECK` y claves foráneas	Toda la base	Datos incoherentes aunque la API tenga un error

---

Parte IV. Seguridad

La seguridad de la API responde a tres preguntas, en este orden:

1. **¿Quién eres?** Autenticación: Identity guarda las cuentas y comprueba la contraseña, y el JWT demuestra en cada petición quién la hace (capítulos 15 y 16).
2. **¿Qué puedes hacer?** Autorización: roles, datos propios y reglas como "no puedes quitarte el rol a ti mismo" (capítulo 17).
3. **¿Cuántas veces puedes intentarlo?** Límites de peticiones, bloqueo de cuentas y verificación de correo, para que nadie pruebe contraseñas o códigos por fuerza bruta (capítulos 17 y 18).

15. ASP.NET Core Identity

15.1 Concepto

Guardar usuarios parece fácil, pero hacerlo bien no lo es: las contraseñas no se pueden guardar tal cual, hay que bloquear cuentas tras varios fallos, normalizar correos para compararlos, gestionar roles... **ASP.NET Core Identity** es la biblioteca oficial que resuelve todo eso. Aporta:

- **Las tablas:** `AspNetUsers`, `AspNetRoles`, `AspNetUserRoles` y algunas más (capítulo 10).
- **Los gestores:** `UserManager<IdentityUser>` (crear usuarios, comprobar contraseñas, bloquear) y `RoleManager<IdentityRole>` (crear roles).
- **Las reglas de contraseña y de bloqueo**, configurables.
- **El hash de contraseñas**, hecho correctamente.

15.2 En Furnistore

En `Program.cs`:

```
builder
    .Services.AddDefaultIdentity<IdentityUser>(options =>
    {
        options.SignIn.RequireConfirmedAccount = false;
        options.Password.RequireDigit = true;
        options.Password.RequiredLength = 8;
        options.Password.RequireLowercase = false;
        options.Password.RequireUppercase = false;
        options.Password.RequireNonAlphanumeric = false;
        options.Lockout.AllowedForNewUsers = true;
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(5);
    })
    .AddRoles<IdentityRole>()
    .AddEntityFrameworkStores<APIFurnistoreContext>();
```

Línea	Qué hace
`AddDefaultIdentity<IdentityUser>`	Registra Identity con el usuario estándar (`IdentityUser`: correo, hash, bloqueo...). Furnistore no lo amplía: los datos de la tienda van en `Clients` (10.3).

Línea	Qué hace
`RequireConfirmedAccount = false`	Identity no exige la confirmación por su cuenta. La exige `AuthService.LoginAsync`, con el código de 6 dígitos propio (capítulo 18).
`Password.*`	Mínimo 8 caracteres y al menos un dígito (RN1). Se relajan mayúsculas y símbolos: la longitud protege más que la variedad, y las reglas demasiado estrictas llevan a contraseñas apuntadas en un papel.
`Lockout.*`	Tras 5 contraseñas incorrectas, la cuenta se bloquea 5 minutos (RN3), también para las cuentas nuevas.
`AddRoles<IdentityRole>()`	Activa los roles (`Admin` y `User`).
`AddEntityFrameworkStores<APIFurnistoreContext>()`	Identity guarda sus datos con EF Core, en la misma base y con el mismo `DbContext` que la tienda. Por eso `APIFurnistoreContext` hereda de `IdentityDbContext` (11.2).

`AddDefaultIdentity` también registra un esquema de autenticación por *cookies*, pero no se usa: el esquema por defecto es el JWT (16.2). La API solo usa de Identity `UserManager` y `RoleManager`, nunca `SignInManager`, que está pensado para sitios con *cookies* de sesión.

15.3 Las contraseñas: un hash, nunca el texto

La columna `AspNetUsers.PasswordHash` no contiene la contraseña, sino un **hash**: el resultado de una función que es fácil de calcular en un sentido e imposible de invertir. Identity usa **PBKDF2** con HMAC-SHA512, una **sal** aleatoria distinta por usuario y decenas de miles de iteraciones:

- La **sal** hace que dos personas con la misma contraseña tengan hashes distintos, y que no sirvan las tablas precalculadas de contraseñas comunes.
- Las **iteraciones** hacen que calcular un hash cueste milisegundos. Para un usuario que inicia sesión no se nota, pero quien robara la base y quisiera probar millones de contraseñas tardaría años.

Al iniciar sesión, `UserManager.CheckPasswordAsync(user, password)` calcula el hash de lo que se escribió, con la misma sal, y lo compara con el guardado. En ningún momento la API puede leer la contraseña original, ni siquiera un administrador.

15.4 El bloqueo de cuentas, y cómo se reutiliza para desactivarlas

`AuthService.LoginAsync` gestiona el bloqueo a mano con `UserManager`:

```
if (!disabled && await userManager.IsLockedOutAsync(user))
    return LockedOut(user);

if (!await userManager.CheckPasswordAsync(user, request.Password))
{
    if (disabled)
        return LoginFailure(email, "bad_password");

    await userManager.AccessFailedAsync(user);

    return await userManager.IsLockedOutAsync(user)
        ? LockedOut(user)
```

```
        : LoginFailure(email, "bad_password");
}
```

- `AccessFailedAsync` suma un fallo (`AccessFailedCount`) y, al llegar a 5, rellena `LockoutEnd` con la hora actual más 5 minutos.
- Un inicio de sesión correcto llama a `ResetAccessFailedCountAsync` y pone el contador a cero.
- Mientras `LockoutEnd` esté en el futuro, `IsLockedOutAsync` devuelve `true` y la API responde 429 `auth.locked_out`, **aunque la contraseña sea correcta**. Si no fuera así, quien prueba contraseñas sabría cuándo acierta.

Desactivar una cuenta reutiliza el mismo mecanismo. No hace falta una columna nueva: desactivar es bloquear hasta el año 9999 (`Auth/AccountLock.cs`):

```
public static readonly DateTimeOffset DisabledUntil = new(9999, 1, 1, 0, 0, 0, TimeSpan.Zero);

public static bool IsDisabled(IdentityUser user) => user.LockoutEnd >= DisabledUntil;
```

Para el código, "bloqueada" (por fallos, durante minutos) y "desactivada" (por un administrador, hasta que la reactive) se distinguen solo por la fecha. Una cuenta desactivada responde 403 `auth.account_disabled`, y sus fallos de contraseña no cuentan, para que no se desbloquee sola cuando caduque el bloqueo.

15.5 Roles

Hay dos roles, `User` y `Admin`, que `Program.cs` crea al arrancar si no existen (3.3):

- **`User`:** se asigna a toda cuenta al registrarse.
- **`Admin`:** lo da un administrador desde el panel (`AdminCustomerService.GrantAdminAsync`) o, para el primero, la consola:

```
dotnet run --project API.Furnistore.API -- admin promote ana@example.com
```

`Commands/AdminCommand.cs` intercepta esos argumentos antes de que el servidor empiece a escuchar, llama al mismo `GrantAdminAsync` que usa el panel (con el autor `console`, que queda en la auditoría) y termina. Usar el mismo servicio garantiza que la consola respete las mismas reglas: por ejemplo, que la cuenta tenga el correo confirmado.

15.6 Por qué Identity

- **No reinventar la criptografía.** Un hash de contraseñas casero es el error de seguridad más común en proyectos de estudiantes. Identity lo resuelve con algoritmos revisados por expertos.
- **Viene integrado con EF Core y ASP.NET Core:** sus tablas son migraciones más, y sus roles los entiende `[Authorize(Roles = "Admin")]` sin configuración extra.
- **Se usa solo lo necesario.** Identity incluye páginas de login, autenticación con Google, doble factor… Furnistore usa los gestores y las tablas, y hace el resto (JWT, códigos por correo) a su manera, porque su cliente es una aplicación Next.js y no páginas generadas por el servidor.

---

16. JWT y refresh tokens

16.1 Concepto

Después de iniciar sesión, cada petición tiene que demostrar quién la hace. Hay dos grandes formas:

- **Sesión en el servidor:** el servidor guarda "la sesión 8f3a... es de Ana" y el cliente envía ese identificador. Cada petición obliga a buscar la sesión.
- **Token firmado (JWT):** el servidor entrega un texto que **contiene** los datos ("soy el usuario 42, con rol Admin, válido hasta las 15:00") y una **firma**. El servidor no guarda nada: le basta con comprobar la firma.

Un **JWT** (*JSON Web Token*) son tres partes en Base64 separadas por puntos: cabecera, contenido (*claims*) y firma. La firma es un HMAC-SHA256 del resto, calculado con una clave secreta que solo conoce la API (`JWT_SECRET`). Quien cambie una sola letra del contenido, por ejemplo para ponerse el rol `Admin`, invalida la firma. El contenido **no está cifrado**: cualquiera puede leerlo, y por eso un JWT nunca lleva datos sensibles.

El problema de un JWT es que **no se puede revocar**: si es válido hasta las 15:00, lo es hasta las 15:00, aunque la persona cierre sesión. La solución estándar es combinar dos tokens:

Token	Duración	Dónde se valida	Para qué
JWT (*access token*)	1 hora	En memoria, comprobando la firma	Cada petición
Refresh token	30 días	En la base de datos	Pedir un JWT nuevo cuando el anterior vence

El JWT dura poco, así que si se filtra sirve poco tiempo. El refresh token dura mucho, pero solo sirve para una cosa, se consulta en la base en cada uso y se puede revocar.

16.2 La configuración

En `Program.cs`, primero se leen los tres valores de las variables de entorno (5.2), y luego se define cómo validar un token:

```
var tokenValidationParameters = new TokenValidationParameters()
{
    ValidateIssuerSigningKey = true,
    IssuerSigningKey = new SymmetricSecurityKey(key),
    ValidateIssuer = true,
    ValidIssuer = jwtIssuer,
    ValidateAudience = true,
    ValidAudience = jwtAudience,
    RequireExpirationTime = true,
    ValidateLifetime = true,
    ClockSkew = TimeSpan.Zero,
};
```

Parámetro	Comprueba
`IssuerSigningKey`	Que la firma se hizo con nuestra clave.
`ValidIssuer`	Que el token lo emitió esta API (`JWT_ISSUER`).

Parámetro	Comprueba
`ValidAudience`	Que el token es para esta API (`JWT_AUDIENCE`), y no para otra que compartiera la clave.
`ValidateLifetime` + `RequireExpirationTime`	Que tiene fecha de caducidad y no ha pasado.
`ClockSkew = TimeSpan.Zero`	Sin margen. Por defecto .NET tolera 5 minutos de diferencia entre relojes, así que un token de 1 hora duraría en la práctica 65 minutos.

Esos parámetros se registran como Singleton y se usan en dos sitios: en `AddJwtBearer`, que valida cada petición, y en `AuthService.RefreshAsync`, que valida el JWT vencido al renovar (16.4).

```
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(jwt =>
{
    jwt.SaveToken = true;
    jwt.TokenValidationParameters = tokenValidationParameters;
});
```

Con esto, el middleware `UseAuthentication()` lee la cabecera `Authorization: Bearer <jwt>` de cada petición, valida el token y, si es correcto, rellena `HttpContext.User` con sus *claims*. Si no hay token o no es válido, la petición sigue como anónima; será la autorización (capítulo 17) la que decida si eso basta.

16.3 Emitir los tokens

`AuthService.IssueTokensAsync` se llama al iniciar sesión y al renovar. Primero, el JWT:

```
var claims = new List<Claim>
{
    new Claim("Id", user.Id),
    new Claim(JwtRegisteredClaimNames.Sub, user.Email!),
    new Claim(JwtRegisteredClaimNames.Email, user.Email!),
    new Claim(JwtRegisteredClaimNames.Jti, jti),
};

foreach (var role in roles)
{
    claims.Add(new Claim(ClaimTypes.Role, role));
}
```

Claim	Para qué
`Id`	El id del usuario en `AspNetUsers`. Los controladores lo leen con `User.UserId()` (17.3).
`sub` y `email`	El correo. La web lo muestra en el menú de la cuenta.
`jti`	Un identificador único de este JWT, que se guarda junto al refresh token para emparejarlos.
`role` (uno por rol)	Lo que lee `[Authorize(Roles = "Admin")]`.

Después, el refresh token: 48 caracteres aleatorios de un alfabeto de 64 (`RandomGenerator`, que usa `RandomNumberGenerator`, el generador criptográfico de .NET, y no `Random`, que es predecible). En la base se guarda **su hash SHA-256**, no el token:

```
var rawRefreshToken = RandomGenerator.GenerateRandomString(48);
var refreshToken = new RefreshToken
{
    JwtId = jti,
    TokenHash = RefreshTokenHasher.Hash(rawRefreshToken),
    AddedDate = now,
    ExpiryDate = now.Add(jwt.RefreshTokenLifetime),
    IsRevoked = false,
    IsUsed = false,
    UserId = user.Id,
};
```

Por la misma razón que las contraseñas: quien robara una copia de la base no podría usar ninguna sesión. Aquí basta con SHA-256, sin sal ni iteraciones, porque el token es aleatorio y largo: no hay "tokens comunes" que probar.

16.4 Renovar: `RefreshAsync` paso a paso

Cuando el JWT vence, la web envía el JWT vencido y el refresh token a `POST /api/authentication/refresh-token`. La API comprueba, en este orden:

1. **La firma del JWT, pero no su caducidad.** Por definición está vencido; lo que importa es que lo emitió esta API.

   ```
   var parameters = tokenValidationParameters.Clone();
   parameters.ValidateLifetime = false;
   ```

2. **El algoritmo es HS256.** Evita trucos con tokens que declaran otro algoritmo, por ejemplo `none`.
3. **El refresh token existe**, buscándolo por su hash.
4. **No está revocado** (por un *logout*).
5. **No se ha usado ya**, salvo que se usara hace menos de 30 segundos (16.5).
6. **No ha caducado** (30 días).
7. **Pertenece a ese JWT:** el `jti` del JWT coincide con el guardado. Un refresh token robado no sirve con un JWT de otra sesión.
8. **El usuario existe y no está desactivado.**

Si todo es correcto, marca el refresh token como usado y emite un par nuevo. Cada fallo devuelve el mismo 401 `auth.invalid_refresh_token`; el motivo concreto queda solo en el log (`Refresh rejected for {UserId}: {Reason}`), para no dar pistas a quien esté probando.

16.5 Rotación y el margen de 30 segundos

Cada refresh token sirve **una sola vez** (RN5). A esto se le llama **rotación**: si alguien roba un refresh token y lo usa, el legítimo deja de funcionar, o al revés, y la sesión robada dura como mucho hasta la siguiente renovación.

Pero la rotación estricta tiene un problema práctico: la web puede lanzar dos peticiones casi a la vez justo cuando el JWT vence, y las dos intentarían renovar con el mismo refresh token. La segunda fallaría y la persona perdería la sesión sin haber hecho nada raro. Por eso existe `RefreshTokenReuseGrace`, de 30 segundos:

```
if (storedToken.IsUsed && !(storedToken.UsedAt > now - jwt.RefreshTokenReuseGrace))
    return RefreshFailure("refresh_token_spent", storedToken.UserId);
```

Un token ya usado sigue valiendo durante 30 segundos tras su primer uso, y después nunca más. Un token **revocado** no tiene margen: la prueba `A_revoked_refresh_token_is_rejected_even_inside_the_grace_window` lo comprueba.

16.6 Cerrar sesión

`LogoutAsync` marca el refresh token como revocado. No hace falta más: sin refresh token no se puede renovar, y el JWT caduca solo. Es idempotente: cerrar sesión con un token desconocido o ya revocado también responde 204, porque el resultado ("esa sesión ya no sirve") es el mismo.

16.7 Lo que un JWT no puede hacer, y cómo se acota

Un JWT ya emitido vale hasta que caduca. Eso tiene tres consecuencias que conviene conocer:

- Tras cerrar sesión, el JWT que la web tenía sigue siendo válido hasta su hora de caducidad. La web lo borra de sus *cookies*, así que en la práctica no se usa.
- Al **desactivar** una cuenta se revocan todos sus refresh tokens, pero su JWT actual sigue funcionando hasta que caduque, como mucho una hora.
- Al **quitar el rol de administrador**, el JWT actual conserva el rol hasta que caduque. Por eso el panel avisa: "Pierde el acceso al panel cuando vuelva a iniciar sesión".

La duración de 1 hora (`JwtConfig:ExpiryTime` en `appsettings.json`) es el compromiso: más corta obligaría a renovar más a menudo, y más larga alargaría esas ventanas. Una API que necesitara cortar el acceso al instante tendría que consultar la base en cada petición (por ejemplo, comprobando el `SecurityStamp` del usuario, que `AdminCustomerService` ya cambia en cada acción sobre la cuenta), y perdería la ventaja del JWT.

16.8 Por qué así

- **JWT y no sesiones en el servidor.** La API no guarda estado por petición, así que cualquier instancia puede atender cualquier petición sin compartir memoria. Es el estándar para una API consumida por otra aplicación.
- **El navegador nunca ve los tokens.** La web es un servidor intermedio (BFF): guarda los dos tokens en *cookies* `HttpOnly`, que el JavaScript del navegador no puede leer, y es ella quien añade la cabecera `Authorization` al llamar a la API. Un script malicioso inyectado en la página no podría robarlos. Los detalles están en el capítulo 14 de la guía de la web.
- **HMAC con una clave compartida y no claves públicas.** Una sola API emite y valida sus propios tokens, así que no hace falta que un tercero pueda verificarlos. Con varios servicios que validaran tokens de un emisor común convendría RSA o ECDSA, que firman con una clave privada y verifican con una pública.

---

17. Autorización, límites de peticiones y CORS

17.1 Autenticación frente a autorización

- **Autenticación:** quién eres. La resuelve el JWT (capítulo 16) y termina en `HttpContext.User`.
- **Autorización:** qué puedes hacer. Se decide en tres niveles, de lo más general a lo más concreto:

Nivel	Dónde	Ejemplo
1. ¿Hace falta sesión?	Atributos en el controlador	El carrito exige sesión; el catálogo no
2. ¿Hace falta un rol?	Atributos en el controlador	Todo `/api/admin/*` exige `Admin`
3. ¿Es suyo? ¿Lo permiten las reglas?	El servicio	Solo ves tus pedidos; no puedes quitarte el rol a ti mismo

17.2 Los atributos

```
[Authorize]
[Route("api/orders")]
public sealed class OrdersController(OrderService orders) : ControllerBase
```

```
[ApiController]
[Authorize(Roles = AdminRoles.Admin)]
public abstract class AdminControllerBase : ControllerBase;
```

Atributo	Efecto	Respuesta si falla
`[AllowAnonymous]`	Cualquiera, con o sin sesión	—
`[Authorize]`	Hace falta un JWT válido	401 Unauthorized
`[Authorize(Roles = "Admin")]`	Además, el *claim* de rol `Admin`	403 Forbidden

La diferencia entre 401 y 403 importa: 401 significa "no sé quién eres" (la web manda al login); 403, "sé quién eres y no puedes" (la web muestra el error).

`AdminControllerBase` concentra la regla. Los siete controladores del panel heredan de él, así que ninguno puede olvidar el `[Authorize(Roles = "Admin")]`. Y por si alguien creara un controlador de administración sin heredar de él, dos pruebas lo vigilan (`AdminAuthorizationTests`, 23.4):

- `Every_admin_endpoint_rejects_anonymous_callers_and_customers` recorre **todos** los endpoints bajo `/api/admin` y comprueba que responden 401 sin token, 403 con un token de cliente y ni 401 ni 403 con uno de administrador.
- `Admin_only_endpoints_live_under_the_admin_prefix` comprueba lo contrario: ningún endpoint que exija `Admin` vive fuera de `/api/admin/`.

Quién puede llamar a qué:

Controlador	Ruta	Acceso
`ProductsController`, `ProductCategoriesController`	`/api/products`, `/api/product-categories`	Cualquiera (solo lectura)
`AuthenticationController`	`/api/authentication/*`	Cualquiera, con límite de peticiones (17.5)
`CartController`, `ClientsController`, `OrdersController`	`/api/cart`, `/api/clients/me`, `/api/orders`	Con sesión, sobre sus propios datos
Los siete del panel	`/api/admin/*`	Rol `Admin`

17.3 Los datos propios

Un cliente solo puede ver su carrito, su perfil y sus pedidos. El truco está en **de dónde sale el id**: nunca de la petición, siempre del token.

```
public static string UserId(this ClaimsPrincipal principal) =>
    principal.FindFirst("Id")?.Value ?? "anonymous";
```

```
[HttpGet("{id:int}")]
public async Task<IActionResult> GetById(int id, CancellationToken cancellationToken) =>
    (await orders.GetByIdAsync(id, User.UserId(), cancellationToken)).ToActionResult(this);
```

El servicio busca el cliente de ese usuario y filtra por él:

```
.FirstOrDefaultAsync(o => o.Id == id && o.ClientId == clientId, cancellationToken);
```

Si el pedido 42 es de otra persona, la consulta no lo encuentra y la respuesta es **404**, no 403. Responder 403 confirmaría que el pedido 42 existe; con 404, quien prueba números no aprende nada. La prueba `Someone_else_cannot_see_or_cancel_the_order` lo comprueba.

17.4 Reglas de autorización que son de negocio

Algunas decisiones no caben en un atributo porque dependen de los datos. Están en los servicios y devuelven 409 con su motivo:

- No puedes desactivarte, quitarte el rol ni borrarte a ti mismo (`customer.self_action`).
- No se puede quitar el rol al último administrador (`customer.last_admin`). Como dos administradores podrían quitarse el rol el uno al otro a la vez, estas acciones se serializan con un bloqueo consultivo (14.7). La prueba `Simultaneous_revocations_always_leave_one_admin` lanza seis revocaciones simultáneas y comprueba que siempre queda uno.
- Un administrador no se puede desactivar ni borrar sin quitarle antes el rol (`customer.is_admin`).
- Una cuenta sin correo confirmado no puede ser administradora (`customer.unconfirmed`).

17.5 Límites de peticiones (*rate limiting*)

Concepto. Sin límites, alguien podría probar mil contraseñas por minuto contra una cuenta, o usar el registro para mandar miles de correos. Un **limitador** cuenta las peticiones de cada cliente en una ventana de tiempo y rechaza las que sobran con **429 Too Many Requests**.

En Furnistore (`Extensions/RateLimitingExtensions.cs`) hay dos políticas, las dos de **ventana fija** por dirección IP:

```
options.AddPolicy(AuthRateLimits.Credentials, context => PerClient(context, permitsPerMinute: 10));
options.AddPolicy(AuthRateLimits.Session, context => PerClient(context, permitsPerMinute: 30));
```

Política	Límite	Endpoints
`auth-credentials`	10 por minuto	`register`, `login`, `resend-confirmation`, `verify-email`
`auth-session`	30 por minuto	`refresh-token`, `logout`

Se aplican con atributos: `[EnableRateLimiting(AuthRateLimits.Session)]` en todo `AuthenticationController`, y `[EnableRateLimiting(AuthRateLimits.Credentials)]` en los cuatro métodos que reciben credenciales o códigos, que sustituye a la del controlador. La sesión tiene un límite más alto porque la web renueva el token de forma automática y legítima.

Cuando se supera el límite, `RejectAsync` responde con un `ProblemDetails` (`code: "rate_limited"`), añade la cabecera `Retry-After` con los segundos que faltan y lo anota en el log.

La IP correcta: *forwarded headers*. Todas las peticiones a la API llegan desde el servidor de Next.js, no desde el navegador de cada persona. Si el limitador usara la IP de la conexión, todos los usuarios compartirían un solo contador. Por eso la web reenvía la IP real del visitante en la cabecera `X-Forwarded-For`, y `app.UseForwardedHeaders()` la usa como `RemoteIpAddress`:

```
builder.Services.Configure<ForwardedHeadersOptions>(options =>
{
    options.ForwardedHeaders = ForwardedHeaders.XForwardedFor;
    var knownProxies = builder.Configuration["ForwardedHeaders:KnownProxies"] ?? string.Empty;
    foreach (var proxy in knownProxies.Split(',', StringSplitOptions.RemoveEmptyEntries |
StringSplitOptions.TrimEntries))
        options.KnownProxies.Add(IPAddress.Parse(proxy));
});
```

Solo se hace caso a esa cabecera si llega de un proxy de confianza. Por defecto ASP.NET Core confía en la propia máquina (*loopback*), que es donde corre la web en desarrollo. Si la web corriera en otro servidor, su IP iría en `ForwardedHeaders__KnownProxies`. Sin esa restricción, cualquiera podría inventarse un `X-Forwarded-For` distinto en cada petición y saltarse el límite.

Los límites se complementan. El limitador frena a un atacante que prueba muchas cuentas desde una IP; el bloqueo de cuenta (15.4), a muchos atacantes que prueban una misma cuenta; y los 5 intentos por código (18.3), a quien intenta adivinar un código de verificación.

17.6 CORS

Concepto. Por seguridad, el navegador no deja que el JavaScript de una web (`https://tienda.com`) lea respuestas de otro origen (`https://api.tienda.com`), salvo que ese otro origen lo permita con cabeceras CORS (*Cross-Origin Resource Sharing*). Es una protección **del navegador**: no afecta a llamadas entre servidores.

En Furnistore:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("WebApp", policy =>
        policy.WithOrigins(corsOrigins)
            .AllowAnyHeader()
            .AllowAnyMethod()
            .AllowCredentials());
});
```

Los orígenes salen de `CORS_ORIGINS` (por defecto, `http://localhost:3000` y `3001`) y la política se aplica con `app.UseCors("WebApp")`.

Por qué apenas interviene. El navegador nunca llama directamente a la API: llama a Next.js, y Next.js llama a la API desde el servidor, donde CORS no existe. Lo único que el navegador pide a la API son las imágenes de `/assets`, y una etiqueta `<img>` no está sujeta a CORS. La política queda como red de seguridad por si algún día la web llamara a la API desde el navegador: en ese caso solo lo podría hacer desde los orígenes de la lista.

17.7 HTTPS

`app.UseHttpsRedirection()` solo se activa fuera de Development: en producción, una petición por `http://` se redirige a `https://`, para que los tokens nunca viajen sin cifrar. En desarrollo todo ocurre en la propia máquina y la API escucha en `http://localhost:5135`.

---

18. Verificación de correo

18.1 El flujo

1. **Registro** (`POST /register`): crea la cuenta sin confirmar, el cliente y un código de 6 dígitos, y lo envía por correo. Responde `{ emailSent }`.
2. **Verificación** (`POST /verify-email` con correo y código): si el código es correcto, marca `EmailConfirmed = true`.
3. **Reenvío** (`POST /resend-confirmation`): genera un código nuevo, como mucho una vez por minuto.
4. **Login**: rechaza las cuentas sin confirmar con 401 `auth.email_not_confirmed`, pero solo si la contraseña era correcta (18.4).

El código sustituyó en octubre a un enlace de confirmación. Un enlace obliga a abrir el correo en el mismo dispositivo y a que la URL de la web esté bien configurada en la API; un código se puede leer en el móvil y escribir en el ordenador.

18.2 El código

`Auth/VerificationCodes.cs`:

```
public static string Generate() =>
    RandomNumberGenerator.GetInt32(0, (int)Math.Pow(10, Length)).ToString($"D{Length}");

public static string Hash(string secret, string userId, string code) =>
    Convert.ToHexStringLower(
        HMACSHA256.HashData(Encoding.UTF8.GetBytes(secret), Encoding.UTF8.GetBytes($"{userId}:{code}"))
    );

public static bool Matches(string expectedHash, string actualHash) =>
    CryptographicOperations.FixedTimeEquals(
        Encoding.ASCII.GetBytes(expectedHash),
        Encoding.ASCII.GetBytes(actualHash)
    );
```

- **Generación:** con el generador criptográfico, de 000000 a 999999. `D6` rellena con ceros a la izquierda.
- **Se guarda el HMAC, no el código.** Un código de 6 dígitos solo tiene un millón de valores: un SHA-256 simple se podría invertir probándolos todos en un segundo. El HMAC mezcla el código con una clave secreta (la del JWT) y con el id del usuario, así que, sin la clave, la tabla `EmailVerificationCodes` no sirve de nada aunque se filtre.
- **Comparación en tiempo constante.** Una comparación normal (`==`) se detiene en el primer carácter distinto, así que tarda un poco más cuanto más se parece. Midiendo esos tiempos con mucha precisión, alguien podría deducir el valor. `FixedTimeEquals` tarda siempre lo mismo.

18.3 Los límites

Límite	Valor	Cómo se aplica
Vigencia	15 minutos	Columna `ExpiresAt`
Intentos fallidos	5 por código	Columna `FailedAttempts`, incrementada de forma atómica
Reenvío	1 por minuto	Se mira si hay un código creado hace menos de 60 segundos
Códigos vivos	1 por usuario	Índice único en `UserId`; cada código nuevo borra el anterior

Los intentos se reservan antes de comprobar el código, con una sola sentencia:

```
var attemptReserved = await db
    .EmailVerificationCodes.Where(c => c.Id == stored.Id && c.FailedAttempts <
VerificationCodes.MaxAttempts)
    .ExecuteUpdateAsync(
        setters => setters.SetProperty(c => c.FailedAttempts, c => c.FailedAttempts + 1),
        cancellationToken
    );
```

Si devuelve 0, el código ya agotó sus intentos y se borra. Hacerlo así, y no "leer el contador, comparar y sumar", evita que alguien lance 50 intentos en paralelo y todos lean "4 fallos" a la vez. La prueba `Parallel_guesses_never_get_more_than_five_attempts` lo comprueba. Tras el quinto fallo el código se quema, aunque el siguiente intento fuera el correcto (`Five_wrong_codes_burn_the_code_even_for_the_right_one`).

18.4 No revelar qué cuentas existen

Si la API respondiera distinto según exista o no una cuenta, cualquiera podría averiguar si `ana@example.com` está registrada. Por eso:

- **Reenviar** siempre responde 202 Accepted: exista la cuenta o no, esté confirmada o no, o haya que esperar el minuto. El motivo real queda en el log.
- **Verificar** con un correo inexistente responde lo mismo que con un código vencido: 400 `auth.code_expired`.
- **El login** solo dice "confirma tu correo" a quien acierta la contraseña. Con una contraseña incorrecta, la respuesta es "correo o contraseña incorrectos", igual que si la cuenta no existiera.

Cada regla tiene su prueba en `AuthFlowTests` (`Verifying_never_reveals_whether_an_account_exists`, `Login_only_reveals_an_unconfirmed_email_to_someone_who_knows_the_password`...).

18.5 El envío

`Application` declara qué necesita, sin saber cómo se envía:

```
public sealed record EmailMessage(string To, string Subject, string HtmlBody, string TextBody);

public interface IVerificationEmailSender
{
    Task SendAsync(EmailMessage message, CancellationToken cancellationToken);
}
```

- **El mensaje** lo arma `AuthEmails.VerificationCode`, con dos versiones: HTML y texto plano, para los clientes de correo que no muestran HTML. El nombre del cliente se codifica con `HtmlEncoder` antes de insertarlo en el HTML: alguien que se registrara como `<script>…` no conseguiría inyectar nada en el correo.

- **El envío** lo hace `SmtpEmailSender` (en `API`) con **MailKit**, por SMTP a Gmail (`smtp.gmail.com:587` con STARTTLS) y con un tiempo máximo de 10 segundos. Las credenciales salen de `SmtpSettings` (patrón Options, 5.3) y de las variables de entorno.
- **Si el envío falla**, el código recién creado se borra y el registro responde `emailSent: false`. La web lo muestra y ofrece reenviar. Así un fallo de Gmail no deja un código guardado que nadie recibió y que bloquearía el reenvío durante un minuto (`A_failed_send_does_not_leave_a_code_that_blocks_the_next_attempt`).

18.6 El modo consola para desarrollo

Sin conexión no hay Gmail. Para poder registrarse en ese caso existe `LogVerificationEmailSender`, que en lugar de enviar el correo escribe el código en la consola de la API. Se activa con `Email:LogOnly = true` en la configuración, y `Program.cs` reemplaza el servicio registrado:

```
if (builder.Configuration.GetValue<bool>("Email:LogOnly"))
{
    builder.Services.Replace(
        ServiceDescriptor.Singleton<IVerificationEmailSender, LogVerificationEmailSender>()
    );
}
```

`Replace` sustituye el registro de `SmtpEmailSender` hecho antes en `AddApplicationServices`. `AuthService` no se entera: sigue pidiendo un `IVerificationEmailSender`. Es la inversión de dependencias de 6.4 en acción.

18.7 Reintentar el registro

Si la respuesta del registro se pierde (se cae la conexión justo después de crear la cuenta), la persona pulsará "Crear cuenta" otra vez y recibiría "ya existe una cuenta con ese correo" por una cuenta que nunca pudo confirmar. Por eso `RegisterAsync` tiene un caso especial: si la cuenta existe, **no está confirmada** y la contraseña coincide, no es un intento de suplantar a nadie, sino la misma persona reintentando, y se le reenvía el código (respetando el minuto de espera). Si la contraseña no coincide, se responde 409 `auth.email_exists`, como siempre.

En ese mismo método se comprueba el carnet antes de crear la cuenta, y se deshace la cuenta si dos registros chocan con el mismo carnet (14.8).

Parte V. Referencia del dominio

Esta parte es de consulta: qué endpoints hay, qué hace cada servicio, qué errores puede devolver la API y con qué sistemas externos habla. Para entender el *porqué* de cada pieza, las partes anteriores son el mejor punto de partida.

19. Controladores y endpoints

19.1 Las convenciones de un controlador

Todos los controladores siguen el mismo molde:

```
[ApiController]
[Authorize]
[Route("api/orders")]
public sealed class OrdersController(OrderService orders) : ControllerBase
{
    [HttpGet("{id:int}")]
    [ProducesResponseType<OrderResponse>(StatusCodes.Status200OK)]
    [ProducesResponseType<ProblemDetails>(StatusCodes.Status404NotFound)]
    public async Task<IActionResult> GetById(int id, CancellationToken cancellationToken) =>
        (await orders.GetByIdAsync(id, User.UserId(), cancellationToken)).ToActionResult(this);
}
```

Pieza	Para qué
`[ApiController]`	Activa comportamientos de API. El más importante: si el cuerpo no pasa la validación de los atributos (`[Required]`, `[StringLength]`...), responde 400 automáticamente, sin entrar en el método.
`[Route]` y `[HttpGet("{id:int}")]`	La URL. `{id:int}` es una **restricción de ruta**: `/api/orders/abc` ni siquiera llega al método; responde 404.
Model binding	Rellena los parámetros: `id` sale de la ruta, un objeto complejo sale del cuerpo JSON y, con `[FromQuery]`, de la *query string* (`?page=2&search=silla`).
`CancellationToken`	Si el cliente corta la conexión, ASP.NET Core cancela el token y las consultas en curso se detienen (8.6).
`[ProducesResponseType]`	Documenta qué devuelve cada código. No cambia el comportamiento, pero alimenta el contrato OpenAPI del que salen los tipos de la web (capítulo 24).
`ToActionResult` / `ToNoContentResult`	Convierten el `Result` del servicio en la respuesta HTTP (8.1 y 21.2).

Códigos de éxito que se usan:

Código	Cuándo	Ejemplo
200 OK	Devuelve datos	Casi todos
201 Created	Crea un recurso; incluye la cabecera `Location` con su URL (`CreatedAtAction`)	Checkout, crear producto
202 Accepted	"Recibido", sin prometer el resultado	Reenviar código de verificación (18.4)

Código	Cuándo	Ejemplo
204 No Content	Hecho, no hay nada que devolver	Guardar perfil, vaciar carrito, borrar, cerrar sesión

19.2 Todos los endpoints

Públicos

Método y ruta	Qué hace
`GET /api/products`	Catálogo paginado de productos activos, con búsqueda, categoría y rango de precio
`GET /api/products/{id}`	Detalle de un producto, también si está archivado: así siguen funcionando los enlaces desde pedidos antiguos
`GET /api/product-categories`	Categorías
`GET /api/product-categories/{id}`	Una categoría
`GET /health`	Estado de la API y de la base de datos (22.4)
`GET /assets/{archivo}`	Imágenes de productos (22.2)

Autenticación (`/api/authentication`, públicos con límite de peticiones)

Método y ruta	Qué hace	Éxito
`POST /register`	Crea cuenta y cliente, envía el código	200 `{ emailSent }`
`POST /verify-email`	Confirma el correo con el código	204
`POST /resend-confirmation`	Envía un código nuevo	202
`POST /login`	Inicia sesión	200 `{ token, refreshToken }`
`POST /refresh-token`	Renueva los tokens	200 `{ token, refreshToken }`
`POST /logout`	Revoca el refresh token	204

Del cliente (con sesión, siempre sobre sus propios datos)

Método y ruta	Qué hace	Éxito
`GET /api/clients/me`	Perfil	200
`PUT /api/clients/me`	Guardar perfil	204
`GET /api/cart`	Carrito con totales calculados	200
`POST /api/cart/items`	Agregar producto	200 (carrito)
`PUT /api/cart/items/{productId}`	Cambiar cantidad	200 (carrito)
`DELETE /api/cart/items/{productId}`	Quitar producto	200 (carrito)
`DELETE /api/cart`	Vaciar	204
`GET /api/orders`	Mis pedidos, paginados	200
`GET /api/orders/{id}`	Detalle de un pedido mío	200

Método y ruta	Qué hace	Éxito
`POST /api/orders/checkout`	Confirmar la compra	201 (pedido)
`POST /api/orders/{id}/cancel`	Cancelar un pedido mío en *Pagado*	200 (pedido)

Las operaciones del carrito devuelven el carrito completo y no solo la línea cambiada: la web lo pinta de nuevo sin otra petición, con los totales que calculó el servidor.

Panel (`/api/admin`, rol `Admin`)

Área	Endpoints
Pedidos	`GET /orders` (búsqueda y filtros), `GET /orders/stats` (cuántos hay en cada estado), `GET /orders/{id}`, `POST /orders/{id}/prepare`, `/ship`, `/deliver`, `/cancel`
Productos	`GET /products`, `GET /products/{id}`, `POST /products` (201), `PUT /products/{id}`, `DELETE /products/{id}` (204), `POST /products/images`
Categorías	`GET /product-categories`, `POST`, `PUT /{id}`, `DELETE /{id}` (204)
Clientes	`GET /customers`, `GET /customers/{id}`, `PUT /customers/{id}`, `DELETE /customers/{id}` (204), `POST /customers/{id}/unlock`, `/disable`, `/enable`, `POST` y `DELETE /customers/{id}/admin-role`
Reportes	`GET /reports/sales`, `/products`, `/operations`, `/cancellations`, `/inventory`, `/customers`, `POST /reports/exports` (204)
Auditoría	`GET /audit`

Las acciones sobre pedidos y clientes devuelven el recurso actualizado completo, con su historial, para que la pantalla se redibuje con una sola petición.

19.3 Detalles que merecen una mención

- **Subir una imagen** (`POST /api/admin/products/images`) recibe el archivo como `multipart/form-data` (`IFormFile file`). `[RequestSizeLimit]` corta la petición si pasa de 5 MB más un pequeño margen para las cabeceras, antes de leerla entera. El servicio comprueba después el tipo (JPG, PNG o WebP) y el tamaño exacto. Responde con la URL, que la web guarda luego en el producto.
- **Cancelar un pedido** acepta el cuerpo vacío: `[FromBody(EmptyBodyBehavior = EmptyBodyBehavior.Allow)] CancelOrderRequest? request`. El motivo es opcional, y así la web no tiene que enviar `{}`.
- **El panel y la tienda tienen respuestas distintas para lo mismo.** `ProductResponse` (público) no incluye ni la versión ni si tiene pedidos; `AdminProductResponse`, sí. Cada contrato expone solo lo que su pantalla necesita (8.3).

---

20. Servicios de aplicación

Todos los servicios están en `API.Furnistore.Application`, se registran como Scoped (4.3) y devuelven `Result` (8.1).

Servicio	Carpeta	Responsabilidad	Detalles destacados
`AuthService`	`Auth/`	Registro, verificación, login, renovación, logout	Capítulos 15, 16 y 18

Servicio	Carpeta	Responsabilidad	Detalles destacados
`ProductService`	`Products/`	Catálogo público	Solo productos activos; búsqueda con `ILIKE` escapado (8.7)
`ProductCategoryService`	`ProductCategories/`	Categorías públicas	—
`CartService`	`Carts/`	Carrito	Comprueba stock y productos archivados al agregar; calcula totales en el servidor
`ClientService`	`Clients/`	Perfil propio	Versión `xmin`; carnet único (14.8)
`OrderService`	`Orders/`	Checkout, mis pedidos, cancelación del cliente	Capítulo 7 y 14.4
`OrderWorkflow`	`Orders/`	Cambios de estado de un pedido	Lo usan `OrderService` y `AdminOrderService` (20.1)
`AdminOrderService`	`Admin/Orders/`	Pedidos en el panel	Búsqueda por número o cliente; estadísticas por estado
`AdminProductService`	`Admin/Products/`	Productos en el panel	Auditoría campo a campo; imágenes; borrar solo si no se vendió
`AdminCategoryService`	`Admin/Categories/`	Categorías en el panel	Nombre único sin distinguir mayúsculas; no se borra con productos
`AdminCustomerService`	`Admin/Customers/`	Clientes en el panel	Acciones de cuenta, roles y borrado (17.4)
`AdminReportService`	`Admin/Reports/`	Los seis reportes y el registro de exportaciones	Guía de la web, 18.6
`AuditLog`	`Admin/Audit/`	**Escribir** en la auditoría	Solo añade la entrada; la guarda quien llame a `SaveChangesAsync` (20.2)
`AuditService`	`Admin/Audit/`	**Leer** la auditoría	Listado con filtros e historial de una entidad

20.1 `OrderWorkflow`: una máquina de estados

Cada cambio de estado de un pedido es un **dato**, no un método distinto:

```
public static readonly OrderTransition Prepare = new(
    "preparar",
    [OrderStatus.Paid],
    OrderStatus.Processing,
    AuditActions.OrderProcessing,
    ApiEvents.OrderProcessing
);
```

Transición	Desde	Hacia
`Prepare`	Pagado	En preparación
`Ship`	En preparación	Enviado

Transición	Desde	Hacia
`Deliver`	Enviado	Entregado
`CustomerCancel`	Pagado	Cancelado
`AdminCancel`	Pagado, En preparación	Cancelado

Y un único método, `ApplyAsync`, los aplica todos con los mismos pasos:

1. Abre una transacción y bloquea la fila del pedido (`SELECT … FOR UPDATE`, 14.5).
2. Si se pasa un dueño (`ownerClientId`, cuando cancela el cliente), comprueba que el pedido sea suyo; si no, 404.
3. Comprueba que el estado actual esté en `From`. Si no, responde 409 `order.invalid_transition` con un mensaje construido a partir del verbo: "No se puede enviar un pedido Entregado".
4. Cambia el estado y guarda la fecha correspondiente (`ProcessingAt`, `ShippedAt`…).
5. Si es una cancelación, guarda el motivo y devuelve el stock.
6. Registra el cambio en la auditoría y en el log, y confirma.

Por qué así. Añadir un estado nuevo (por ejemplo, *Devuelto*) sería añadir una transición a la lista, no copiar un método con sus bloqueos y su auditoría. Y la regla de qué cambio se puede hacer desde qué estado queda escrita en un solo sitio, legible como una tabla. Que cliente y administrador cancelen con transiciones distintas (`CustomerCancel` y `AdminCancel`) es justo la regla de negocio RN15.

20.2 `AuditLog`: anotar sin guardar

```
public void Record(
    string actorUserId,
    string action,
    string entityType,
    object entityId,
    string summary,
    AuditChanges? changes = null
) =>
    db.AuditEntries.Add(new AuditEntry { … });
```

`Record` solo **añade** la entrada al `DbContext`; no llama a `SaveChangesAsync`. Como el servicio que la usa comparte el mismo `DbContext` (es Scoped, 4.3), la entrada se guarda en el mismo `SaveChangesAsync` que el cambio que describe, dentro de la misma transacción. Así es imposible que quede un cambio sin su registro o un registro de un cambio que falló.

`AuditChanges` describe qué cambió, campo a campo:

```
var changes = new AuditChanges().Track("status", previous, transition.To);
```

`Track` solo anota el campo si el valor cambió; `Set` lo anota siempre (por ejemplo, el total de un pedido nuevo). Los valores se guardan como texto en un formato invariable (`120.5`, `2026-10-17`, `true`), sin depender del idioma del servidor, y el conjunto se serializa a JSON en la columna `Changes` (`jsonb`). La web traduce los nombres de campo y los valores para mostrarlos ("Precio: 120 → 135").

---

21. Catálogo de errores

21.1 Los tres orígenes de un error

Origen	Cuándo	Forma de la respuesta
Validación de atributos	El cuerpo o la *query* no cumplen `[Required]`, `[StringLength]`, `[IdentityCard]`…	400 `ValidationProblemDetails`, con `errors` por campo y sin `code`
Regla de negocio	El servicio devuelve `Result.Fail(error)`	`ProblemDetails` con `title` (mensaje en español) y `code` estable
Excepción inesperada	Un error de programación, la base caída…	500 `ProblemDetails` con `traceId`, sin detalles internos

Más un cuarto, de infraestructura: el límite de peticiones responde 429 con `code: "rate_limited"` (17.5).

21.2 De `ErrorType` a código HTTP

Cada `Error` tiene un **código** estable, un **mensaje** para el usuario y un **tipo**. `ResultExtensions` traduce el tipo:

```
private static int StatusFor(ErrorType type) =>
    type switch
    {
        ErrorType.Validation => StatusCodes.Status400BadRequest,
        ErrorType.Unauthorized => StatusCodes.Status401Unauthorized,
        ErrorType.Forbidden => StatusCodes.Status403Forbidden,
        ErrorType.NotFound => StatusCodes.Status404NotFound,
        ErrorType.Conflict => StatusCodes.Status409Conflict,
        ErrorType.TooManyRequests => StatusCodes.Status429TooManyRequests,
        _ => StatusCodes.Status500InternalServerError,
    };
```

La diferencia entre 400 y 409 es útil para razonar: **400** significa "lo que enviaste está mal" (un rango de fechas al revés, un código que no son 6 dígitos); **409**, "lo que enviaste está bien, pero choca con el estado actual" (no hay stock, el pedido ya se envió, alguien guardó antes que tú).

Una respuesta de error de negocio tiene esta forma:

```
{
  "title": "No hay stock suficiente de Silla Roble Nórdico.",
  "status": 409,
  "code": "checkout.insufficient_stock",
  "traceId": "0HNPCAPND4L02:00000001"
}
```

La web decide por `code`, nunca por el texto: el texto puede cambiar de redacción sin romper nada, y el código no.

21.3 Todos los códigos

Autenticación

Código	HTTP	Cuándo
`auth.email_exists`	409	Registro con un correo ya usado

Código	HTTP	Cuándo
`auth.registration_failed`	400	Identity rechazó la cuenta (por ejemplo, contraseña inválida)
`auth.invalid_credentials`	401	Correo o contraseña incorrectos, o la cuenta no existe
`auth.email_not_confirmed`	401	Contraseña correcta, correo sin confirmar
`auth.locked_out`	429	Cuenta bloqueada por intentos fallidos
`auth.account_disabled`	403	Cuenta desactivada por un administrador
`auth.code_invalid`	400	Código de verificación incorrecto
`auth.code_expired`	400	Código vencido, inexistente o de una cuenta que no existe
`auth.code_attempts_exceeded`	429	Quinto fallo con el mismo código
`auth.confirmation_failed`	500	No se pudo guardar la confirmación
`auth.invalid_refresh_token`	401	Cualquier fallo al renovar (16.4)
`auth.refresh_token_expired`	401	El refresh token pasó sus 30 días

Tienda

Código	HTTP	Cuándo
`product.not_found`	404	Producto inexistente
`category.not_found`	404	Categoría inexistente
`cart.client_not_found`	404	La cuenta no tiene cliente asociado
`cart.product_not_found`	400	Se agrega un producto que no existe
`cart.product_unavailable`	409	Se agrega un producto archivado
`cart.insufficient_stock`	409	Se pide más cantidad que el stock
`cart.item_not_found`	404	Se cambia o quita una línea que no está en el carrito
`client.self_not_found`	404	La cuenta no tiene cliente asociado
`client.identity_card_taken`	409	El carnet ya pertenece a otra cuenta (también en el registro)
`client.version_conflict`	409	El perfil cambió mientras se editaba
`checkout.client_not_found`	404	La cuenta no tiene cliente asociado
`checkout.empty_cart`	400	Confirmar con el carrito vacío
`checkout.profile_incomplete`	400	Falta teléfono, carnet o dirección
`checkout.product_unavailable`	409	Un producto del carrito se archivó; el mensaje lo nombra
`checkout.insufficient_stock`	409	No queda stock suficiente; el mensaje nombra el producto
`checkout.price_changed`	409	El total cambió desde que el cliente lo vio
`order.not_found`	404	Pedido inexistente o de otra persona
`order.invalid_transition`	409	Cambio de estado no permitido

Panel

Código	HTTP	Cuándo
`admin.version_conflict`	409	Otro administrador guardó antes (14.6)
`query.invalid_sort`	400	Ordenación por un campo no permitido
`product.category_not_found`	400	La categoría elegida no existe
`product.referenced_by_order`	409	Borrar un producto que ya se vendió
`product.image_type`	400	La imagen no es JPG, PNG ni WebP
`product.image_size`	400	La imagen pasa de 5 MB
`product.image_upload_failed`	500	No se pudo guardar la imagen
`category.name_taken`	409	Ya hay una categoría con ese nombre
`category.has_products`	409	Borrar una categoría con productos
`customer.not_found`	404	Cliente inexistente
`customer.self_action`	409	Desactivarse, quitarse el rol o borrarse a uno mismo
`customer.last_admin`	409	Quitar el rol al último administrador
`customer.is_admin`	409	Desactivar o borrar a un administrador
`customer.has_orders`	409	Borrar un cliente con pedidos
`customer.unconfirmed`	409	Dar el rol a una cuenta sin confirmar
`customer.disabled`	409	Dar el rol o desbloquear una cuenta desactivada
`customer.already_admin`, `customer.not_admin`	409	El rol ya está, o no está
`customer.already_disabled`, `customer.not_disabled`	409	La cuenta ya está, o no está, desactivada
`customer.not_locked`	409	Desbloquear una cuenta que no está bloqueada
`report.invalid_range`	400	La fecha inicial es posterior a la final
`report.range_too_long`	400	El rango supera los 3 años

21.4 Los errores inesperados

`Middleware/GlobalExceptionHandler.cs` atrapa cualquier excepción que nadie manejó:

```
logger.LogError(
    ApiEvents.UnhandledException,
    exception,
    "Unhandled exception on {Method} {Path} [{TraceId}]",
    context.Request.Method,
    context.Request.Path,
    context.TraceIdentifier
);

var problem = new ProblemDetails
{
    Status = StatusCodes.Status500InternalServerError,
    Title = "Ocurrió un error inesperado.",
    Instance = $"{context.Request.Method} {context.Request.Path}",
```

```
};
problem.Extensions["traceId"] = context.TraceIdentifier;
```

- **Al cliente:** un mensaje genérico y el `traceId`. Nunca la traza de la excepción, que revelaría nombres de tablas, rutas del servidor o versiones de bibliotecas.
- **Al log:** todo, con el mismo `traceId`. Si alguien reporta un error, ese identificador lleva directamente a su línea del log.
- **Una excepción no es un error.** Si el cliente cerró la conexión (`OperationCanceledException` con la petición abortada), no se registra nada: nadie va a leer la respuesta.

---

22. Integraciones externas

22.1 Correo: Gmail por SMTP

Ver 18.5. Requiere `SmtpSettings__UserName` y `SmtpSettings__Password` (una contraseña de aplicación de Gmail, no la de la cuenta). Si faltan, la API arranca con un aviso en el log y el registro responde `emailSent: false`. Sin conexión, `Email:LogOnly` escribe los códigos en la consola (18.6).

22.2 Imágenes: la carpeta `assets/`

Desde el 16 de octubre las imágenes de productos se guardan en la carpeta `assets/` de la raíz del repositorio, para que la aplicación funcione sin conexión:

1. `LocalImageStorage.UploadAsync` escribe el archivo con un nombre aleatorio (`{Guid}.png`) y devuelve su URL: `http://localhost:5135/assets/673efe….png`. El nombre aleatorio evita choques y que alguien adivine o sobrescriba imágenes de otros productos.
2. `Program.cs` sirve esa carpeta como archivos estáticos:

   ```
   var assetsFolder = LocalImageStorage.Folder(app.Environment);
   Directory.CreateDirectory(assetsFolder);
   app.UseStaticFiles(new StaticFileOptions
   {
       FileProvider = new PhysicalFileProvider(assetsFolder),
       RequestPath = LocalImageStorage.RequestPath,
   });
   ```

   `UseStaticFiles` va antes de `UseRouting`: una petición a `/assets/…` se resuelve leyendo el disco, sin pasar por controladores, autenticación ni límites. Las imágenes del catálogo son públicas.
3. La web muestra la imagen con `next/image`, que solo acepta las direcciones permitidas en `next.config.ts` (las del `API_BASE_URL`).

`SupabaseImageStorage` y `SupabaseStorageSettings` siguen en el código, sin uso: volver a Supabase sería cambiar una línea del registro (6.4).

22.3 PostgreSQL

Capítulos 11 a 14. La API solo necesita `DATABASE_URL`. En desarrollo, la base corre en Docker (`docker compose up -d` en la raíz del repositorio, puerto 5433).

22.4 El *health check*

```
builder.Services.AddHealthChecks()
    .AddDbContextCheck<APIFurnistoreContext>("database", tags: ["db"]);
…
app.MapHealthChecks("/health");
```

`GET /health` responde `Healthy` si la API está viva **y** puede consultar la base, y `Unhealthy` (503) si no. Lo usan los orquestadores y balanceadores para saber si deben mandar tráfico a esta instancia, y sirve a cualquiera para comprobar de un vistazo que todo está levantado.

Parte VI. Calidad y operación

23. Pruebas

23.1 Concepto: qué probar y contra qué

Una **prueba automática** es código que ejecuta otro código y comprueba que el resultado es el esperado. Se clasifican por cuánto del sistema involucran:

Tipo	Qué ejecuta	Ventaja	Inconveniente
Unitaria	Una clase aislada, con sus dependencias simuladas	Rapidísima	No prueba el SQL ni la base de datos
De integración	Varias capas juntas, con una base de datos real	Prueba lo que de verdad se ejecuta	Más lenta
De extremo a extremo	El sistema completo, desde un navegador	La más realista	Lenta y frágil

Furnistore apuesta por las **de integración contra un PostgreSQL real**. La mayor parte de su lógica delicada vive en la base: transacciones, bloqueos, `UPDATE … WHERE Stock >= cantidad`, índices únicos, restricciones `CHECK`. Una prueba unitaria con la base simulada daría por buena una consulta que en PostgreSQL falla, y no podría probar que dos checkouts simultáneos no venden la última unidad dos veces.

23.2 Las herramientas

El proyecto `API.Furnistore.IntegrationTests` usa:

Paquete	Para qué
xUnit	El marco de pruebas: `[Fact]` marca una prueba, `[Theory]` + `[InlineData]` una prueba con varios juegos de datos, y `Assert.*` las comprobaciones.
Testcontainers	Arranca un contenedor Docker de PostgreSQL al empezar y lo destruye al terminar. Cada ejecución empieza con una base vacía.
Microsoft.AspNetCore.Mvc.Testing	Levanta la API completa en memoria (`WebApplicationFactory`), para probar por HTTP.
coverlet	Mide qué líneas cubren las pruebas.

23.3 La base de datos de las pruebas: `PostgresFixture`

```
public sealed class PostgresFixture : IAsyncLifetime
{
    private readonly PostgreSqlContainer container = new PostgreSqlBuilder("postgres:16-
alpine").Build();
    …
    public async Task InitializeAsync()
    {
        await container.StartAsync();

        services = BuildServices(container.GetConnectionString(), Images);
```

```
        await using var db = CreateContext();
        await db.Database.MigrateAsync();
    }
}
```

1. **`IAsyncLifetime`** permite código asíncrono al preparar (`InitializeAsync`) y al limpiar (`DisposeAsync`).
2. **El contenedor** arranca un PostgreSQL de verdad, con su propio puerto aleatorio.
3. **`MigrateAsync()`** aplica las 22 migraciones. De paso, eso prueba que el historial de migraciones funciona desde una base vacía.
4. **`BuildServices`** monta un contenedor de dependencias igual al de la API, pero solo con lo necesario: el `DbContext`, Identity y los servicios. La diferencia: en lugar de guardar imágenes en disco, registra `FakeImageStorage`.

Cada clase de prueba recibe el *fixture* con `IClassFixture<PostgresFixture>`: xUnit crea **un contenedor por clase** y lo comparte entre sus pruebas. Como varias pruebas usan la misma base, cada una crea sus propios datos con valores únicos (correos con un `Guid`, carnets con un contador), y así no se pisan.

23.4 Cómo es una prueba

`TestData` reúne los ayudantes para preparar datos (`CreateClientAsync`, `CreateProductAsync`, `PlaceOrderAsync`...), y `fixture.RunAsync` ejecuta un servicio en su propio *scope*, como lo haría una petición:

```
[Fact]
public async Task A_product_that_was_ordered_cannot_be_deleted_only_archived()
{
    var (userId, clientId) = await data.CreateClientAsync();
    var product = await data.CreateProductAsync("Taburete vendido", 20m, stock: 5);
    await data.PlaceOrderAsync(userId, clientId, product, 1, 20m);

    var detail = await Products(service => service.GetByIdAsync(product, CancellationToken.None));
    Assert.True(detail.Value.HasOrders);

    var deleted = await fixture.RunAsync<AdminProductService, Result>(service =>
        service.DeleteAsync(product, "admin-1", CancellationToken.None)
    );
    Assert.Equal("product.referenced_by_order", deleted.Error!.Code);
}
```

Tres partes: **preparar** los datos, **actuar** llamando al servicio y **comprobar** el resultado. Los nombres de las pruebas son frases que describen la regla, y la lista de pruebas se lee como una especificación del negocio.

Dobles de prueba. Lo que habla con el exterior se sustituye por una versión controlable:

Doble	Sustituye a	Permite
`FakeImageStorage`	`LocalImageStorage`	Ver qué se subió y simular un fallo (`Fail = true`)
`Outbox`	`SmtpEmailSender`	Leer el código que se habría enviado (`LastCode`) y simular un fallo de Gmail
`TestClock`	`TimeProvider.System`	Avanzar el reloj 16 minutos para probar que un código vence, sin esperar

`TestClock` es la razón de que los servicios pidan la hora a `TimeProvider` en lugar de llamar a `DateTime.UtcNow` (4.3).

Pruebas por HTTP. `AdminAuthorizationTests` levanta la API entera con `WebApplicationFactory<Program>` (por eso `Program.cs` termina con `public partial class Program;`), fabrica JWT firmados con roles distintos y recorre todos los endpoints del panel (17.2).

Pruebas de concurrencia. Lanzan varias operaciones a la vez con `Task.WhenAll` y comprueban que el resultado es coherente:

Prueba	Comprueba
`Concurrent_checkouts_of_the_same_cart_create_exactly_one_order`	El bloqueo por cliente (14.4)
`Concurrent_clients_never_oversell_the_last_unit`	La reserva atómica de stock (14.4)
`Two_admins_preparing_at_once_produce_a_single_transition`	`SELECT … FOR UPDATE` (14.5)
`Simultaneous_revocations_always_leave_one_admin`	El bloqueo global de roles (14.7)
`Parallel_guesses_never_get_more_than_five_attempts`	La reserva atómica de intentos (18.3)

23.5 Qué está cubierto

72 casos en 66 métodos (un `[Theory]` del carnet prueba siete números distintos):

Archivo	Pruebas	Qué cubre
`AuthFlowTests`	16	Registro, código, login, bloqueo, renovación, logout
`CheckoutTests`	11	Checkout feliz, todos sus rechazos y la concurrencia
`AdminBackofficeTests`	10	Productos, categorías, pedidos y clientes en el panel
`ReportTests`	9	Los seis reportes, rangos y exportaciones
`OrderLifecycleTests`	7	Estados, cancelaciones y pedidos ajenos
`CatalogAndCartTests`	4	Catálogo público y carrito
`ProfileAndImageTests`	3 (uno con 7 casos)	Carnet, perfil y subida de imágenes
`AdminAuthorizationTests`	2	Que todo el panel exige `Admin`
`AdminRoleTests`	2	Reglas de roles
`AdminRoleConcurrencyTests`	1	Revocaciones simultáneas
`ReshapeDomainModelMigrationTests`	1	Que la migración que transforma datos convierte las filas antiguas y se puede revertir

23.6 Ejecutarlas

Hace falta Docker en marcha (Testcontainers lo usa para crear la base):

```
cd apps/api
dotnet test API.Furnistore.IntegrationTests/API.Furnistore.IntegrationTests.csproj
```

Para ejecutar solo una clase o una prueba:

```
dotnet test API.Furnistore.IntegrationTests/API.Furnistore.IntegrationTests.csproj --filter
"FullyQualifiedName~CheckoutTests"
```

La primera vez descarga la imagen `postgres:16-alpine`. Todo el conjunto tarda menos de un minuto.

---

24. Contrato OpenAPI

24.1 Concepto

OpenAPI (antes Swagger) es un formato estándar, en JSON, que describe una API: sus rutas, qué recibe cada una, qué devuelve y con qué códigos. A partir de esa descripción se pueden generar documentación navegable, clientes y tipos para otros lenguajes.

24.2 En Furnistore: de los controladores al TypeScript

El contrato hace de puente entre las dos aplicaciones, en tres pasos:

```
Controladores C# ──(Swashbuckle)──► apps/api/openapi.json ──(openapi-typescript)──► apps/web/src/lib/
api/schema.d.ts
```

1. Describir. `Extensions/ApiContractExtensions.cs` configura **Swashbuckle**, que lee los controladores y sus atributos:

```
services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new OpenApiInfo { Title = "furnistore_API", Version = "v1" });
    c.SupportNonNullableReferenceTypes();
    c.UseAllOfToExtendReferenceSchemas();
    …
});
```

- `SupportNonNullableReferenceTypes` hace que un `string` de C# sea obligatorio en el contrato, y un `string?`, opcional. Así la nulabilidad de C# llega intacta a TypeScript.
- `[ProducesResponseType]` en cada endpoint indica qué tipo devuelve cada código (19.1).
- Los atributos de validación también pasan: `[StringLength(60, MinimumLength = 2)]` se convierte en `minLength` y `maxLength`.
- Los `enum` se escriben como texto (`"Paid"`, no `0`), gracias a `JsonStringEnumConverter` (8.8).

En Development, la API sirve además la documentación navegable en `/swagger`, donde se puede probar cada endpoint con un JWT pegado en "Authorize".

2. Generar el archivo. Desde `apps/api`:

```
npm run openapi
```

Compila en Release y ejecuta `dotnet swagger tofile`, la herramienta de Swashbuckle declarada en `.config/dotnet-tools.json`. Esa herramienta busca en el ensamblado una clase llamada `SwaggerHostFactory` y, si existe, la usa para construir la aplicación en lugar de ejecutar `Program.cs`:

```
public static IHost CreateHost()
{
    var builder = WebApplication.CreateBuilder();
    builder.Services.AddApiContract().AddApplicationPart(typeof(SwaggerHostFactory).Assembly);
```

```
    return builder.Build();
}
```

Es un *host* mínimo, con solo los controladores y Swashbuckle. Generar el contrato no necesita base de datos, secretos ni conexión.

3. Generar los tipos. Desde `apps/web`:

```
npm run gen:api
```

`openapi-typescript` convierte `openapi.json` en `schema.d.ts`, y la web usa esos tipos en sus llamadas (`ApiSchemas["RegisterRequest"]`).

24.3 Por qué así

- **Una sola fuente de verdad.** Los contratos se escriben una vez, en C#. Si se añade un campo obligatorio a `RegisterRequest`, tras regenerar, TypeScript marca en la web todos los sitios que no lo envían. Sin el contrato, el error aparecería en producción como un 400.
- **El contrato está en el repositorio.** `openapi.json` y `schema.d.ts` se suben con el código, así que un cambio de contrato se ve en el *diff* del mismo commit que lo provoca.
- **El coste:** hay que acordarse de regenerar después de cambiar un contrato. Si se olvida, la web compila con tipos viejos; el síntoma típico es un campo que la API exige y la web no envía.

---

25. Recetas para modificar el proyecto

25.1 Comandos de uso diario

Desde `apps/api`:

Tarea	Comando
Levantar la base	`docker compose up -d` (en la raíz del repositorio)
Arrancar la API	`dotnet run --project API.Furnistore.API`
Arrancar con recarga automática	`npm run dev` (`dotnet watch`)
Compilar	`dotnet build API.sln`
Pruebas	`dotnet test API.Furnistore.IntegrationTests/API.Furnistore.IntegrationTests.csproj`
Crear una migración	`dotnet ef migrations add Nombre --project API.Furnistore.Data --startup-project API.Furnistore.API -- --environment Development`
Aplicar migraciones	`dotnet ef database update --project API.Furnistore.Data --startup-project API.Furnistore.API -- --environment Development`
Regenerar el contrato	`npm run openapi` y, en `apps/web`, `npm run gen:api`
Hacer administrador a alguien	`dotnet run --project API.Furnistore.API -- admin promote correo@ejemplo.com`

25.2 Añadir un endpoint

1. **El contrato** (`Application/<Área>/<Área>Contracts.cs`): un `record` para la petición, con sus atributos de validación, y otro para la respuesta. Nunca la entidad (8.3).
2. **El servicio**: un método que devuelve `Result<T>`, con `Error.Validation`, `Error.Conflict`… para los fallos esperados, y con un código nuevo con el formato `área.motivo` si hace falta (capítulo 21).
3. **El controlador**: un método fino que saca el usuario con `User.UserId()`, llama al servicio y devuelve `ToActionResult(this)`. Con `[ProducesResponseType]` para cada código posible.
4. **Si es del panel**, el controlador hereda de `AdminControllerBase` y la ruta empieza por `api/admin/`. Las pruebas de autorización lo comprobarán solas.
5. **Si cambia algo**, el servicio registra la acción con `audit.Record(…)` antes de `SaveChangesAsync`, con su constante en `AuditActions`.
6. **Una prueba** en la clase correspondiente: el caso feliz y cada rechazo.
7. **Regenerar el contrato** (24.2).

25.3 Añadir un campo a una tabla

1. La propiedad en la entidad (`Shared/<Entidad>.cs`). Si puede faltar en las filas existentes, debe ser anulable (`string?`) o tener un valor por defecto.
2. Sus reglas en la configuración (`Data/Configurations/<Entidad>Configuration.cs`): longitud, índice, `CHECK`.
3. `dotnet ef migrations add AddNombreDelCampo` y **leer** la migración generada antes de aplicarla.
4. `dotnet ef database update`.
5. Añadirlo a los contratos y servicios que lo usen, y regenerar el contrato.

25.4 Añadir una regla de negocio

1. Si es de formato (longitud, patrón), un atributo en el contrato: no llega ni al servicio.
2. Si depende de los datos, una comprobación en el servicio que devuelva `Result.Fail(Error.…("área.motivo", "Mensaje para el usuario."))`.
3. Si es crítica, además una restricción en la base (`CHECK`, índice único), para que se cumpla aunque el código tenga un error (8.4).
4. Una prueba con el nombre de la regla.
5. En la web, si hace falta, un mensaje propio para ese código.

25.5 Añadir un reporte

Los pasos de la API y de la web están juntos en la receta 22.5 de la guía de la web.

---

26. Deuda conocida y glosario

26.1 Deuda conocida

Detalles encontrados al escribir esta guía. Ninguno rompe nada; son mejoras pendientes:

- **El JWT no se puede cortar antes de que caduque** (16.7). Desactivar una cuenta o quitarle el rol tarda hasta una hora en tener efecto completo.

- **`OrderService.CheckoutAsync` usa `DateTime.UtcNow`** en lugar del `TimeProvider` inyectado (4.3), así que la fecha de un pedido no se puede controlar en las pruebas. La fecha estimada de entrega también se calcula sobre la fecha UTC y no sobre la de La Habana.
- **Código de Supabase sin uso**: `SupabaseImageStorage`, `SupabaseStorageSettings` y el aviso de arranque de Storage sin configurar (3.3) quedaron de cuando las imágenes se subían allí.
- **El dígito de control del carnet no se valida** porque no se conoce la fórmula oficial (`IdentityCardAttribute`).
- **Comentarios heredados** en `Program.cs` y `AuthService`, de las primeras versiones, con faltas de ortografía. No afectan al comportamiento.
- **La clave del JWT firma también los códigos de verificación** (18.2). Funciona, pero si se cambiara `JWT_SECRET`, los códigos pendientes dejarían de valer. Una clave propia para los códigos sería más limpia.

26.2 Glosario

Término	Significado
ASP.NET Core	El marco web de .NET: servidor, *middleware*, controladores, autenticación.
Claim	Un dato dentro de un JWT: el id del usuario, su correo, su rol.
Contenedor (de dependencias)	El objeto de .NET que construye los servicios y sus dependencias (capítulo 4).
Contenedor (Docker)	Un proceso aislado con su propio sistema de archivos; aquí, el PostgreSQL.
Controlador	Clase que recibe peticiones HTTP y las delega en un servicio.
DbContext	La puerta de EF Core a la base de datos: consultas, cambios y transacciones.
DTO / contrato	Clase que define la forma de una petición o respuesta, distinta de la entidad.
EF Core	Entity Framework Core, el ORM de .NET (capítulo 11).
Endpoint	Una ruta con su método HTTP: `POST /api/orders/checkout`.
Entidad	Clase que representa una tabla: `Product`, `Order`.
Fixture	En las pruebas, la preparación compartida (aquí, la base de datos).
Hash	Resultado de una función de un solo sentido; se guarda en lugar de contraseñas y tokens.
HMAC	Hash que además usa una clave secreta: sin la clave no se puede calcular ni falsificar.
Identity	La biblioteca de cuentas, contraseñas y roles de ASP.NET Core (capítulo 15).
Idempotente	Que repetirla da el mismo resultado: cerrar sesión dos veces, cancelar dos veces.
JWT	*JSON Web Token*: token firmado que demuestra quién hace la petición (capítulo 16).
Kestrel	El servidor HTTP que viene con ASP.NET Core.
LINQ	Las consultas integradas en C# (`Where`, `Select`…), que EF Core traduce a SQL.
Middleware	Pieza del *pipeline* que procesa cada petición antes y después de los controladores.

Término	Significado
Migración	Cambio versionado del esquema de la base (capítulo 13).
NuGet	El gestor de paquetes de .NET.
OpenAPI	Formato estándar para describir una API (capítulo 24).
ORM	*Object-Relational Mapper*: traduce entre objetos y tablas.
ProblemDetails	Formato estándar (RFC 9457) de las respuestas de error.
Rate limiting	Límite de peticiones por cliente y tiempo (17.5).
Refresh token	Token de larga duración que solo sirve para pedir un JWT nuevo.
Result	El tipo que devuelven los servicios: éxito con un valor, o un `Error` (8.1).
Rol	Permiso con nombre (`Admin`, `User`) que se asigna a una cuenta.
Scoped / Singleton / Transient	Cuánto vive un servicio en el contenedor (4.2).
Swashbuckle	La biblioteca que genera el contrato OpenAPI a partir de los controladores.
Testcontainers	Biblioteca que arranca contenedores Docker desde las pruebas.
Transacción	Grupo de cambios que se guardan todos o ninguno (capítulo 14).
xmin	Columna de sistema de PostgreSQL que cambia con cada actualización; sirve de versión (14.6).
xUnit	El marco de pruebas de .NET que usa el proyecto.

Fin de la guía.